

Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky



Návrh a testování prostředí pro AI agenty v softwarovém vývoji

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Autor: Thanh An Nguyen

Vedoucí práce: Ing. Jiří Korčák

Praha, květen 2026

Prohlášení

Prohlašuji, že jsem bakalářskou práci zpracoval samostatně a že jsem uvedl všechny použité prameny a literaturu, ze kterých jsem čerpal.

Potvrzuji, že při přípravě předložené práce **byly použity** nástroje umělé inteligence. Konkrétně byly použity těmito způsoby:

- úprava a rozšíření textových částí,
- vyhledávání a shrnutí literatury,
- generování částí zdrojových kódů.

Každé použití je dokumentováno v příloze D.

Potvrzuji, že:

- má práce je originální a byla zpracována v souladu s etickými standardy, zejména s Etickým kodexem Vysoké školy ekonomické v Praze; že jsou v práci citovány všechny použité informační zdroje a že generovaný obsah byl mnou ověřen;
- přijímám plnou odpovědnost za celý obsah závěrečné práce.

V Praze dne 27. března 2026

Thanh An Nguyen

Poděkování

Poděkování.

Abstrakt

AI coding agenti dokáží generovat funkční kód, ale bez vhodných instrukcí nedodržují softwarově-inženýrské praktiky. Existující benchmarky měří pouze výsledek (vyřešil agent úkol?), nikoli kvalitu procesu ani kódu. Formalizovaný postup pro návrh a vyhodnocování instrukcí dosud chybí. Práce si klade tři cíle: (1) navrhnout sadu metrik měřících proces, kvalitu a efektivitu práce agenta; (2) demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami; (3) ablacemi identifikovat, které složky instrukční sady jsou nezbytné. Sadu metrik a postup ověřujeme na případové studii systému upomínek faktur. Agent (model Minimax prostřednictvím nástroje OpenCode) prošel pěti pilotními iteracemi a čtyřmi ablačními běhy. Chování bylo měřeno 19 metrikami ve třech kategoriích: procesní (P), produktové (Q) a efektivita (E). Metriky zachytily zlepšení ze 4/10 na 9/10 splněných kritérií během tří iterací. Analýza odhalila vzorec vývoje instrukcí: obecné pravidlo → konkrétní příkaz → verifikační krok. Ablace ukázaly, že verifikační kroky nejsou redundantní — jejich odebrání zhoršilo čtyři metriky. Sekce kódových konvencí byla pro automatizované metriky převážně redundantní. Nedeterminismus modelu se ukázal jako dominantní faktor variability procesní compliance. Přenositelným přínosem jsou sada metrik a iterativní postup; konkrétní naměřené hodnoty jsou vázány na podmínky případové studie.

Klíčová slova

AI coding agent, AGENTS.md, metriky kvality software, iterativní návrh, ablační studie

Abstract

AI coding agents can generate functional code, but without proper instructions they fail to follow software engineering practices. Existing benchmarks measure only the outcome (did the agent solve the task?), not process quality or code quality. A formalized process for designing and evaluating instructions is still lacking. This thesis pursues three goals: (1) design a metric suite measuring agent process, quality, and efficiency; (2) demonstrate an iterative instruction design process driven by these metrics; (3) use ablation to identify which instruction components are necessary. The metric suite and process are evaluated on a single case study of a dunning system. The agent (Minimax model via the OpenCode tool) underwent five pilot iterations and four ablation runs. Behavior was measured using 19 metrics in three categories: process (P), product (Q), and efficiency (E). The metrics captured improvement from 4/10 to 9/10 satisfied criteria over three iterations. Analysis revealed an instruction evolution pattern:

general rule → specific command → verification step. Ablation showed that verification steps are not redundant—removing them degraded four metrics. The code convention section proved largely redundant for automated metrics. Model non-determinism emerged as the dominant factor in process compliance variability. The transferable contributions are the metric suite and the iterative process; the specific measured values are bound to the case study conditions.

Keywords

AI coding agent, AGENTS.md, software quality metrics, iterative design, ablation study

Obsah

Úvod	11
1 Vymezení problému a cílů práce	12
1.1 Motivace	12
1.2 Cíle práce	14
1.3 Rozsah práce	15
2 Teoretická východiska	17
2.1 Softwarové inženýrství a životní cyklus	17
2.1.1 Definice a komplexita software	17
2.1.2 Fáze životního cyklu	19
2.1.3 Modely a metodiky	19
2.1.4 Měření kvality software	20
2.2 AI coding agenti a scaffolding	22
2.2.1 Základní pojmy	23
2.2.2 Typy coding agentů	24
2.2.3 Jak agenti mění SDLC	24
2.2.4 Problém kontextu	25
2.2.5 Přístupy k memory a kontextu	25
2.2.6 Struktura instrukcí	26
2.3 Evaluace agentů a instrukcí	28
2.3.1 Hodnocení agentů a benchmarky	28
2.3.2 Evaluace instrukcí	29
3 Metodika	35
3.1 Výzkumný přístup	35
3.2 Sada metrik	36
3.2.1 Procesní metriky (P1–P8)	36
3.2.2 Produktové metriky (Q1–Q8)	38
3.2.3 Metriky efektivity (E1–E3)	39
3.2.4 Aplikace LLM-as-judge	40
3.3 Experimentální design	41
3.3.1 Výběr projektu	41
3.3.2 Fixní proměnné	42
3.3.3 Iterativní cyklus	43
3.3.4 Pilotní fáze	47
3.3.5 Komparativní variace	47
3.4 Přehledová tabulka	48
3.5 Omezení a validita	48

4 Praktická část	51
4.1 Příprava experimentu	51
4.1.1 Konstrukce specifikace	51
4.1.2 Referenční implementace	53
4.1.3 Konstrukce baseline instrukcí	54
4.2 Pilotní iterace	57
4.2.1 Pilot-r1: baseline	57
4.2.2 Pilot-r2	59
4.2.3 Pilot-r3	61
4.2.4 Pilot-r4	64
4.2.5 Pilot-r5: verifikace kontraktu	65
4.3 Komparativní variace	69
4.3.1 Výběr složek pro ablaci	69
4.3.2 Ablace A: bez verifikačních kroků	71
4.3.3 Ablace B: bez Package Quality	73
4.3.4 Závěr komparativní fáze	74
4.4 Souhrnné výsledky	75
5 Vyhodnocení a diskuse	78
5.1 Interpretace výsledků	78
5.1.1 Cíl 1: Sada metrik	79
5.1.2 Cíl 2: Iterativní postup	84
5.1.3 Cíl 3: Ablace	85
5.2 Porovnání s literaturou	85
5.3 Limity ve světle výsledků	88
5.4 Doporučení pro praxi	89
5.5 Náměty pro další výzkum	91
Závěr	94
Bibliography	95
A Formulář v plném znění	104
B Zdrojové kódy výpočetních procedur	105
C Baseline AGENTS.md	106
D Využití nástrojů umělé inteligence	107

Seznam obrázků

4.1	Baseline <code>AGENTS.md</code> (pilot-r1) — kompletní znění instrukcí použitých jako výchozí bod pilotních iterací.	55
4.2	Vizuální diff změn <code>AGENTS.md</code> mezi pilot-r1 a pilot-r2	59
4.3	Vizuální diff změn <code>AGENTS.md</code> mezi pilot-r2 a pilot-r3	61
4.4	Verze <code>AGENTS.md</code> po pilot-r3 — kompletní znění po iteracích $r1 \rightarrow r2 \rightarrow r3$	62
4.5	Vizuální diff změn <code>AGENTS.md</code> mezi pilot-r3 a pilot-r4	63
4.6	Vizuální diff změn <code>AGENTS.md</code> mezi pilot-r3 a pilot-r5	65
4.7	Souhrnná evoluce <code>AGENTS.md</code> : pilot-r1 $\rightarrow$ pilot-r3. Zelené řádky = přidáno, červené = odebráno.	68
4.8	Změny <code>AGENTS.md</code> pro ablaci A	70
4.9	Změny <code>AGENTS.md</code> pro ablaci B: odebrána celá sekce Package Quality (konvence, strict TypeScript, JSDoc, čisté API).	70
5.1	Míra splnění jednotlivých metrik napříč devíti běhy. Procesní metriky P2 a P5 byly splněny nejméně často (3/9), zatímco Q1 a Q6 zůstaly stabilní ve všech bžích. Přerušovaná čára odděluje procesní a produktové metriky.	80
5.2	Cena běhu vs. počet splněných deterministických kritérií	81

Seznam tabulek

3.1	Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ	48
4.1	Mapování sekcí baseline AGENTS.md na metriky	56
4.2	Pilot-r1: naměřené metriky	57
4.3	Pilot-r2: metriky s změnou oproti r1	59
4.4	Pilot-r3: metriky s změnou oproti r2	61
4.5	Pilot-r4: metriky s změnou oproti r3	64
4.6	Pilot-r5: metriky s změnou oproti r3	66
4.7	Ablace A: srovnání s r3 (bez verifikačních kroků)	72
4.8	Ablace B: srovnání s r3 (bez sekce Package Quality)	73
4.9	Souhrnné výsledky všech běhů (pilot + ablance)	76

Seznam použitých zkratk

E1	vstup/výstup/cache tokeny	P8	PR description quality
E2	trvání	Q1	API contract match
E3	kompakce kontextu	Q2	referenční test pass rate
P1	issues before code	Q3	mutation score
P2	branch per issue	Q4	AC coverage
P3	test-first commits	Q5	lint warnings
P4	PRs linked to issues	Q6	typecheck errors
P5	no existing test modifications	Q7	složitost kódu
P6	commit message quality	Q8	design quality
P7	issue description quality		

Úvod

[DRAFT]

AI agenti se stávají součástí softwarového vývoje: dokáží implementovat funkcionalitu, psát testy, spravovat verzovací historii a komunikovat prostřednictvím issues a pull requestů. Kvalita jejich výstupu však závisí na instrukcích, které dostanou — na obsahu souboru `AGENTS.md` definujícího pracovní postup, omezení a kontext projektu. Jak tyto instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované praktiky, zůstává otevřenou otázkou.

Tato práce navrhuje sadu 19 metrik ve třech kategoriích — procesní (procesní metriky (P1–P8)), produktové (produktové metriky (Q1–Q8)) a metriky efektivity (E1–E3) — které měří nejen výsledek, ale i proces a kvalitu práce agenta. Na případové studii systému upomínek faktur demonstruje iterativní postup návrhu instrukcí s využitím těchto metrik a komparativními variacemi (ablacemi) identifikuje, které složky instrukcí přispívají k měřenému chování agenta. Sadu metrik a postup ověřujeme na případové studii jednoho projektu (R. K. Yin, 2018).

Práce je členěna do pěti kapitol. Kapitola 1 vymezuje problém, formuluje tři cíle práce a ohraničuje její rozsah. Kapitola 2 shrnuje teoretická východiska: softwarové inženýrství a životní cyklus vývoje, AI coding agenty, scaffolding a instrukce a měření kvality softwaru. Kapitola 3 popisuje metodiku — výzkumný přístup, sadu metrik, experimentální design a omezení validity. Kapitola 4 zachycuje průběh případové studie: pilotní fázi pěti iterací a komparativní variace. Kapitola 5 interpretuje výsledky ve vztahu k cílům práce, porovnává je s existujícím výzkumem a diskutuje limity.

1. Vymezení problému a cílů práce

1.1 Motivace

[DRAFT]

AI agenti se používají k vývoji softwaru, ale výsledky ukazují že to není tak jednoduché jak se čekalo. Randomizovaná studie METR (METR, 2025) na 246 úlohách ukázala, že s AI nástroji byli vývojáři o 19 % pomalejší. Agent často produkuje funkční výstup, nedodržuje přitom vývojové praktiky: slučuje nesouvisející změny, přeskakuje testy, ignoruje specifikaci.

Otázka není jen jestli agent úkol vyřešil, ale jak. Existující benchmarky jako SWE-bench (Jimenez et al., 2024) měří pouze výsledek: vyřešil agent úkol, ano nebo ne. Kvalitu práce, transparentnost procesu a možnost řídit chování agenta v průběhu vývoje tyto benchmarky nepostihují. Agent může hlásit úkol jako dokončený, přestože implementace pokrývá jen část požadavků, a bez metrik procesu to není jak systematicky odhalit.

Chování agenta lze přizpůsobit trénováním modelu (fine-tuning) nebo instrukcemi v kontextovém okně (in-context learning). Shin et al. (Shin et al., 2025) ukazují, že automatizované prompty systematicky nepřekonávají fine-tuned modely, ale iterativní zpřesňování instrukcí výrazně zlepšuje výsledky na úlohách s kódem. S rostoucí velikostí kontextových oken se instrukce stávají praktičtější alternativou: jsou dostupné komukoliv, iterativně upravitelné a neomezují emergentní schopnosti modelu. Pro trénování modelů existují zavedené postupy; pro systematické navrhování a vyhodnocování instrukcí obdobný postup dosud nevznikl.

Tato práce navrhuje evaluační systém který měří nejen výsledek, ale i proces a kvalitu práce agenta, a popisuje postup jak s jeho pomocí iterativně vylepšovat instrukce.

[RAW]

Předchozí verze motivace (audit trail):

Velké jazykové modely (LLM) dnes umí generovat kód, ale to neznamená, že umí vyvíjet software. Randomizovaná studie METR (METR, 2025) na 16 zkušených vývojářích a 246 úlohách ukázala, že s AI nástroji byli vývojáři o 19 % pomalejší — přestože sami odhadovali zrychlení o 24 %. Když projekt roste, bývá těžší jej rozšiřovat — jak pro člověka, tak pro LLM. Vývojáři přicházejí o kontext a hlubokou znalost kódové základny, zatímco agenti jsou limitováni omezeným context window. Otázka tedy není, co modely umí, ale jak je zasadit do vývojového procesu tak, aby produkovaly kvalitní výstup.

Ukazuje se, že odpověď leží spíš v instrukcích než v samotných modelech. Breunig (Breunig, 2025) zjistil, že změna promptu mění chování agenta výrazněji než výměna modelu. Lulla et al. (Lulla et al., 2026) dokládají, že přítomnost instrukčního souboru (AGENTS.md) je spojena se zkrácením mediánové doby běhu o 28,6 % a snížením výstupních tokenů o 16,6 %.

Scaffolding — struktura instrukcí a procesních pravidel, která agenta provází vývojem — má na výsledek zásadní vliv. Ale nikdo systematicky nezkoumal, *ktelé* složky tohoto scaffoldingu jsou pro dodržování softwarově-inženýrských praktik nezbytné a které jsou zbytečné.

Tato práce zkoumá právě to: jak musí být strukturovány instrukce pro AI coding agenta, aby dodržoval specifikací řízený vývoj, test-driven development a granulární správu verzí — a co z těch instrukcí je skutečně kritické.

Původní poznámky:

Studie společnosti METR.ORG ukazuje, že LLM zkušené vývojáře spíše zpomaluje. S rychlým vývojem schopností modelů se situace pravděpodobně mění. Ale to neznamená, že je LLM samo o sobě dostatečné k vypracování dlouhotrvajících úkolů. Není to problém pouze LLM, když projekt roste, bývá těžší jej rozšiřovat jak pro člověka, tak i pro LLM. AI programování tenhle problém ještě více prohlubuje. Vývojáři přicházejí o kontext a hlubokou znalost kódové základny (codebase), zatímco velké jazykové modely (LLM) jsou limitovány omezenou pamětí (context window). Jak nastavit harness/scaffolding tak, aby v tom mohli fungovat agenti a lidé to stále měli pod kontrolou?

[RAW]

Review poznámky (2026-03-21):

(a) **Motivace nemotivuje cíl 3 (ablace):** Motivace vysvětluje proč potřebujeme metriky (cíle 1) a iterativní postup (cíle 2), ale neříká proč potřebujeme zjišťovat které složky instrukcí jsou nezbytné. Chybí věta typu: “Zůstává otevřená otázka, které složky instrukcí jsou pro dodržování praktik skutečně nezbytné a které jsou redundantní.” Starší verze motivace (audit trail, řádek 39) to obsahuje (“nikdo systematicky nezkoumal, *ktelé* složky”), ale aktuální DRAFT ne.

(b) **Řádek 27 — silný claim bez citace:** “pro systematické navrhování a vyhodnocování instrukcí obdobný postup dosud nevznikl” — buď citovat systematic review ukazující gap, nebo zeslabit na “je méně formalizován” s odkazem na kap02.

(c) **Řádek 56 — “exit kritéria” bez vysvětlení:** Čtenář narazí na pojem poprvé v cíli 2, ale definice je až v kap03. Buď přidat závorku “(měřitelné prahy pro úspěšnost běhu, viz sekce 3.3.4)”, nebo přeformulovat na “stanovených požadavků”.

(d) **Terminologie “evaluační systém” vs “sada metrik”:** Řádek 29 říká “evaluační systém”, cíle 1 říká “sada metrik”. Jsou to totéž? Pokud evaluační systém = sada metrik + postup + exit kritéria, definovat explicitně. Pokud totéž, sjednotit.

TODO (2026-03-26) — po přeformulaci cílů (DRAFT v6, PoC framing):

- **Motivace nemotivuje cíle 3:** přidat větu o tom, že zůstává otevřená otázka které složky instrukcí jsou nezbytné (viz bod (a) výše).

- **Rozsah:** přidat větu o feasibility demonstration — práce ukazuje že přístup funguje na jednom případě, ne že platí obecně. Sladit s PoC framingem cílů.
- **Rozsah ř. 152:** “evaluační systém” sjednotit s cíli — cíl 1 říká “sadu metrik”, cíl 2 říká “iterativní postup”. Buď definovat “evaluační systém = metriky + postup” nebo nahradit konkrétními pojmy.
- **Cíl 2:** “exit kritéria” zmizela z cíle (teď říká “měřitelné zlepšení”), takže bod (c) výše je vyřešen.

1.2 Cíle práce

[DRAFT]

1. Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývající proces, kvalitu kódu a efektivitu — dimenze, které stávající benchmarky neměří.
2. Na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a ukázat, že vede k měřitelnému zlepšení dodržování vývojových praktik.
3. Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

[RAW]

Audit trail — předchozí verze cílů (DRAFT v5):

1. Navrhnout sadu metrik která měří proces a kvalitu práce AI agenta, ne jen výsledek.
2. Iterativním postupem navrhnout instrukce které dovedou agenta k dodržování stanovených exit kritérií.
3. Ablacemi identifikovat, které složky instrukcí přispívají k měřenému chování agenta a které jsou redundantní.

Audit trail — předchozí verze cílů (DRAFT v4):

1. Navrhnout sadu metrik která měří proces a kvalitu práce AI agenta, ne jen výsledek.
2. Na případové studii demonstrovat iterativní postup pro návrh instrukcí s využitím těchto metrik.
3. Popsat pozorované tendence v chování agenta a vliv jednotlivých složek instrukcí na tyto tendence.

Poznámky ze schůzky s vedoucím (2026-03-16): Cíle práce = 3 fáze: (1) research gap — procesní metriky, (2) pilotní fáze — docílit exit kritérií, (3) komparativní fáze — ablace/substituce. Research question potřebuje přesná kritéria a kvantifikaci. **Pozn.:** substituce se nakonec realizovala implicitně v pilotní fázi (r1→r3), komparativní fáze obsahuje jen ablaci.

Předchozí verze cílů (audit trail v3):

1. Navrhnout instrukce které dovedou AI coding agenta k dodržování strukturovaného vývojového postupu na případové studii (systém upomínek faktur).
2. Navrhnout sadu metrik která měří proces a kvalitu práce agenta, ne jen výsledek.
3. Navrhnout a demonstrovat iterativní postup pro systematický návrh a vyhodnocování instrukcí.

Předchozí verze cílů (audit trail v2):

1. Popsat, jak se řízení softwarových projektů mění v kontextu AI agentů, a zmapovat co víme o vlivu instrukcí na jejich chování.
2. Iterativně navrhnout instrukce, které dovedou AI coding agenta k dodržování vývojového postupu na case study projektu (systém upomínek faktur).
3. Zjistit, které složky těchto instrukcí jsou pro dodržování vývojového postupu nezbytné, které jsou zbytečné, a jaký kontext agent potřebuje k tomu, aby vytvářel a využíval projektové artefakty.

Původní verze:

1. Popsat jak se řízení SWE projektů mění v kontextu agentních systémů (teoretický rámec)
2. Navrhnout a implementovat experimentální prostředí (case study: systém upomínek faktur)
3. Prozkoumat vliv různých nastavení scaffoldingu na schopnost agenta provést kvalitní práci
4. Identifikovat jaký kontext je pro agenty klíčový a jak instrukce/procesy ovlivňují schopnost agenta tento kontext vytvářet a využívat

1.3 Rozsah práce

[DRAFT]

Evaluační systém a postup pro návrh instrukcí nelze navrhnout čistě teoreticky. Je potřeba je ověřit v praxi: ukázat, že metriky skutečně zachytí rozdíly v chování agenta, když dostane jiné instrukce, a že iterativní postup vede ke zlepšení. K tomu slouží případová studie, ve které opakovaně pouštíme agenta s různými variantami instrukcí a pokaždé měříme, co se změnilo.

Aby bylo možné vliv instrukcí měřit, musí být všechno ostatní stejné: model, nástroje, specifikace i výchozí stav repozitáře. Jedinou proměnnou jsou instrukce které agent dostane. Kdyby se měnilo víc věcí najednou, nebylo by jasné, co způsobilo změnu v chování.

Případová studie probíhá na jednom projektu: systému upomínek faktur. Jde o projekt s deterministickou logikou, kde stejný vstup vždy dá stejný výstup. To je důležité, protože

výsledky lze ověřit automatizovanými testy, ne subjektivním posouzením. Agent pracuje od specifikace k implementaci, tedy od požadavků přes návrh po funkční kód a testy.

Práce neporovnává různé modely ani programovací jazyky a neporovnává agenta s lidským vývojářem. Navržený evaluační systém a iterativní postup jsou koncipovány obecně: kdokoli je může vzít a použít na svém projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Zdůvodnění volby projektu a podrobnosti experimentálního designu popisuje kapitola 3.

[RAW]

Poznámky k propojení:

- Řízení vyžaduje holistický pohled (vidět celek, ne jen část) - proto celý SDLC, ne jedna fáze
- Billing Reminder Engine jako case study: malý projekt, ale reálné nuance (state machine, business days, edge cases)
- Deterministická logika (stejný vstup = stejný výstup) + jasná pravidla správnosti = objektivně testovatelné
- Hraniční případy (víkendy, svátky, grace periods) = místa kde lze testovat kvalitu agentní práce

TODO: Vysvětlit termíny:

- state machine - ?
- edge cases / hraniční případy - ?
- SDLC - ?

Scope SDLC (k diskuzi):

- Primární plán: celý SDLC včetně deployment/maintenance
- Fallback: zúžit na implementation + testing (hlavní doména coding agents)
- Poznámka: i impl + testing má feedback loop (implementace → testy → chyba → úprava) = stále systémový pohled

2. Teoretická východiska

[RAW]

Tato kapitola vysvětluje teoretická východiska práce. Nejprve je popsáno softwarové inženýrství, životní cyklus vývoje a měření kvality software. Následují AI coding agenti jako noví aktéři v tomto procesu a scaffolding s instrukcemi které jejich chování řídí. Nakonec je zařazena evaluace agentů a instrukcí.

2.1 Softwarové inženýrství a životní cyklus

[RAW]

Úvod sekce: Sloučení starých 2.1 + 2.2. Základ pro zbytek kapitoly: co je SWE, proč je SW složitý, jak se vyvíjí.

2.1.1 Definice a komplexita software

[RAW]

Předchozí obsah z 2.1.1 (Definice):

Softwarové inženýrství je disciplína, která se zabývá celým životním cyklem software – od specifikace až po údržbu (IEEE Computer Society, 2024, s. xxxvii).

Na rozdíl od programování, které se soustředí na implementaci a technické aspekty jako algoritmy a datové struktury, softwarové inženýrství přistupuje k vývoji software holisticky – zahrnuje nejen technickou stránku, ale i organizační aspekty jako řízení projektů a rozpočty (Sommerville, 2016, s. 21).

Základní koncepty SWE:

K řešení těchto otázek SWE využívá tři klíčové koncepty:

1. Abstrakce

→ colburn2000:

Skrývání detailů, práce na vyšší úrovni bez znalosti implementace.

→ swebok2024 s. 370 (Dijkstra):

“The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise.”

2. Modularita (information hiding)

→ [parnas1972](#):

Rozdělení systému na nezávislé části s jasným rozhraním. Každý modul skrývá rozhodnutí která se mohou změnit.

3. Architektura

→ [perry1992](#):

“Software architecture is a set of architectural elements that have a particular form.”

→ [garlan1993](#):

High-level struktura systému – komponenty, konektory, konfigurace.

Předchozí obsah z 2.1.3 (Komplexita):

Brooks – No Silver Bullet:

- **Essential complexity** – inherentní složitost problému
→ [brooks1987](#) s. 6:
“The complexity of software is an essential property, not an accidental one.”
 - Business logika, requirements, doménová znalost
 - Nelze odstranit – je to podstata toho co řešíme
- **Accidental complexity** – složitost kterou si přidáváme
 - Nástroje, technologie, jazyky, frameworky
 - Lze redukovat lepšími nástroji a abstrakcemi
- **Vlastnosti software** – proč je jiný než fyzické systémy:
→ [brooks1987](#) s. 6:
“Software entities are more complex for their size than perhaps any other human construct, because no two parts are alike.”
 - Neviditelný, snadno měnitelný, nemá fyzická omezení
 - **Nelineární růst complexity:**
→ [brooks1987](#) s. 6:
“A scaling-up of a software entity is not merely a repetition of the same elements in larger size, it is necessarily an increase in the number of different elements.”
- **Evoluce a růst complexity v čase** (Lehman’s Laws):
→ [lehman1980](#) s. 9:
 - Software musí být neustále adaptován (zákon I)
 - Komplexita roste s časem pokud se aktivně neredukuje (zákon II)

Propojení: Abstrakce řeší accidental complexity – ale essential zůstává. To motivuje potřebu metrik (sekce 2.1.4) a strukturovaných postupů.

2.1.2 Fáze životního cyklu

[RAW]

Předchozí obsah z 2.2.1:

Sommerville – 4 základní aktivity:

→ sommerville2016 s. 24:

“A software process is a sequence of activities that leads to the production of a software product. Four fundamental activities are common to all software processes:”

1. **Software specification** – zákazníci a inženýři definují co se má vytvořit
2. **Software development** – software se navrhuje a implementuje
3. **Software validation** – ověřuje se že software dělá co zákazník požaduje
4. **Software evolution** – software se mění podle měnících se požadavků

→ sommerville2016 s. 55:

“The the four basic process activities of specification, development, validation, and evolution are organized differently in different development processes.”

Klíčový bod: Tyto 4 aktivity jsou abstrakce – každá metodika je organizuje jinak (viz 2.1.3).

2.1.3 Modely a metodiky

[RAW]

Předchozí obsah z 2.2.2:

Klasifikace metodik (Boehm & Turner 2003):

→ boehm2003balancing:

Metodiky nejsou binární volba, ale **spektrum** mezi plan-driven a agile.

5 dimenzí pro klasifikaci: Size, Criticality, Dynamism, Personnel, Culture.

→ boehm2002agile:

“Both agile and plan-driven approaches have their home grounds where each clearly works best.”

Plan-driven metodiky:

1. **Waterfall (sekvenční):**

→ royce1970 – primární zdroj

→ boehm1988 s. 3:

“The waterfall model’s basic scheme has encountered some fundamental difficulties.”

2. Spiral Model (risk-driven):

→ boehm1988 – kombinuje iterativní vývoj s analýzou rizik

Agile metodiky:

→ agilemanifesto2001 – 4 hodnoty, 12 principů

3. Extreme Programming (XP):

→ beck2000 – pair programming, TDD, continuous integration

4. Scrum:

→ scrumguide2020: sprinty, role (PO, SM, Developers)

5. Spec-Driven Development (SDD):

→ sdd2026 – Emerging metodika pro éru AI coding agentů:

Specifikace je source of truth, kód je odvozený.

Tři úrovně rigidity:

1. **Spec-first** – specifikace před kódem
2. **Spec-anchored** – specifikace žije vedle kódu, testy vynucují sync
3. **Spec-as-source** – specifikace JE kód

Klíčový bod: S příchodem AI agentů se přidává nová dimenze: SDD spec-first umožňuje sekvenční fáze v rámci malých incrementů, ale iterativní mezi incrementy.

2.1.4 Měření kvality software

[RAW]

Sekce 2.1.1–2.1.3 popsaly obor a životní cyklus. Zbývá otázka: jak výsledky hodnotit? Tato sekce poskytuje teoretický základ pro sadu metrik navrženou v kapitole 3. Popisujeme CO a PROČ měřit; konkrétní volba metrik, nástrojů a exit kritérií je v kap03.

Klasifikace softwarových metrik

[RAW]

Fenton & Bieman (Fenton & Bieman, 2014): tři kategorie měřitelných entit v SE:

- **Proces** — aktivity během vývoje (review, testování, integrace). Příklady metrik: defect injection rate, schedule variance.
- **Produkt** — výstupy vývoje (kód, specifikace, testy). Příklady: LOC, cyklomatická složitost, defect density.
- **Zdroje** — lidé, nástroje, výpočetní kapacita. Příklady: team size, cost, tool usage.

Proč měřit proces i produkt, ne jen výsledek: proces ovlivňuje kvalitu produktu, ale není jím determinován. Agent může dodat fungující kód chaotickým procesem — nebo dodržet proces a dodat nefunkční kód.

Interní vs externí atributy: interní měříme přímo z entity (LOC, složitost), externí vyžadují pozorování v kontextu (spolehlivost, udržitelnost).

V kontextu AI agentů: Yin et al. (Z. Yin et al., 2025) nezávisle dospěli ke třem dimenzím hodnocení: effectiveness (produkt), efficiency (proces) a overhead (zdroje).

Kap03 na to navazuje: P (proces), Q (kvalita produktu), E (efektivita) — naše pracovní kódy pro Fentonovy kategorie.

Testování a mutation testing

[RAW]

Strukturální coverage vs mutation testing: Strukturální coverage (line, branch) měří kolik kódu testy *projdou*, ale ne zda *detekují* chyby. Mutation testing zavádí drobné změny (mutanty) do kódu a měří kolik jich testy odhalí. Papadakis et al. (Papadakis et al., 2019): přehledová studie, 36 % chyb odhalitelných pouze mutation testingem. Harman et al. (Harman et al., 2025): produkční nasazení (Meta), 70 % mutantů neodhalených i při plném coverage.

Test oracle problem: Kdo říká co je správný výsledek? Mathews et al. (Mathews & Nagappan, 2024): nástroje pro automatické generování testů systematicky filtrují failing testy — až 68,1 % test suites validuje chybné chování místo jeho odhalení. Chen et al. (Chen et al., 2025): na 500 úlohách SWE-bench agent-generované testy slouží jako observační feedback, ne validační nástroj.

Obrana: TDD ze specifikace — expected values odvozené z acceptance criteria, ne z pozorování kódu.

Kap03 na to navazuje: Q2 (referenční testy ze spec), Q3 (mutation score), Q4 (AC coverage).

Statická analýza a složitost kódu

[RAW]

Cyklomatická složitost: McCabe (McCabe, 1976): počet lineárně nezávislých cest grafem řízení toku. Práh 10 jako de facto standard udržitelnosti. Vyšší složitost koreluje s vyšší chybovostí a nižší testovatelností.

Linting: Automatizovaná kontrola kódových konvencí a potenciálních chyb. Fixní konfigurace zajišťuje konzistentní měření napříč běhy.

Typová kontrola: TypeScript strict mode jako statická prevence celé kategorie runtime chyb. Počet explicitních `any` jako proxy pro obcházení typového systému.

Kap03 na to navazuje: Q1 (API contract via tsc), Q5 (lint), Q6 (typecheck), Q7 (cyclomatická složitost).

LLM-as-judge

[RAW]

Metoda: Zheng et al. (Zheng et al., 2023) (MT-Bench): použití LLM jako automatizovaného hodnotitele. Tři systematické biasy: position bias (pořadí odpovědí), verbosity bias (delší = lepší), self-enhancement bias (preference vlastních výstupů).

Self-preference mechanismus: Panickssery et al. (Panickssery et al., 2024) (NeurIPS): kauzální vztah self-recognition → self-preference. Model rozpozná vlastní výstupy (nižší perplexita) a systematicky je hodnotí lépe.

Mitigace: Verga et al. (Verga et al., 2024) (PoLL): panel diverzních modelů lepší než single judge. Implikace: judge z jiné modelové rodiny než agent eliminuje self-preference.

Kap03 na to navazuje: Kapitola 3 z této teorie přebírá praktickou aplikaci LLM-as-judge: volbu modelu z odlišné modelové rodiny, třístupňovou škálu a fixní rubriky. V literatuře se dále řeší i validace shody s lidským hodnocením, ta však není hlavním předmětem této práce.

2.2 AI coding agenti a scaffolding

[RAW]

Úvod sekce: Definuje základní pojmy (LLM, agent, tool calls) a popisuje jak AI agenti mění softwarové inženýrství. Navazuje na problém tacit knowledge a popisuje přístupy k scaffoldingu a instrukci agentů.

2.2.1 Základní pojmy

[RAW]

1. Large Language Model (LLM):

→ [vaswani2017](#) – Transformer architektura:

“*Attention Is All You Need*” – self-attention mechanismus.

LLM = velký jazykový model založený na transformer architektuře.

2. LLM-based Agent:

→ [liu2024llmagents](#):

Agent = LLM rozšířený o schopnost interagovat s prostředím.

4 klíčové komponenty:

- **Planning** – dekompozice komplexních úkolů na pod-úkoly
 - **Memory** – krátkodobá (kontext) a dlouhodobá (RAG, databáze)
 - **Perception** – vnímání prostředí (čtení souborů, výstupů)
 - **Action** – provádění akcí (tool calls, zápis souborů)
-

3. Tool Calls (Function Calling):

→ [yao2022react](#) – ReAct framework:

Cyklus: Thought → Action → Observation.

→ [schick2023toolformer](#):

LLM se může naučit kdy a jak použít nástroje.

4. Prompt Engineering:

[DOPLNIT] Definovat prompt engineering jako disciplínu formulování instrukcí pro LLM.

Důležité rozlišení pro BP:

- **Prompt engineering** = jak formuluješ konkrétní instrukci
 - **Context engineering** = co všechno agent dostane jako kontext (viz sekce 2.2.6)
 - **Scaffolding** = struktury v prostředí které agenta vedou
-

2.2.2 Typy coding agentů

[RAW]

Evoluce podle autonomie:

→ guo2025benchmarks:

Tři paradigmatata s rostoucí autonomií:

1. **Prompt-based** – člověk instruuje, LLM odpovídá (ChatGPT, copilot)
2. **Fine-tune-based** – model adaptován na SE doménu (CodeLlama, StarCoder)
3. **Agent-based** – autonomní plánování, akce, učení (Devin, Claude Code, OpenHands)

Copilot vs Autonomous Agent:

- **Copilot** – asistuje člověku, návrhy, autocomplete
- **Autonomous agent** – samostatně plánuje a vykonává úkoly

Příklady nástrojů (2024-2025): Devin, Claude Code, Cursor, OpenHands, Agentless (Xia et al., 2024).

2.2.3 Jak agenti mění SDLC

[RAW]

Které fáze agenti ovlivňují:

→ jin2024 – 6 klíčových oblastí: Requirement Engineering, Code Generation, Autonomous Decision-making, Software Design, Test Generation, Software Maintenance.

Co zůstává stejné: Fáze, artefakty, quality gates, potřeba jasných requirements.

Co se mění:

- **Rychlost** – minuty místo hodin/dnů
- **Explicitnost** – vše musí být napsané (agent nemá tacit knowledge)
- **Role** – developer se stává reviewerem a specifikátorem
- **Batch size** – menší úkoly, častější iterace

Micro-waterfall hypotéza: Sekvenční kroky zůstávají – jen se zmenšuje batch size a

zrychluje feedback loop. SDD formalizuje tento pattern (Piskala, 2026).

Empirická podpora:

→ `watanabe2025agentprs` (ACM TOSEM): 567 Claude Code PRs, 83.8% acceptance rate, 45% potřebovalo revizi.

→ `ehsani2026failedprs` (MSR 2026): 33k agent PRs. Malé, focused úkoly mají vyšší úspěšnost.

2.2.4 Problém kontextu

[RAW]

Předchozí obsah:

Agent nemá tacit knowledge → potřebuje explicitní kontext.

→ `nonaka1995`:

Articulation = převod tacit knowledge na explicit.

→ `hassan2025sase` – SASE framework:

SE for Humans (SE4H) vs SE for Agents (SE4A).

Co agent potřebuje vědět:

- **Requirements** – co má být výsledkem
- **Architektura** – kde se změna má udělat
- **Konvence** – coding standards, patterns, gotchas
- **Historie** – proč je kód takový jaký je (rationale)

2.2.5 Přístupy k memory a kontextu

[RAW]

Předchozí obsah:

1. Memory systémy pro agenty:

→ `memorymechanism2024`, `amem2025`

Kategorie: Short-term, Long-term, Parametric, Non-parametric.

2. Context Engineering:

→ [contextengsurvey2025](#), [ace2025](#)

3. Strukturované artefakty:

→ [hassan2025sase](#) – SASE: BriefingScript, MentorScript, LoopScript.

4. Repository-level context:

→ [contextmodule2024](#), [bugfixcontext2025](#)

5. Git-based přístupy:

→ [gcc2025](#) – Git Context Controller

6. Specifikace jako vstup (SWE-bench): GitHub Issues jako standardní formát specifikace pro agenty. Traceability: Issue → branch → commits → PR → merge (Gotel & Finkelstein, 1994).

7. Co dělá specifikaci efektivní pro LLM:

→ [rope2024](#): kvalita requirements = kvalita výstupu

→ [ullrich2025](#): requirements příliš abstraktní pro přímý LLM vstup

→ [yang2025underspec](#): pod-specifikované prompty 2× náchylnější k regresi

→ [specine2025](#): 10 alignment rules, Tier 1: příklady, účel, výstupní požadavky

→ [ticoder2024](#): testy jako součást specifikace (+45.97% Pass@1)

→ [domaincodegen2024](#): doménový kontext

→ [newcomb2025prepost](#): pre/postconditions

→ [wen2024io](#): I/O specifikace

→ [clarifygpt2024](#): klarifikace ambiguity

→ [reprompt2025](#): RE metodologie na prompty

→ [swebenchpro2025](#): strukturované requirements v benchmarcích

Syntéza tiers: Tier 1 (nejvyšší dopad): Description/účel, AC s příklady, výstupní specifikace. Tier 2: vstupní specifikace, doménový slovník, edge cases. Tier 3: pre/postconditions, error handling, behavioral model.

2.2.6 Struktura instrukcí

[RAW]

Předchozí obsah (přejmenováno z “Praktické techniky”):

1. Agent Harness:

Anthropic — “Effective harnesses for long-running agents”: init.sh, progress tracking, git commits jako checkpointy.

2. Meta-prompt soubory:

→ hassan2025sase – AGENT.md pattern:

CLAUDE.md, .clinerules, AGENT.md. “Employee handbook” pro AI.

3. Living Documents: Version-controlled, iterativní refinement, auditable history.

4. Prompt template komponenty (Mao et al.):

→ mao2025fse – 7 komponent prompt šablony, empirické pořadí efektivity.

5. Script balance (Hassan et al.):

→ hassan2025sase – BriefingScript / LoopScript / MentorScript balance.

6. Content effectiveness (Lulla et al.):

→ lulla2026 – AGENTS.md přítomnost: −28,6 % mediánový runtime, −16,6 % výstupní tokeny. Neizoluje efekt jednotlivých typů obsahu.

7. Prompt sensitivity:

→ razavi2025, breunig2025 – přestrukturování > opakování. Malá změna formulace mění chování víc než výměna modelu.

→ shin2025prompt – explicitní instrukce s kontextem výrazně zlepšují výkon agenta.

Formát specifikace: GitHub Issues

Specifikace jako GitHub Issues. Volba vychází z:

1. Akademický standard – SWE-bench (Jimenez et al., 2024)
2. Agilní RE praxe (Cao & Ramesh, 2008)
3. Open source praxe (Scacchi, 2002)
4. Nativní čitelnost pro agenty (GitHub API/CLI)
5. Traceability (Gotel & Finkelstein, 1994)

Dvě vrstvy specifikace:

1. **Requirements** (CO): Title, Description, AC (Given/When/Then), Domain glossary

2. **Architecture** (JAK): Type definitions, Invariants, Behavioral model, Constraints

Zasazení do SASE frameworku: Specifikace = BriefingScript. AGENTS.md = LoopScript + MentorScript.

Human-AI Collaboration:

→ `humanai2024`: AI zlepšuje efektivitu, human oversight zůstává kritický.

RAW TODO (2026-03-24): Mechanismy účinku instrukcí a emergentní chování.

Při psaní kap02 doplnit sekci o tom, jak instrukce působí na chování agenta. Pilotní data (kap04) a diskuze (kap05) naznačují dva mechanismy: (1) *vynucení* — konkrétní verifikační kroky přímo kontrolují výstup, (2) *aktivace* — obecné konvence (nápovědy, enabling constraints) připomínají modelu latentní znalosti z tréninku. Klíčový poznatek: nejslabší instrukce (hint/nápověda) a nejsilnější (verifikační krok) fungují, ale prostřední (pravidlo) ne — to připomíná fenomén chain-of-thought (Wei et al. 2022).

Relevantní literatura:

- Enabling constraints (Juarrero 2023, Cynefin/Snowden 2007) — omezení která umožňují emergentní chování
- Expertise reversal effect (Kalyuga et al. 2003) — instrukce pro nováčka škodí expertovi
- Instruction specificity (Kim et al. 2025 DETAIL, Zi et al. 2025) — specifická zlepšuje výsledky ale přílišný detail omezuje reasoning
- Chain-of-thought (Wei et al. 2022) — minimální hint aktivuje latentní schopnost
- In-context learning (Min et al. 2022) — demonstrace fungují jako strukturální nápovědy, ne jako příkazy

2.3 **Evaluace agentů a instrukcí**

[RAW]

Úvod sekce: Předchozí sekce popsaly co agenti jsou a jak je řídit. Zbývá otázka hodnocení: jak měřit výkon agenta a jak vyhodnocovat kvalitu instrukcí samotných?

2.3.1 **Hodnocení agentů a benchmarky**

[RAW]

NOVÁ SEKCE — přesunuto z různých míst + doplnit.

SWE-bench:

→ **swebench2024** (ICLR 2024): De facto benchmark pro AI coding agenty. 2294 úloh z reálných GitHub repozitářů. Měří: vyřešil agent úkol, ano nebo ne (binary pass/fail).

Další benchmarky:

- SWE-bench Pro (Scale AI, 2025) – rozšíření o explicitní Requirements a Interface sekce
- SWE-EVO (Y. Wei et al., 2025) – evoluční úlohy, 21% úspěšnost
- FeatureBench („FeatureBench: Benchmarking Agentic Coding for Complex Feature Development“, 2026) – feature-level, 11%
- ACE-Bench („ACE-Bench: End-to-End Feature Development Benchmark“, 2025) – end-to-end, 7.5%

Multi-issue gap: SWE-EVO 21%, FeatureBench 11%, ACE-Bench 7.5%. Naše case study (5 issues, celý dunning system) cílí do tohoto rozsahu.

Co benchmarky měří a co NE:

- Měří: funkční korektnost (pass/fail)
- Neměří: proces (jak agent pracoval), kvalita kódu (mutation score, lint, složitost), procesní záznamy (commit messages, issue descriptions)

Gap: Existující benchmarky jsou navrženy pro “vyřešil agent bug?”, ne “vyvinul agent software správně a správným postupem?” Tento gap adresuje naše práce (kap01).

TODO (2026-03-16): Explicitně zdůraznit že literatura nemá nic co by měřilo procesní stránku práce agenta — benchmarky měří výsledek (pass/fail), ale ne *jak* agent pracoval (dodržování workflow, kvalita commit messages, TDD, branch management). Toto je klíčový research gap který motivuje cíl 1 v kap01. Propojit s Fenton & Bieman procesní metriky — existují v tradičním SWE, ale nikdo je neaplikoval na AI agenty systematicky.

2.3.2 Evaluace instrukcí

[RAW]

Co tato sekce musí říct (aby validovala kap03 metodiku):

1. Problém: instrukce pro agenty se liší od jednoduchých NL promptů. Jsou strukturované dokumenty (sekce, workflow, constraints). Jedna iterace = desítky minut + tisíce tokenů — nelze iterovat rychle. Cílem není maximalizovat accuracy na test setu, ale řídit chování agenta přes více dimenzí (P/Q/E).

2. APO přístupy ((Agarwal et al., 2024), („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025)): Score → Critique → Synthesize → opakuj. Fungují pro jednoduché NL tasky. Předpoklady které pro nás neplatí: (a) atomický prompt, ne

strukturovaný dokument; (b) rychlé iterace (API call), ne full agent run; (c) jeden cíl (accuracy), ne P/Q/E; (d) fix = vygeneruj novou verzi — my potřebujeme vědět PROČ selhalo.

3. Zhu et al. (Zhu et al., 2026): instrukce jsou nezávislá proměnná. Výsledky agenta nelze interpretovat bez kontroly prostředí. Potvrzuje naše fixní proměnné (empty repo, stejná spec, stejný model, build.md).

4. Gap: Neexistuje zavedená metodika pro iterativní, theory-grounded návrh instrukcí pro coding agenty. Existuje: praktický hill climbing (Cline blog, +10pp bez akademického popisu), APO pro jednoduché tasky. Chybí: systematický postup s diagnostikou příčin a citacemi z literatury.

→ Odtud plyne, proč v kap03 děláme Diagnózu manuálně (FSE/SASE/Lulla/Breunig), a ne automaticky.

TODO (2026-03-25) — pojmenovat náš přístup vůči APO v DRAFT textu: Naše metodika = human-in-the-loop iterativní optimalizace instrukcí. APO (Zhou et al. 2022, (Agarwal et al., 2024)) = automatická smyčka (score→critique→generate). Klíčové rozdíly proč APO nestačí pro náš kontext viz bod 2 výše. Zvážit: zmínit že průmysl (MiniMax M2.7, DSPy) dělá plnou automatizaci — náš přístup je manuální předstupeň který explicitně dokumentuje kauzalitu. Citace: Zhou et al. ICLR 2023 (APE, arXiv 2211.01910) — stáhnout a přidat do sources.

Fine-tuning vs in-context learning:

Dva přístupy jak přizpůsobit LLM konkrétnímu úkolu:

- **Fine-tuning** — trénování parametrů modelu. Permanentní, drahé.
- **In-context learning (ICL)** — instrukce v kontextovém okně. Ephemeral, levné.

Shin et al. (Shin et al., 2025) ukazují, že ani jeden přístup systematicky nedominuje, ale explicitní instrukce s kontextem výrazně zlepšují výkon agenta na úlohách s kódem.

Prompt evaluation nástroje: Langfuse, Braintrust, Promptfoo — měří kvalitu odpovědí LLM, ale nehodnotí proces coding agenta.

Cline hill climbing: Praktický příklad iterativního zlepšování instrukcí (47% → 57% na Terminal Bench). Totožný přístup s naším, ale bez akademického popisu a měření procesu.

Empirická evidence: fungují context files?

→ gloaguen2025agentsmd — Evaluating AGENTS.md (ETH Zurich): LLM-generované context files snižují úspěšnost, developer-written jen marginálně zlepšují. Instrukce agent

následuje, ale to nezlepšuje výsledky. Redundantní s dokumentací.

→ **skillsbench2025** — SkillsBench: Curated Skills +16,2pp. Self-generated nulový efekt. 2–3 fokusované moduly optimální.

Syntéza: Zbytečný kontext škodí, fokusovaný pomáhá.

[RAW]

Audit trail — zrušené sekce při restrukturalizaci (2026-03-01): Kompletní původní obsah zachován pro případ potřeby.

Narativní flow (revidovaný):

1. **Složitost systémů roste** – software je čím dál komplexnější

→ **sommerville2016** s. 582:

“The root cause of these problems is, as it was in the 1960s, that we are trying to build systems that are larger and more complex than before. We are attempting to build these ‘mega-systems’ using methods and technology that were never designed for this purpose.”

2. **Abstrakce jako nástroj** – reakce na složitost:

- Assembler → C → Java → frameworky
- Každá vrstva skrývá detaily (information hiding)

→ **colburn2000** s. 1:

“Abstraction through information hiding is a primary factor in computer science progress and success through an examination of the ubiquitous role of information hiding in programming languages, operating systems, network architecture, and design patterns.”

- Dijkstra: abstrakce pomáhá být přesný, ne vágní

→ **swebok2024** s. 370

3. **Přesto 1968 krize** – proč?

- Složitost rostla rychleji než naše schopnost ji zvládat
- NATO konference: “software crisis”

→ **nato1968** s. 78 (Perlis keynote):

“I believe it is because we recognize that a practical problem of considerable difficulty and importance has arisen: The successful design, production and maintenance of useful software systems.”

4. **Reakce: vznik SWE** – disciplína pro řízení složitosti

→ **sommerville2016** s. 592:

“In software engineering, we have seen the incredibly rapid development of the discipline to help manage the increasing size and complexity of software systems... the approach that has been the basis of complexity management in software engineering is called reductionism.”

5. **Dnes: AI jako nový problém**

- Není to jen další vrstva abstrakce
- Je to ztráta determinismu – “black box”

- Proto mechanistic interpretability

→ bereska2024 s. 1:

“Understanding AI systems’ inner workings is critical for ensuring value alignment and safety.”

Klíčový insight: Abstrakce $\neq$ složitost. Abstrakce je NÁSTROJ pro zvládání složitosti. AI přináší kvalitativně nový problém (nedeterminismus), ne jen “více abstrakce”.

1. SWE je fundamentálně týmová disciplína

Brooks’s Law – komunikační overhead:

→ brooks1975 s. 25:

“Adding manpower to a late software project makes it later.”

→ mcconnell2004 s. 713:

“Merely increasing the number of people increases the complexity and amount of project communication.”

Komunikační kanály rostou: $n(n - 1)/2$

2. Role a zodpovědnosti

Tradiční role: Developer, Tester/QA, Architekt, Project Manager, Product Owner.

Role v Scrum:

→ scrumguide2020: Product Owner, Scrum Master, Developers.

3. Conway’s Law – organizace $\leftrightarrow$ architektura

→ conway1968:

“Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.”

4. Koordinace práce

→ malone1994:

Coordination = managing dependencies between activities.

Typy závislostí: shared resources, producer-consumer, simultaneity constraints.

Tacit vs explicit knowledge:

→ nonaka1995:

“There are two types of knowledge: explicit knowledge, contained in manuals and procedures, and tacit knowledge, learned only by experience, and communicated only indirectly.”

- **Tacit knowledge** – v hlavách lidí, těžko artikulovatelné
- **Explicit knowledge** – dokumentace, procesy, kód
- Konverze tacit → explicit = klíčový proces (externalization)

Quality gates pro agenty:

- Agent přebírá některé role (developer, částečně tester)
- Quality gates zůstávají – kdo je teď dělá?
- Implicitní znalosti z meetingů → explicitní instrukce pro agenta

1. Artefakty podle fází:

- **Specification** → Requirements, user stories, use cases, behavioral models
Spektrum formátů: Formální (IEEE 830), Structured (šablony), Semi-structured (GitHub Issues), Neformální (konverzace)
V open source: issue trackery jako de facto requirements management (Scacchi, 2002)
V agilních týmech: user stories místo SRS (Cao & Ramesh, 2008)
- **Design** → Architektura, design docs, diagramy, API kontrakty
- **Implementation** → Zdrojový kód, konfigurace
- **Validation** → Testy (unit, integration, e2e), test reports
- **Evolution** → Change requests, release notes, patches

Traceability:

→ gotel1994:

“Requirements traceability refers to the ability to describe and follow the life of a requirement, in both a forwards and backwards direction.”

Propojení: Requirement → Design → Code → Test.

2. Nástroje podle činností:

- **Vývoj:** IDE (VS Code, IntelliJ), editory, debuggery
- **Verzování:** Git – distribuovaný version control

→ chacon2014: “everything as code”

- **Koordinace:** Issue tracking (Jira, GitHub Issues)
- **Kvalita:** CI/CD, test frameworks, linters
 - humble2010: automatizace buildů, testů, deploymentu
 - forsgren2018: DevOps praktiky a měřitelný dopad
- **Dokumentace:** Wikis, Markdown, generátory dokumentace

3. Komunikační nástroje:

→ schmidt1992 – CSCW:

“Articulation work” – práce potřebná ke koordinaci spolupráce.

Synchronní: Slack, Teams, meetings. Asynchronní: Email, code review, dokumentace.

4. Standardy a formáty: UML (Fowler, 2004), OpenAPI/Swagger, Markdown, JSON Schema, Conventional Commits.

Zdroje zrušených sekcí:

nato1968, bereska2024, brooks1975, mcconnell2004,
conway1968, malone1994, nonaka1995, scrumguide2020,
gotel1994, chacon2014, humble2010, forsgren2018,
schmidt1992, fowler2004uml, scacchi2002, cao2008.

3. Metodika

3.1 Výzkumný přístup

[DRAFT]

Navrhujeme sadu metrik a iterativní postup pro hodnocení AI coding agentů. Ověřujeme je na případové studii systému upomínek faktur: opakovaně pouštíme agenta s různými variantami instrukcí a pokaždé měříme, co se změnilo. Tento přístup — navrhovat řešení, ověřovat ho v praxi a na základě výsledků upravovat — odpovídá iterativnímu cyklu návrhu a vyhodnocení (Hevner et al., 2004; Peffers et al., 2008).

V naší práci tento cyklus realizujeme čtyřmi kroky: Spuštění (spustit agenta), Měření (změřit výstup), Diagnóza (identifikovat příčiny) a Úprava (upravit instrukce). Podrobný popis kroků uvádí sekce 3.3.4. Postup má dvě fáze: pilotní iterace, kde cyklus opakujeme dokud agent nesplní stanovená exit kritéria (sekce 3.3.4), a komparativní variace, kde z fungujících instrukcí systematicky odebíráme jednotlivé části a měříme dopad na chování agenta (sekce 3.3.5).

Pro ověření jsme zvolili případovou studii na jednom projektu. Řízený experiment by vyžadoval dostatečný počet běhů pro statistickou sílu, což je při ceně jednoho běhu (tisíce tokenů, desítky minut) nepraktické. Případová studie umožňuje opakované běhy s různými instrukcemi a podrobnou analýzu každého běhu. Z jednoho projektu nelze statisticky generalizovat; Yin (R. K. Yin, 2018) pro tento typ výzkumu používá pojem analytická generalizace — na jednom případě ukazujeme princip, který lze ověřit na dalších projektech. Yin tento design nazývá *embedded single-case*: zkoumáme jeden projekt, ale v rámci něj provádíme více běhů agenta, z nichž každý má vlastní instrukce, git historii a sadu naměřených metrik.

[RAW]

Audit trail — předchozí verze 3.1 (DRAFT v3, DSR jako rámeček):

Sekce se jmenovala “Návrhový výzkum (DSR)”. Text stavěl na Hevner et al. (Hevner et al., 2004) (iterativní cyklus návrhu a vyhodnocení) a Peffers et al. (Peffers et al., 2008) (6 aktivit: problém, cíle, návrh, demonstrace, vyhodnocení, komunikace). DSR terminologie (“artefakt”) přidávala komplexitu bez přidané hodnoty pro čtenáře — přejmenováno na “Výzkumný přístup”, Hevner/Peffers zachováni jako citace.

Audit trail — předchozí verze 3.1 (DRAFT v1):

Hevner et al. rozlišují behaviorální a návrhový výzkum. DSR navrhuje nové artefakty a ověřuje jestli řeší daný problém.

RAW TODO (2026-03-24): Původní plán počítal i se substitucí jako samostatnou fází, ale

pilotní iterace substituční efekt demonstrovaly implicitně (r1 obecná pravidla → r3 konkrétní příkazy). Komparativní fáze proto obsahuje jen ablace. Při revizi sjednotit terminologii — v kap03.tex 3.3.5 i kap04 úvod.

3.2 Sada metrik

[DRAFT]

Každý experimentální běh probíhá tak, že agent dostane prázdné GitHub repo, specifikaci v Issue #1 a instrukce v `AGENTS.md`, a autonomně implementuje zadaný projekt. Výstup jednoho běhu (kód, testy, git historie) měříme sadou 19 metrik. Postup běhu a experimentální design popisuje sekce 3.3.

Sada metrik pokrývá tři otázky: jak agent pracoval, co vyrobil a za jakou cenu. Fenton a Bieman (Fenton & Bieman, 2014) klasifikují měřitelné entity v softwarovém inženýrství do tří kategorií: proces, produkt a zdroje. V kontextu AI agentů dospívají k obdobnému rozdělení Yin et al. (2025) (Z. Yin et al., 2025), kteří hodnotí agentní frameworky ve třech dimenzích: effectiveness (výsledek), efficiency (průběh práce) a overhead (spotřebované zdroje). Naše sada metrik sleduje stejnou strukturu a obsahuje 19 metrik ve třech kategoriích:

- P – proces (P1–P8)** Dodržel agent předepsaný postup? Měříme dodržování workflow pravidel z instrukcí (P1–P5) a kvalitu procesních výstupů jako commit messages, issue a PR descriptions (P6–P8).
- Q – kvalita produktu (Q1–Q8)** Je kód kvalitní? Měříme funkční korektnost (Q1–Q2), kvalitu testů (Q3–Q4) a udržitelnost kódu (Q5–Q8).
- E – efektivita (E1–E3)** Za jakou cenu? Zaznamenáváme tokeny z exportu (E1: špička promptu na krok, součet výstupů a součet `cache.read+cache.write` přes kroky — vše v tisících vedle sebe), čas (E2) a stabilitu session (E3).

Výběr konkrétních metrik vychází z potřeb případové studie a dostupných nástrojů; sada si neklade nárok na úplnost. Teoretické základy jednotlivých metrik (proč jsou validní, jaká je evidence) popisuje sekce 2.1.4. Tato sekce se zaměřuje na *jak* konkrétně měříme: jaký nástroj, s jakou konfigurací, co je vstup a co výstup. U každé metriky uvádíme v krátkosti co měří a pak podrobně jak.

3.2.1 Procesní metriky (P1–P8)

[DRAFT]

Procesní metriky (Fenton & Bieman, 2014) měří *jak* agent pracuje. Instrukce v `AGENTS.md` definují strukturovaný vývojový postup: agent má odvozovat práci ze specifikace, organizovat ji přes issues a branches, psát testy před implementací a dokumentovat rozhodnutí v commit messages a PR descriptions.

P1–P5: compliance (binární)

P1 (issues before code) ověřuje, zda agent vytvořil issues s timestampem před prvním kódovým commitem. Měříme porovnáním `created_at` nejstaršího agentova issue (GitHub API) s časem prvního commitu obsahujícího soubory v `src/` nebo `tests/`.

P2 (branch per issue) ověřuje, zda agent vytvořil pro každé issue vlastní větev. Měříme počet remote branches (bez `main`) vůči počtu issues: podmínka $branches \geq issues$ je splněna tehdy, nenastalo-li slučování více issues do jedné větve.

P3 (test-first commits) ověřuje, zda agent psal testy před implementací. Primární indikátor: existuje alespoň jeden commit s prefixem `test:` a zároveň alespoň jeden `feat:`. Přesnější měření poskytuje behavioral trace (sekce 3.3.3): počet větví, kde první zápis do `src/` předcházela prvnímu zápisu do `tests/` (`tddOrderViolations`).

P4 (PRs linked to issues) ověřuje, zda každý PR obsahuje odkaz na issue. GitHub API vrací tělo PR; podmínka: všechna PR těla obsahují regex `Closes #N`.

P5 (no existing test modifications) ověřuje, že agent nepřepisoval již existující testové soubory. Metrika zachycuje `modified` změny (`--diff-filter=M`) na testovacích souborech, tedy situaci kdy agent dříve vytvořený test později upraví místo toho, aby opravil implementaci. Nově přidané testy se do P5 nezapočítávají.

P6–P8: kvalita procesních artefaktů (LLM-as-judge)

P6 (commit message quality) hodnotí popisnost commit messages: atomicitu, konvenční prefix a srozumitelnost co a proč bylo změněno. Hodnocení provádí LLM-as-judge na škále 1–3 (sekce 3.2.4).

P7 (issue description quality) hodnotí popisnost issue descriptions: jasnost scope, přítomnost acceptance criteria, dostatečnost pro implementaci. Hodnocení provádí LLM-as-judge na škále 1–3.

P8 (PR description quality) hodnotí popisnost PR descriptions: přítomnost odkazu na issue, popis co a proč, dostatečnost pro code review. Hodnocení provádí LLM-as-judge na škále 1–3.

[RAW]

Audit trail — předchozí obsah 3.2.1: Procesní cíle (spec-first, strukturovaný postup, TDD, transparentnost) a původní popis metrik P1/P2 včetně exit kritérií (P1=5/5, P2=3/3). Exit kritéria přesunuta do tabulky 3.1 a sekce 3.3.4.

V refactoru P metrik (2026-03-04): P1 byl binární checklist 5 položek, P2 byl LLM-as-judge se 3 dimenzemi a gating rules. Nyní: P1–P5 jsou samostatné binární metriky (bez souhrnného skóre), P6–P8 jsou samostatné LLM-as-judge metriky bez hard gates (P3 jako binární metrika

nahrazuje gating rule pro test: commit).

3.2.2 Produktové metriky (Q1–Q8)

[DRAFT]

Produktové metriky (Fenton & Bieman, 2014) měří *co* agent vyrobil. Pokrývají tři oblasti: funguje implementace správně (Q1–Q2), detekují agentovy testy skutečné chyby (Q3–Q4) a je kód udržitelný (Q5–Q8).

Funkční korektnost (Q1–Q2)

Q1 (API contract match) ověřuje, zda agentův kód dodržuje definované rozhraní. Referenční typy se importují a zkompilují proti agentovu kódu (`tsc`). Výsledek je binární: typy sedí, nebo ne. Q1 je vstupní podmínkou pro Q2: pokud agentův kód neimplementuje správné API, referenční testy nelze ani zkompilovat, a výsledek Q2 by byl nesmyslný.

Q2 (referenční test pass rate) měří kolik z referenčních testů projde na agentově implementaci. Vitest spustí 42 testů proti agentovu kódu; výsledek je počet passing testů z 42. Testy ověřují chování přes veřejné API (black-box, sekce 2.1.4), takže je lze spustit na libovolném běhu nezávisle na interní struktuře. Konstrukce referenční test suite popisuje sekce 4.1.2.

Kvalita testů (Q3–Q4)

Q3 (mutation score) měří, jestli agentovy testy skutečně detekují chyby (sekce 2.1.4). Stryker s konfigurací `--mutate 'src/**/*.ts'` a mutátory pro TypeScript (conditional, arithmetic, string, logical operators) systematicky zavádí drobné změny do zdrojového kódu. Vstup: agentovy testy + zdrojový kód. Výstup: procento zabitých mutantů z celkového počtu.

Q4 (AC coverage) měří, kolik z 25 acceptance criteria má odpovídající test v agentově test suite. Na rozdíl od Q2 (funguje implementace?) se Q4 ptá, jestli agent *testoval všechno co měl*. Judge dostane seznam 25 AC ze specifikace a agentovu test suite. Pro každé AC určí, zda existuje test který ho pokrývá. Výstup: počet pokrytých AC z 25. Hodnocení provádí LLM-as-judge (sekce 3.2.4).

Kvalita kódu (Q5–Q8)

Q5 (lint warnings) počítá varování a chyby z ESLint s fixní konfigurací (pravidla: recommended + strict TypeScript rules). Konfigurace je součástí experimentální infrastruktury, ne agentova repo — agent ji nemůže měnit. Výstup: celkový počet warnings + errors.

Q6 (typecheck errors) počítá chyby z `tsc --noEmit` ve strict mode. Doplnuje počet explicitních `any` ve zdrojových souborech (`grep` v `src/**/*.ts`) jako proxy pro obcházení typového systému.

Q7 (složitost kódu) měří cyklomatickou složitost per funkce. ESLint `complexity` rule reportuje funkce překračující nastavený práh. Výstup: maximální složitost across all functions. Prah ≤ 10 per funkce vychází z McCabe (sekce 2.1.4).

Q8 (design quality) hodnotí aspekty které automatizované nástroje nezachytí: pojmenování, oddělení zodpovědností, idiomatický TypeScript, kvalita dokumentace a zbytečná komplexita. Hodnocení provádí LLM-as-judge (sekce 3.2.4). Celkové skóre Q8 je *minimum* pěti dimenzí, nikoliv průměr. Volba minima je záměrná: jeden slabý rozměr má stáhnout celkový výsledek dolů, aby slabiny nebyly maskované silnými dimenzemi.

[RAW]

Audit trail — předchozí obsah 3.2.2: Produktové cíle (funkční korektnost, kompatibilní API, kvalitní testy, kvalitní kód) a původní popis metrik Q1–Q8 včetně exit kritérií. Exit kritéria přesunuta do tabulky 3.1 a sekce 3.3.4.

3.2.3 Metriky efektivity (E1–E3)

[RAW]

TODO (2026-03-26): Tento odstavec je příliš technický pro metodiku — detailní výpočet tokenů z `transcript.json` (jak počítat cache, proč ne `sumInputTokens`) patří do kap04 nebo přílohy. Zde by mělo být jen CO měříme (tokeny, čas, kompakce) a PROČ (porovnání nákladů). Odkaz na konkrétní hodnotu `pilot-r1` (ř. 291) nepatří do metodiky vůbec.

[DRAFT]

Metriky efektivity měří zdroje spotřebované při vývoji. Nemají exit kritérium a slouží k porovnání nákladů mezi iteracemi.

E1 (vstup/výstup/cache tokeny) z exportu `transcript.json` počítáme tak, aby šlo běhy srovnat: **výstup** = součet `output+reasoning` přes všechny asistentní kroky (celkově vygenerovaný text a reasoning); **vstup** = **maximum** na jednom asistentním kroku ze součtu `input+cache.read+cache.write`; Σ **cache** = součet `cache.read+cache.write` přes všechny zúčtované kroky (kumulativní účetnictví v exportu — může být velké číslo, slouží k porovnání mezi běhy, ne jako jedna špička). Důvod pro rozdělení: poskytovatel v exportu často **odděluje** část promptu do `cache.read`; samotné pole `input` pak podstatně **podhodnocuje** objem kontextu v jednom requestu. **Součet vstupů přes všechny kroky neuvádíme** — opakovaně by započítával stejný kontext v každém turnu a mezi běhy by vytvářel nerealistické skoky (např. při delší session). V tabulkách jsou tři složky v tisících tokenů, zaokrouhlené na celé číslo (např. **115 / 60 / 11 528** u `pilot-r1`: max. vstup vč. cache $\approx 115\,000$ na jednom

kroku, součet výstupů $\approx 60\,000$, součet cache $\approx 11,5$ mil. tokenů). **Diagnosticke** ponecháváme `sumInputTokens` (součet pole `input`) a `peakInputTokens` (max. jen `input`) ve skriptech; do hlavičky E1 tabulek nepatří. Pole `tokens.total` v exportu **do E1 nepočítáme** (míchá promptové a generované tokeny; pro šířku promptu používáme `input+cache`). Sledování objemu tokenů motivuje zjištění Gloaguen et al. (Gloaguen et al., 2025), že context files zvyšují inference cost o více než 20 %.

E2 (trvání) měří wall-clock time v minutách od spuštění agenta po poslední commit.

E3 (kompakce kontextu) počítá události, kdy se podle změny `snapshot` mezi kroky v transcriptu OpenCode provedl zápis kompakce kontextu (heuristika). Doplnkově sledujeme úspěšné dokončení beze ztráty session a počet restartů z auto-continue (`metrics.csv`).

Se dvěma běhy per variaci jsou tyto hodnoty deskriptivní, ne inferenční.

[RAW]

Audit trail — předchozí obsah 3.2.3: Původní popis E1–E3 se zdrojů (SWE-bench, AgentBench).

3.2.4 Aplikace LLM-as-judge

[DRAFT]

Pět metrik (P6, P7, P8, Q4, Q8) hodnotí vlastnosti které nelze extrahovat automatizovaným nástrojem. Pro jejich hodnocení používáme metodu LLM-as-judge (sekce 2.1.4).

Volba modelu. Jako judge používáme GLM-5 (Zhipu AI). Agentní běhy provádí model Minimax — judge je tedy z jiné modelové rodiny. Důvod: model má tendenci hodnotit výstupy svého vlastního druhu lépe, i když jsou objektivně slabší (self-preference bias (Panickssery et al., 2024)). Volba judge z jiné modelové rodiny než agent tento bias snižuje (Verga et al., 2024).

Škála. Hodnocení probíhá na škále 1–3 per dimenze: 1 = nevyhovující, 2 = přijatelné, 3 = dobré. Jemnější škály (1–5, 1–10) produkují při malém počtu hodnocených artefaktů nižší shodu mezi hodnotiteli (Zheng et al., 2023); třístupňová škála je pro tento rozsah spolehlivější.

Rubrika. Judge dostane artefakty agentova běhu (zdrojový kód, commit messages, issue a PR descriptions) spolu s rubrikou obsahující popis každého stupně. Prompt je fixní across all runs, aby hodnocení bylo srovnatelné mezi iteracemi. Rubriky jsou uvedeny v příloze.

Interpretace výsledků. LLM-as-judge zde neslouží jako plně objektivní měření, ale jako strukturované posouzení vlastností, které nelze spolehlivě vyhodnotit deterministickým skriptem. Výsledky těchto metrik proto interpretujeme jako podpůrné kvalitativní indikátory a ne jako hlavní důkazní osu experimentu.

[RAW]

Audit trail — LLM-as-judge obsah přesunut do kap02. Teorie (Zheng 2023, Panickssery 2024, Verga 2024 PoLL) v kap02. Aplikace (GLM-5, škála, rubriky, interpretace výsledků) zde v kap03.

3.3 Experimentální design

3.3.1 Výběr projektu

[DRAFT]

Případová studie potřebuje projekt, na kterém lze opakovaně spouštět agenta s různými instrukcemi a objektivně měřit výsledky. Projekt musí mít deterministickou logiku (stejný vstup vždy dá stejný výstup), aby bylo možné ověřit korektnost automatizovanými testy a mutation testingem. Zároveň musí být dostatečně malý pro více experimentálních běhů, ale obsahovat reálné nuance (hraniční případy, doménová pravidla), aby měl agent co řešit.

Zvolili jsme systém upomínek faktur: systém pro automatické odesílání připomínek k nezaplaceným fakturám. Obsahuje stavový automat (nová, po splatnosti, upomínaná, eskalovaná), časové výpočty (pracovní dny, ochranné lhůty) a pravidla pro eskalaci. Specifikace definuje 25 acceptance criteria, API kontrakt (TypeScript typy a signatury) a doménový slovník. Tato kombinace deterministické logiky a hraničních případů (víkendy, svátky, grace periods) umožňuje objektivně testovat jak funkční korektnost (Q1, Q2), tak kvalitu testů (Q3, Q4).

[RAW]

TODO (2026-03-16) — rozepsat systém upomínek faktur do detailu:

Sekce “Výběr projektu” musí obsahovat víc než jen odstavec. Popsat:

1. Co systém dělá (stories/funkcionalita):

- Stavový automat: faktura prochází stavy (new → overdue → reminded → escalated → terminal)
- Eskalační flow: 8 stavových přechodů, každý s vlastními pravidly
- Časové výpočty: pracovní dny (skip víkendy + svátky), ochranné lhůty (grace periods)
- Akce: send_reminder, escalate, pause/resume dunning procesu
- Platby: partial payments, full payments, jak ovlivňují stav
- Konfigurovatelné timeouty: zákazník si nastaví lhůty
- Pure function: process(state, event, now) → nový stav + action descriptory

2. Proč je vhodný pro výzkum (edge cases, determinismus):

- **Determinismus:** stejný vstup = stejný výstup, žádné side effects → objektivně testovat

vatelné

- **Edge cases bohaté na nuance:** víkendy a svátky v business days výpočtu, grace periods přes víkend, pause uprostřed eskalace, resume po delší době, kumulace elapsed time
- **Doménová pravidla:** ne triviální CRUD, ale business logika s invarianty (nelze eskalovat terminated fakturu, nelze resumovat co nebylo paused)
- **Přiměřená velikost:** dost malý na opakované běhy (desítky minut), dost velký aby agent měl co řešit (25 AC, stavový automat)
- **Objektivní testovatelnost:** mutation testing funguje (deterministická logika), referenční testy jednoznačné

Toto nahrazuje potřebu “produktových cílů” — popis projektu ukazuje PROČ je vhodný jako evaluační prostředí.

Audit trail — předchozí obsah 3.3.1: Kritéria výběru (hard logic, jasné invarianty, testovatelné, přiměřená velikost, reálný use case) a stručný popis projektu.

3.3.2 Fixní proměnné

[DRAFT]

Aby bylo možné měřit vliv instrukcí, musí být všechno ostatní konstantní. Jedinou proměnnou mezi běhy je obsah souboru `AGENTS.md`, procedurálních instrukcí, které definují pracovní postup, omezení a quality gates. Na rozdíl od generických context files, které typicky popisují adresářovou strukturu a build příkazy (Gloaguen et al., 2025), jde o fokusované instrukce s konkrétními procedurami (Li et al., 2025).

Fixní proměnné jsou:

Prázdné GitHub repo obsahuje pouze `AGENTS.md`, konfiguraci agenta a auto-continue plugin. Žádný existující kód, testy ani `package.json`. Agent musí inicializovat projekt a zvolit strukturu sám.

Specifikace v GitHub Issue #1 obsahuje 25 acceptance criteria, API kontrakt (TypeScript typy a signatury), doménový slovník a out of scope. Jediný vstup který agent dostane kromě instrukcí.

Auto-continue plugin je hook který detekuje kdy se agent zastaví a automaticky ho restartuje. Počítadlo restartů a kontrola otevřených issues zajišťují běh bez manuálního zásahu. Metrika E3 z toho čerpá.

Model MiniMax-M2.5 byl zvolen jako model schopný autonomně dokončit zadaný úkol. Cílem práce je vyhodnotit sadu metrik a iterativní postup, ne srovnávat výkon modelů — volba konkrétního modelu je v tomto ohledu sekundární. MiniMax-M2.5 je zároveň z jiné modelové rodiny než judge (GLM-5, Zhipu AI), čímž se eliminuje self-preference bias (sekce 2.1.4).

Běhové prostředí (Docker container) zajišťuje, že agent nemá přístup ke globální konfiguraci nástroje, MCP serverům ani rozšířením nainstalovaným na hostitelském systému. Izolace zaručuje, že chování agenta je determinováno výhradně obsahem `AGENTS.md` a specifikací — ne vedlejšími vstupy z prostředí. Sdílený autentizační svazek zajišťuje konzistentní přístup k API napříč běhy.

System prompt agenta (`build.md`) nahrazuje výchozí system prompt nástroje OpenCode, jehož instrukce o verzování konfliktvaly s procesními požadavky experimentu. Vlastní system prompt obsahuje pouze obecné kódové konvence (styl, bezpečnost, zákaz komentářů) a odkaz na `AGENTS.md`. Veškeré procesní instrukce jsou výhradně v `AGENTS.md`, aby procesní chování agenta bylo řízeno jedinou proměnnou experimentu.

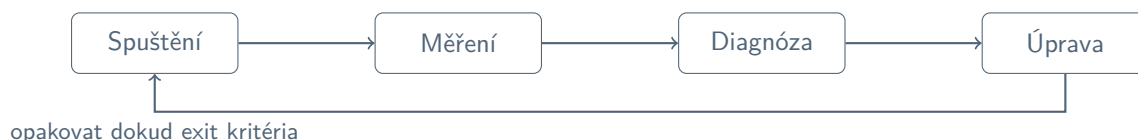
[RAW]

Audit trail — předchozí obsah 3.3.2: Popis fixních proměnných včetně detailů o auto-continue pluginu (`session.idle` hook) a system promptu (`build.md, mode: primary`).

3.3.3 Iterativní cyklus

[DRAFT]

Pilotní fáze postupuje opakováním čtyřkrokového cyklu, dokud agent nesplní exit kritéria. Každý průchod cyklem je jedna experimentální iterace — vstupem je aktuální verze `AGENTS.md`, výstupem upravená verze pro příští běh.



Spuštění. Skript `new-run.ts` připraví izolované prostředí: vytvoří prázdné GitHub repo, zkopíruje aktuální `AGENTS.md`, vytvoří Issue #1 se specifikací a spustí agenta v Docker kontejneru. Agent dostane výhradně prázdné repo, instrukce a specifikaci — žádný existující kód ani závislosti. Po dokončení se exportuje session transcript (`transcript.json`).

Měření. Skript `analyze-run.ts` extrahuje deterministické metriky automaticky: git log a GitHub API (P1), výstup Vitest a Stryker (Q2, Q3), statická analýza (Q5–Q7), session metadata (E1–E3). Kvalitativní metriky (P6, P7, P8, Q4, Q8) hodnotí LLM-as-judge GLM-5 podle fixní rubriky (sekce 3.2.4). Výstupem je `FINDINGS.md` s tabulkou metrik a faktickým popisem chování agenta.

Diagnóza. Z naměřených hodnot a behaviorálního popisu se identifikuje kde a proč agent nedodržel očekávané chování. Analýza postupuje podle čtyř rámců:

- Mao et al. (Mao et al., 2025) — která složka instrukcí selhala a proč

- Hassan et al. (Hassan et al., 2025) — zda obsah vyvažuje proceduru, kontext a omezení
- Lulla et al. (Lulla et al., 2026) — zda každý řádek přidává hodnotu
- Razavi, Breunig (Breunig, 2025; Razavi & Fard, 2025) — proč agent instrukci opakovaně ignoruje

Výstupem je `DIAGNOSIS.md` s identifikovanými příčinami a návrhem změn.

Úprava. Na základě diagnózy se upraví `AGENTS.md`. Každá změna odpovídá jednomu identifikovanému problému, je podložena citací a zaznamenána v changelogu — aby bylo zpětně dohledatelné proč daná úprava vznikla a co měla opravit. Přestrukturování instrukcí je preferováno před jejich rozšiřováním: redundantní obsah zvyšuje inference cost bez přínosu k úspěšnosti (Gloaguen et al., 2025) a fokusované instrukce překonávají vyčerpávající dokumentaci (Li et al., 2025).

[RAW]

TODO (2026-03-25) — doplnit do Diagnózy větu o AI asistenci: Diagnostická analýza probíhala za asistence nástroje Claude (Anthropic); výzkumník formuloval hypotézy o příčinách selhání, AI asistent poskytoval analytickou podporu a návrhy změn na základě literatury. Finální rozhodnutí o každé úpravě instrukcí bylo vždy na výzkumníkovi. Konzultovat s vedoucím — kam přesně v Diagnóze tuto větu zařadit.

[RAW]

Review poznámka (2026-03-21) — Hassan SASE jako diagnostický rámec: Hassan et al. (Hassan et al., 2025) je vision paper, ne empirická studie. BriefingScript/LoopScript/MentorScript taxonomie je konceptuální nástroj, ne empiricky validovaný poměr. V textu používáme jako analytický rámec pro klasifikaci obsahu instrukcí, ne jako evidence “co funguje”. Ujistit se že v kap02 i kap03 je toto jasné — nepřipisovat Hassanovi empirické výsledky.

Rozepsat každý ze čtyř kroků podrobněji — co přesně se děje, jaké nástroje, jaké výstupy.

Spuštění — operacionalizace:

Vstup kroku: aktuální verze `AGENTS.md` (jediná měnící se proměnná).

Skript `new-run.ts` automatizuje přípravu prostředí: (1) vytvoří prázdné GitHub repo s názvem `bp-billing-reminder-pilot-rN`, (2) zkopíruje aktuální `AGENTS.md` a `build.md` (system prompt override), (3) vytvoří Issue #1 se specifikací (25 AC, API kontrakt, doménový slovník), (4) spustí OpenCode s modelem Minimax-2.5 v neinteraktivním režimu.

Auto-continue plugin detekuje kdy se agent zastaví (`session.idle` hook) a automaticky ho restartuje. Počítadlo restartů a kontrola otevřených issues zajišťují doběhnutí bez manuálního zásahu.

Agent dostane výhradně: prázdné repo + `AGENTS.md` + `build.md` + Issue #1. Žádný existující kód, testy ani `package.json`.

Výstupy kroku: agentovo GitHub repo (kód, testy, git log, issues, PRs), session transcript (`transcript.json` přes `opencode export <sessionID>`).

Měření — operacionalizace:

Skript `analyze-run.ts` extrahuje metriky z pěti zdrojů:

- git log + GitHub API → P1 (timestamps issues vs. commity, branch names, PR body regex)
- Vitest výstup (parse *N passed*) → Q2
- Stryker JSON report → Q3 (mutation score)
- ESLint JSON report → Q5 (warnings), Q7 (complexity rule)
- `tsc --noEmit stderr` → Q6
- `transcript.json` → E1 (vstup / výstup / Σ cache v tisících), E2 (trvání), E3 (kompakce kontextu)

Skript `judge.ts` spouští LLM-as-judge (GLM-5) pro P6, P7, P8, Q4, Q8. Workflow: pošle artefakty + rubriku; výstup = JSON se score a rationale. P6: commit messages hodnocené samostatně 1–3 (atomicita, konvenční prefix, popisnost). P7: issue descriptions hodnocené samostatně 1–3 (scope, AC, detail pro implementaci). P8: PR descriptions hodnocené samostatně 1–3 (odkaz na issue, co a proč, dostatečnost pro review). Q4: judge dostane seznam 25 AC ze specifikace + agentovu test suite; partial coverage počítá jako pokryté. Q8: pět dimenzí (naming, SoC, idiomatic TS, documentation, complexity); overall = MINIMUM pěti dimenzí — jeden slabý score stáhne celkový.

Výstupy kroku: `FINDINGS.md` (tabulka P/Q/E metrik + behavioral summary). Soubory `p2-result.json`, `q4-result.json`, `q8-result.json`.

Behavioral summary (3–5 vět) popisuje co agent skutečně udělal: kolik issues vytvořil, zda TDD dodržel, zda commitoval průběžně nebo blob na konci — faktická pozorování z git logu a GitHub API, ne hodnocení.

Měření — poznámka k duplikaci: Tabulka zdroj → metrika → nástroj by duplikovala sloupec Nástroj z přehledové tabulky 3.1. Alternativa: rozšířit přehledovou tabulku o sloupec *Zdroj dat* (git log, GitHub API, Vitest, Stryker atd.). Nebo jen odstavec textu: “P1 bere data z git logu a GitHub API, Q2–Q3 z Vitest/Stryker výstupu, E1–E3 z transcript.json.”

Diagnóza — operacionalizace 4 nástrojů:

Výstup `FINDINGS.md` identifikuje *co* nesplnil. `DIAGNOSIS.md` vysvětluje *proč* a navrhuje fix.

Přehled rámců jako tabulka — zvážit pro finální text:

Rámec	Otázka	Co analyzuje	Výstup
FSE/Mao (Mao et al., 2025)	Co chybí/přebývá?	7 komponent AGENTS.md	komponentní analýza
SASE/Hassan (Hassan et al., 2025)	Je balance typů obsahu ok?	Brief/Loop/Mentor poměr	poměr + co dominuje
Lulla (Lulla et al., 2026)	Přidává každý řádek hodnotu?	efektivita obsahu	kandidáti na odstranění
Breunig/Razavi (Breunig, 2025; Razavi & Fard, 2025)	Proč ignoruje opakovaně?	recovery strategie	4-krokový postup

FSE 2025 (Mao et al.) — projít AGENTS.md proti 7 komponentám v doporučeném pořadí: Role → Directive → Context → Workflow → Output → Constraints → Examples. Finding 2: Directive musí být imperativní, ne otázky. Finding 4: Constraints = exclusion type nejčastější (46 % výskytů v korpusu) — *Never X > Avoid X*. Finding 9: Instrukce AFTER context, ne before — information decay při opačném pořadí. Constraints seskupit do jedné sekce, ne rozházet po dokumentu.

SASE script balance (Hassan 2025) — klasifikovat řádky AGENTS.md: BriefingScript = co + proč (cíl, kontext, success criteria); LoopScript = jak (workflow, kroky, iterace); MentorScript = co ne + error recovery (constraints, guardrails). Cílový poměr: přibližně vyrovnaný. LoopScript dominance ⇒ agent neumí reagovat na odchylky. Chybějící MentorScript ⇒ agent neví jak postupovat při chybě.

Lulla 2026 — přítomnost AGENTS.md: −28,6 % mediánový runtime, −16,6 % výstupní tokeny. Paper nerozlišuje efektivitu jednotlivých typů obsahu (to je spekulace v sekci Research Roadmap). Číslo +14–22 % tokenů NENÍ z Lully. Filtr (AGENTbench meta-princip): “*Kdyby tento řádek chyběl, udělal by agent neočividnou chybu?*” — pokud ne, kandidát na odstranění.

Breunig / Razavi — pokud agent opakovaně ignoruje instrukci přes iterace: nepřidávat znovu, přestrukturovat. 4-krokový recovery: (1) jiný formát nebo jiná pozice v dokumentu, (2) rozložit na menší kroky — identifikovat přesně kde se odchyluje, (3) přidat validační krok — agent si sám ověří compliance, (4) negative example — ukázat co NEDĚLAT (Razavi 2025: targeted clarification).

Výstup kroku: DIAGNOSIS.md (FSE tabulka 7 komponent + SASE balance poměr + Lulla audit checklist + Breunig findings + návrh fixů s citacemi).

Úprava — operacionalizace:

Pravidla pro změny vycházejí z diagnostiky:

- Jedna změna = jeden identifikovaný problém (nekombinovat nesouvisející fixy).
- Každá změna musí mít citaci — žádné ad hoc “zkusíme tohle”.
- Preferovat přestrukturování před přidáváním — AGENTbench: víc textu ≠ lepší výsledky.
- Testovat každý nový řádek proti meta-principu (Kdyby chyběl ...?).

Formát záznamu per změna: bylo → je → pozorování (rN) → diagnóza (FSE/SASE/Lulla-/Breunig) → literatura.

Výstupy kroku: aktualizovaný AGENTS.md + záznam changelog/pilot-rN-to-rN+1.md.

3.3.4 Pilotní fáze

[DRAFT]

Pilotní fáze běží dokud agent nesplní exit kritéria ze sloupce *Exit kritérium* v tabulce 3.1. Kritéria mají dva typy: deterministické metriky (P1–P5, Q1–Q2, Q4–Q6) mají přirozenou binární hranici — buď splněno nebo ne; minimální standard (P6–P8, Q3, Q7–Q8) je práh zdůvodněný literaturou. E1–E3 jsou záznamové metriky bez kritéria, slouží k porovnání efektivity mezi běhy.

S ohledem na scope práce (jeden projekt, jeden model) stačí jeden úspěšný běh bez manuálního zásahu. Každá iterace produkuje aktualizovaný `AGENTS.md`, záznam změn se zdůvodněním a kompletní P/Q/E metriky.

[RAW]

Audit trail — **předchozí obsah 3.3.3:** Podrobný popis cyklu Spuštění/Měření/Diagnóza/-Úprava, diagnostický rámec (Mao component analysis, Hassan script balance, Lulla content effectiveness, Razavi/Breunig prompt sensitivity), výstupy iterace (`AGENTS.md diff`, `CHANGELOG`, `FINDINGS.md`), exit kritéria.

3.3.5 Komparativní variace

[DRAFT]

Z fungující sady instrukcí (výstup pilotní fáze) systematicky odebíráme jednotlivé složky a měříme dopad na chování agenta. Tento postup — ablace — odpovídá na otázku: potřebuje agent danou část instrukcí, nebo je redundantní?

Konkrétní variace nelze specifikovat předem. Pilotní fáze *generuje* hypotézy o tom, které složky instrukcí jsou pro chování agenta podstatné, a komparativní fáze je testuje. Výběr variací vychází z diagnostiky pilotních běhů (sekce 3.3.4): pokud se v pilotu ukáže, že agent konzistentně nedodržuje určité pravidlo, ablace může testovat, zda jeho odebrání změní i ostatní metriky. Podobně pokud určitá sekce instrukcí koreluje s vysokou kvalitou výstupu, ablace ověří, zda je tento vztah kauzální.

Každá ablace se provádí ve dvou nezávislých bězích se stejným nastavením, aby bylo možné odlišit systematický efekt změny od přirozené variability nedeterministického modelu. I se dvěma běhy nelze dosáhnout statistické průkaznosti (sekce 3.5); výsledky komparativní fáze proto interpretujeme jako indikativní, nikoliv kauzální.

[RAW]

Audit trail — **předchozí obsah 3.3.5:** DRAFT v1 zmiňoval ablaci i substituce. Substituce v kap04 jako samostatná fáze neprobíhaly (pilotní iterace je demonstrovaly implicitně:

r1 obecná pravidla → r3 konkrétní příkazy). DRAFT v2 zúžen jen na ablace v souladu s cílem 3 a kap04.

3.4 Přehledová tabulka

[DRAFT]

Tabulka 3.1 shrnuje všechny metriky na jednom místě. Sloupec *typ* rozlišuje deterministická exit kritéria (přirozená binární hranice), minimální standard (práh zdůvodněný literaturou) a záznamové metriky (bez kritéria, slouží k porovnání mezi běhy). deterministické metriky (P1–P5, Q1–Q3, Q5–Q7, E1–E3) sbírá jeden skript; kvalitativní metriky (P6–P8, Q4, Q8) hodnotí LLM-as-judge (sekce 3.2.4).

Tabulka 3.1: Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ

Kód	Metrika	Nástroj	Exit kritérium	Typ
<i>Procesní (P) — jak agent pracuje</i>				
P1	Issues before code	git, GitHub API	pass	deter.
P2	Branch per issue	git, GitHub API	pass	deter.
P3	Test-first commits	git log	pass	deter.
P4	PRs linked to issues	GitHub API	pass	deter.
P5	No existing test modifications	git diff	pass	deter.
P6	Commit message quality	LLM-as-judge (GLM-5)	≥ 2/3	min.
P7	Issue description quality	LLM-as-judge (GLM-5)	≥ 2/3	min.
P8	PR description quality	LLM-as-judge (GLM-5)	≥ 2/3	min.
<i>Produktové (Q) — co agent vyrobil</i>				
Q1	API contract match	tsc (import + typecheck)	match	deter.
Q2	Referenční test pass rate	Vitest (42 testů)	42/42	deter.
Q3	Mutation score	Stryker	≥ 70 %	min.
Q4	AC coverage agentových testů (25 krit.)	LLM-as-judge (GLM-5)	25/25	deter.
Q5	Lint warnings	ESLint	0	deter.
Q6	Typecheck errors	tsc --noEmit	0	deter.
Q7	Cykломatická složitost	ESLint (complexity)	≤ 10/fn	min.
Q8	Design quality	LLM-as-judge (GLM-5)	≥ 2/3	min.
<i>Efektivita (E) — za jakou cenu</i>				
E1	Vstup / výstup / Σ cache (tis.) z exportu	OpenCode export	—	záznam
E2	Trvání (minuty)	session timestamps	—	záznam
E3	Kompakce + dokončení + restarty	transcript + auto-continue	—	záznam

3.5 Omezení a validita

[DRAFT]

Každý výzkumný design má omezení. Následující analýza vychází z kategorizace Runeson a Höst (Runeson & Höst, 2009).

Konstruktová validita (měříme to co chceme měřit?). Jednotlivé metriky vychází z existující teorie: mutation testing (Papadakis et al., 2019), cyklomatická složitost (McCabe, 1976), taxonomie procesních a produktových metrik (Fenton & Bieman, 2014). Konkrétní kombinace metrik do sady a volba exit kritérií jsou autorské a představují návrh, ne ověřený standard. Pět metrik (P6, P7, P8, Q4, Q8) obsahují subjektivní složku (LLM-as-judge). Tyto metriky proto slouží jako podpůrné kvalitativní indikátory, zatímco hlavní opora vyhodnocení stojí na deterministických metrikách a auditovatelných artefaktech. Spolehlivost LLM-as-judge nebyla validována proti lidskému hodnocení (např. Cohenovým κ) z důvodu časových omezení; výsledky kvalitativních metrik proto nelze interpretovat jako objektivní měření. Experiment rozlišuje dva druhy testů: referenční testy (měřené metrikou Q2, spouštěné po běhu skriptem, agentovi neviditelné) a agentovy vlastní testy (psané agentem během běhu, viditelné a spustitelné agentem). Metrika P5 se vztahuje výhradně k agentovým vlastním testům. Referenční testy jsou odvozeny ze specifikace metodou TDD (Mathews & Nagappan, 2024), čímž se snižuje riziko test oracle problému. Při validaci měřicí infrastruktury byly identifikovány známé nedostatky (Q4, P7, Q6) a změny prostředí mezi běhy (Docker izolace, E1 neporovnatelnost, temperature); tyto poznatky jsou konzistentní napříč běhy a jsou diskutovány v sekci 5.3. Správnost skriptu `analyze-run.ts` pro metriky P1–P5 byla ověřena manuálním srovnáním výstupů s raw daty z git logu a GitHub API pro běhy r4 a r5; žádná diskrepance nebyla identifikována.

Interní validita (jsou závěry podložené daty?). Hlavní hrozba je nedeterminismus LLM: stejné instrukce mohou při různých bězích dát různé výsledky (Razavi & Fard, 2025). Dva běhy per variaci jsou mitigací, ale neposkytují statistickou sílu. Další hrozbou je confirmation bias: autor navrhuje instrukce i evaluuje výsledky. Tuto hrozbu oslabují automatizované metriky (P1, Q1–Q3, Q5–Q7), které nepodléhají subjektivnímu posouzení, a volba judge modelu z jiné rodiny. Mitigací je také auditovatelný řetězec rozhodnutí (changelog per iterace, korekční poznámky v `FINDINGS.md`) a oddělení měřicí infrastruktury od agentova prostředí. Jak se tyto hrozby projeví v praxi — včetně diagnostické chyby výzkumníka, změn prostředí mezi běhy, nenastavené temperature modelu a tichých ukončení nástroje — diskutuje sekce 5.3.

Externí validita (lze zobecnit?). Případová studie na jednom projektu, jednom modelu a jednom agentním nástroji neumožňuje statistickou generalizaci. Yin (R. K. Yin, 2018) pro tento typ výzkumu rozlišuje analytickou generalizaci: z jednoho případu lze ukázat principy, ne statistické zákonitosti. Systém upomínek má deterministickou logiku a výsledky nemusí platit pro projekty s uživatelským rozhraním, strojovým učením nebo nedeterministickými výstupy. Přenositelné jsou metriky a postup (kdokoliv je může použít na svém projektu), konkrétní naměřené hodnoty a instrukce platí pro tuto studii.

Výzkum nezahrnuje lidské účastníky. Veškerá data (git log, metriky, session transcripts) jsou generována experimentální infrastrukturou. Zdrojový kód a experimentální data jsou veřejně dostupná.

[RAW]

TODO (2026-03-25) — doplnit do 3.5 odstavec o využití AI: Při přípravě práce byl průběžně využíván nástroj Claude (Anthropic) jako AI asistent — pro úpravu a rozšíření textových částí, vyhledávání literatury a diagnostickou analýzu v kroku Diagnóza. Výzkumný design, volba metrik, experimentální rozhodnutí a interpretace výsledků jsou autorovy vlastní; generovaný obsah byl vždy autorem posouzen a upraven. Podrobný popis využití AI je uveden v příloze D. Konzultovat s vedoucím — zda patří do 3.5 nebo jako samostatná podsekcce.

[RAW]

Audit trail — předchozí obsah 3.5: Struktura: construct validity (Fenton, Papadakis, McCabe, LLM-as-judge κ , test oracle), internal validity (prompt sensitivity, LLM nedeterminismus, confirmation bias), external validity (analytická generalizace, deterministická logika, přenositelnost), etika.

[RAW]

Audit trail — obsah ze staré 3.3 (Cíle artefaktu):

Empirická evidence naznačuje, že typ a struktura instrukcí zásadně ovlivňuje jejich efektivitu. Gloaguen et al. (Gloaguen et al., 2025) zjistili, že generické repository-level context files mají marginální nebo negativní dopad. Li et al. (Li et al., 2025) ukazují, že fokusované procedurální instrukce zlepšují pass rate o 16,2pp. Princip: *minimální, cílený a procedurální* kontext je efektivnější než vyčerpávající dokumentace.

4. Praktická část

[DRAFT]

Tato kapitola popisuje provedení případové studie. Struktura sleduje chronologii experimentu: nejprve příprava tří výchozích podkladů (specifikace, referenční implementace, instrukce), pak pilotní iterace kde se instrukce opakovaně upravují na základě naměřených metrik, a nakonec komparativní variace kde z fungující sady systematicky odebíráme jednotlivé části a měříme dopad.

[RAW]

Audit trail — předchozí verze úvodu kap04:

Tato kapitola popisuje provedení experimentu a jeho výsledky. Struktura sleduje chronologii: příprava artefaktů (specifikace, referenční implementace, instrukce) → pilotní iterace → komparativní variace → souhrnné výsledky.

4.1 Příprava experimentu

[DRAFT]

Před spuštěním pilotních běhů bylo nutné připravit tři výchozí podklady každého experimentálního běhu: specifikaci projektu (co má agent implementovat), referenční implementaci (měřicí nástroj pro Q2 – referenční test pass rate) a baseline instrukce (jak má agent pracovat).

[RAW]

Audit trail — předchozí verze úvodu 4.1:

Před spuštěním pilotních běhů bylo nutné připravit tři artefakty: specifikaci projektu (co má agent implementovat), referenční implementaci (baseline pro měření kvality) a výchozí instrukce (jak má agent pracovat).

4.1.1 Konstrukce specifikace

[DRAFT]

Specifikace definuje co má agent implementovat. V této práci má formu strukturované GitHub issue (Issue #1), kterou agent dostává jako zadání. Tato volba navazuje na metodické vymezení fixních vstupů v sekci 3.3: agent kromě instrukcí v souboru **AGENTS.md** pracuje právě s touto specifikací a žádný další architektonický návrh nedostává. Issue se skládá ze dvou částí: **Requirements** (co business potřebuje) a **API Contract** (co musí implementace exportovat).

Behavioral model, state diagram ani explicitně formulované invarianty nejsou součástí zadání; návrh interní architektury je ponechán agentovi. API kontrakt, tedy exportované funkce a typy, představuje jediné technické omezení implementace a současně slouží jako vstup pro metriku Q1 (API contract match).

Podoba specifikace vychází ze tří principů důležitých pro tuto evaluační úlohu: jednoznačnosti požadavků, jejich testovatelnosti a omezení interpretační volnosti agenta. Specifikace je proto záměrně stručná a strukturovaná. Stručnost reaguje na poznatek, že delší kontext může zhoršovat schopnost jazykového modelu spolehlivě využít relevantní informaci (Liu et al., 2024). Současně byla zvolena taková struktura, která minimalizuje redundantní instrukce a udržuje přímou vazbu mezi požadavkem, implementací a následným vyhodnocením (Bockeler, 2025).

Specifikace obsahuje:

- **25 acceptance criteria** ve formátu Given/When/Then s konkrétními hodnotami — eskalační flow (8 stavových přechodů), platby, terminální stavy, pause/resume, manuální advance, konfigurovatelné timeouty, výpočet pracovních dní
- **API kontrakt** v TypeScriptu — veřejná funkce `process(state, event, now)` vrací nový stav a action descriptor; typy pro 12 stavů, 6 událostí, 3 typy akcí; konfigurační interface pro timeouty a svátky
- **Doménový slovník** — 9 pojmů definujících business doménu (dunning, grace period, business days, action descriptor aj.)
- **Out of scope** — 6 explicitně vyloučených oblastí (payment retry, late fees, partial payments, email sending, scheduling, persistence)

Formát acceptance criteria přímo ovlivňuje měřitelnost. Struktura Given/When/Then odděluje výchozí stav, podnět a očekávaný výsledek, což usnadňuje převod jednotlivých požadavků do testovacích scénářů. V této práci z ní proto přímo vycházejí referenční testy (metrika Q2 (referenční test pass rate)) i mapování acceptance criteria na agentem vytvořené testy (metrika Q4 (AC coverage)). Zvolený formát tak neslouží jen k zápisu business požadavků, ale i k jejich konzistentnímu vyhodnocení v rámci experimentu.

Specifikace definuje systém jako pure function: `process(state, event, now)` vrací nový stav a action descriptor místo provádění side effects. Toto rozhodnutí bylo přijato proto, aby bylo možné nad jedním deterministickým rozhraním konzistentně vyhodnocovat referenční testy (metrika Q2) i mutation testing (metrika Q3). Pevné TypeScript signatury současně zajišťují, že stejné testovací prostředí lze použít napříč jednotlivými běhy nezávisle na interní struktuře agentem vytvořeného řešení.

Specifikace obsahuje explicitní sekci out of scope, která vymezuje 6 oblastí mimo zadání. V kontextu této studie tato sekce nevymezuje jen hranice implementace, ale také omezuje riziko, že agent rozšíří řešení nad rámec sledovaného zadání a zkomplikuje tak interpretaci výsledků.

[RAW]

Audit trail — předchozí verze 4.1.1:

Jak vznikla specifikace dunning systému (Issue #1).

Specifikace definuje *co* má agent implementovat. Její konstrukce je součástí provedení experimentu — metodické zdůvodnění výběru projektu viz sekce 3.3.1.

Šablona specifikace. Pro specifikaci byla navržena strukturovaná GitHub issue šablona (YAML form). Výchozí verze měla tři vrstvy inspirované IEEE 830: [enumerate 3 vrstvy] Vrstva Specification byla při revizi odstraněna (zjednodušení na 2 vrstvy). Důvody: [itemize redundancy, context rot, údržba] Výsledná 2-vrstvá šablona: Requirements + Architecture.

Obsah specifikace. [itemize 25 AC, API kontrakt, slovník, behavioral model, out of scope]

Designová rozhodnutí a jejich vliv na experiment.

[itemize behavior-driven AC, infrastructure-agnostic, strict API contract, explicitní out of scope]

4.1.2 Referenční implementace

[DRAFT]

Referenční implementace slouží jako měřicí nástroj: definuje strop kvality proti kterému se porovnávají agentovy běhy. Její role v metrikách popisuje sekce 3.2.2.

Testy byly napsány metodou *spec-first TDD* (sekce 2.1.4): expected values pochází ze specifikace, ne z pozorování kódu.

Výsledná implementace obsahuje 42 behavioral testů pokrývajících všech 25 acceptance criteria (některá kritéria vyžadují více testových případů pro hraniční situace), dosahuje mutation score 91,9 %. Kód kompiluje bez typecheck errors, neobsahuje žádné **any** a má 3 lint warnings (kosmetické, bez vlivu na funkční korektnost). LLM-as-judge (Q8 (design quality)) ohodnotil design quality na 3/3 ve všech dimenzích. Tyto výsledky validují, že specifikace je implementovatelná a že exit kritéria (sekce 3.3.4) jsou dosažitelná lidským vývojářem.

[RAW]

Audit trail — předchozí verze 4.1.2:

Human-expert baseline pro měření kvality.

Referenční implementace slouží jako měřicí nástroj — definuje strop kvality proti kterému se porovnávají agentovy běhy. Podrobnosti o jejím použití v měření viz sekce 3.5 (kap. 3). Zde popisujeme *jak* vznikla.

Postup. Testy byly napsány metodou spec-first TDD (Mathews & Nagappan, 2024). **Výsledky:** [itemize 42 behavioral testů, mutation score, typecheck, Q8]

Referenční implementace validuje, že specifikace je implementovatelná a že exit kritéria (kap. 3) jsou dosažitelná lidským vývojářem.

4.1.3 Konstrukce baseline instrukcí

[DRAFT]

Baseline verze `AGENTS.md` byla konstruována čistě z literatury. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních.

Konstrukce proběhla ve třech krocích: smazání předchozí ad-hoc verze, návrh struktury podle pořadí komponent, které Mao et al. (Mao et al., 2025) zjistili v analýze 2 163 produkčních šablon, a mapování tří dimenzí chování (sekce 3.2) na konkrétní instrukce. Mao identifikují sedm typů komponent (Role, Directive, Context, Workflow, Output, Constraints, Examples); naše baseline sekce (Role, Goal, Specification, Environment, Process, Package Quality, Constraints) jsou adaptací tohoto rámce na doménu případové studie.

Mao et al. zjistili, že v produkčních šablonách se Role a Directive nejčastěji objevují na začátku dokumentu, zatímco Constraints na konci. Toto pozorované pořadí tvoří páteř baseline dokumentu.

Sekce Package Quality definuje očekávání na kvalitu kódu: modulární struktura (typy, business logika a veřejné API v oddělených souborech), striktní typový systém bez obcházení (`any`), JSDoc dokumentace na exportech a jasně definovaný vstupní bod (`index.ts` re-exportuje pouze veřejné API). Tyto požadavky přímo ovlivňují metriky kvality kódu (Q5–Q8). Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost instrukčního souboru je spojena s nižší mediánovou dobou běhu (−28,6 %), ačkoli účinnost jednotlivých typů obsahu dosud nebyla izolována.

Procesní sekce obsahuje stručné instrukce pro spec-first TDD (Mathews & Nagappan, 2024), dekompozici do sub-issues, branch-per-issue a conventional commits. Li et al. (Li et al., 2025) ukazují, že procedurální guidance (“jak pracovat”) je efektivnější než popisná dokumentace (“jak vypadá codebase”). Z procesní sekce byly odstraněny bash příklady, protože redundantní obsah zvyšuje inference cost bez benefitu (Gloaguen et al., 2025).

Sekce Constraints obsahuje explicitní zákazy: nekombinovat issues, nemodifikovat již existující testy, nepřepisovat git historii. Mao et al. (Mao et al., 2025) zjistili, že exclusion constraints (46 %) jsou nejčastějším typem omezení v reálných šablonách (46 % výskytů).

Výsledný dokument má 53 řádků a ~350 slov. Tabulka 4.1 ukazuje mapování jednotlivých sekcí na metriky a obrázek 4.1 uvádí kompletní znění.

Baseline AGENTS.md

```
# AGENTS.md --- Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture,
spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular,
documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria --- every one must be satisfied and tested
- API Contract --- exact TypeScript signatures to export
- Domain Glossary --- use these terms in code and documentation
- Out of Scope --- do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues --- one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR --- no type errors allowed

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture --- separate files for types, business logic, utilities, and public API re-exports. No
single-file implementations.
- Strict TypeScript --- no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation --- JSDoc on all exported functions and types. Inline comments for non-obvious domain logic
(e.g., business day calculations, escalation thresholds).
- Clean API surface --- `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec --- fix the code, not the test.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.1: Baseline AGENTS.md (pilot-r1) — kompletní znění instrukcí použitých jako výchozí bod pilotních iterací.

Tabulka 4.1: Mapování sekcí baseline AGENTS.md na metriky

Cíl	Sekce AGENTS.md	Metriky
Spec-first	Specification, Process krok 2	P1
Strukturovaný workflow	Process kroky 1–4	P2, P3, P4
TDD	Process krok 2, Constraints	P3, P5
Modulární kód	Package Quality	Q5, Q7, Q8
API kompatibilita	Specification (API Contract)	Q1
Funkční korektnost	Specification (AC)	Q2, Q4

Specifikace, referenční implementace a baseline instrukce tvoří výchozí bod experimentu. Následující sekce popisuje průběh pilotních iterací cyklem Spuštění/Měření/Diagnóza/Úprava (sekce 3.3.4).

[RAW]

TODO (2026-03-16) — zmínit metodiku přidání architektury:

V sekci o konstrukci baseline instrukcí zmínit jak a proč byla přidána sekce Package Quality. **Pozn.:** Lulla et al. (Lulla et al., 2026) měří pouze přítomnost vs nepřítomnost AGENTS.md, neizolují efekt jednotlivých typů obsahu. Nelze citovat jako důkaz že “architektura + konvence = nejúčinnější”. Popsat rozhodovací proces: proč právě tyto požadavky (modulární struktura, striktní typy, JSDoc), jak to mapuje na metriky Q5–Q8.

Audit trail — předchozí verze 4.1.3:

Konstrukce výchozí verze AGENTS.md na základě empirických frameworků.

Na rozdíl od přístupu “napsat instrukce a iterovat” byla baseline verze konstruována čistě z literatury, bez vlivu předchozích experimentálních běhů. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních.

Postup konstrukce: [enumerate smazání, FSE order, mapování] **Klíčová rozhodnutí:** [itemize FSE component order, Package Quality, zkrácení workflow, procesní instrukce, constraints]

Výsledný dokument: 53 řádků, ~350 slov.

Mapování na dimenze chování: [tabulka cíl/sekce/metriky]

Tři artefakty tvoří výchozí bod experimentu. Následující sekce popisuje iterační cyklus Spuštění/Měření/Diagnóza/Úprava.

Audit trail — obsah zrušené sekce 4.2.1 (Baseline AGENTS.md):

Baseline instrukce (konstruované v sekci 4.1.3) jsou výchozím bodem pilotní iterace. Dokument

má 7 sekcí (Role, Goal, Specification, Environment, Process, Package Quality, Constraints), 53 řádků a ~350 slov. Každá sekce odpovídá empiricky zdůvodněné komponentě (Mao et al., 2025). Kompletní znění viz přílohu C.

4.2 Pilotní iterace

[DRAFT]

Každá iterace sleduje cyklus Spuštění/Měření/Diagnóza/Úprava (sekce 3.3.4). U prvního běhu (baseline) uvádíme kompletní tabulku metrik a podrobnou diagnostiku. U dalších běhů popisujeme pouze změny oproti předchozí iteraci; souhrnný přehled všech běhů obsahuje sekce 4.4.

4.2.1 Pilot-r1: baseline

[DRAFT]

Baseline ukázal, že agent zvládne napsat fungující kód, ale nedodržuje předepsaný pracovní postup. První běh s baseline instrukcemi (příloha C), agent Minimax přes OpenCode, jedna session bez restartů. Tabulka 4.2 shrnuje naměřené metriky.

Tabulka 4.2: Pilot-r1: naměřené metriky

Kód	Metrika	Hodnota	Kritérium	Splněno?
P1	Issues before code	✓	splněno	✓
P2	Branch per issue	×	branches $\geq$ issues	×
P3	Test-first commits	×	test: před feat:	×
P4	PRs linked to issues	✓	všechny PR	✓
P5	Existující testy nezměněny	×	0 změn	×
P6	Commit msg quality	2/3	$\geq 2/3$	✓
P7	Issue quality	2/3	$\geq 2/3$	✓
P8	PR quality	3/3	$\geq 2/3$	✓
Q1	API contract match	match	match	✓
Q2	Ref. test pass rate	39/42	42/42	×
Q3	Mutation score	84 %	≥ 70 %	✓
Q4	AC coverage (agentovy testy)	23/25	25/25	×
Q5	Lint warnings	2	0	×
Q6	Typecheck errors	0	0	✓
Q7	Složitost kódu	2 viol.	0 viol.	×
Q8	Design quality	1/3	$\geq 2/3$	×
E1	Vstup / výstup / Σ cache (tis.)	115 / 60 / 11528	—	záznam
E2	Trvání	32,7 min	—	záznam
E3	Kompakce kontextu	0	—	záznam

Ze 10 deterministických metrik splnil agent 4 (P1, P4, Q1, Q6).

Chování agenta.

Agent vytvořil 11 GitHub issues a rozložil implementaci do 4 pull requestů, každý s vlastní branch. Zvolil modulární strukturu (4 soubory: `types.ts`, `businessDays.ts`, `dunning.ts`, `index.ts`), což odpovídá požadavkům sekce Package Quality v instrukcích. Napsal 59 vlastních testů, všechny procházejí.

Diagnostika.

Tři procesní metriky z pěti nebyly splněny: agent sloučil 11 issues do 4 branches (P2), nepoužil oddělené commity pro testy a kód (P3) a upravil vlastní test aby seděl na implementaci (P5). Ve všech třech případech agent vybral rychlejší cestu místo té předepsané.

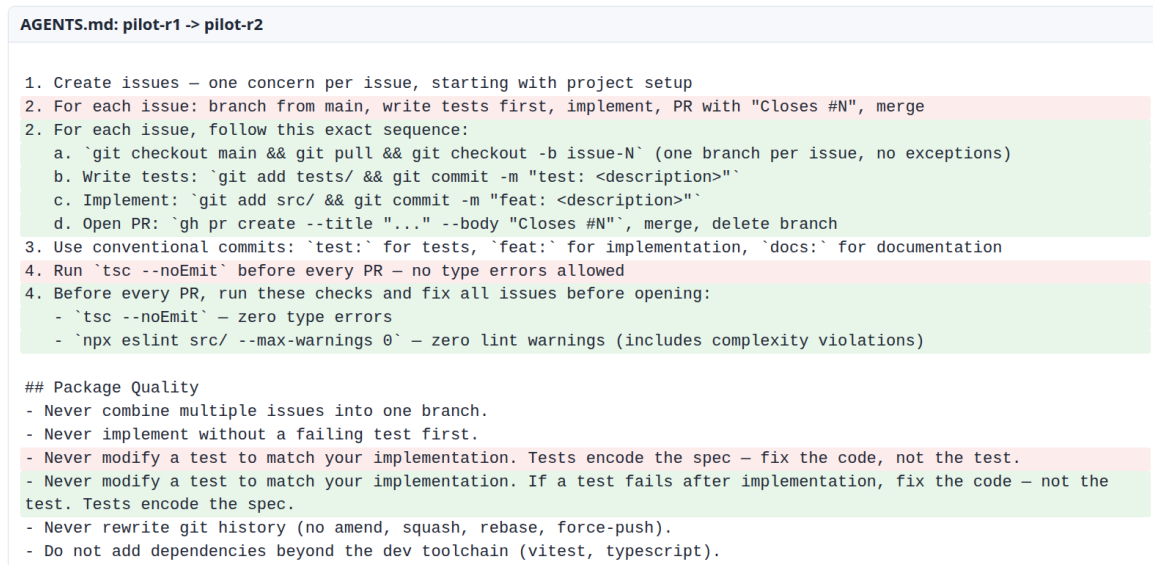
Z referenčních testů selhaly 3 z 42 (Q2), převážně v pause/resume logice a výpočtu pracovních dní. Q8 = 1/3 kvůli chybějící JSDoc dokumentaci na veřejném API; ostatní dimenze dosáhly 3/3.

Změna instrukcí pro r2.

Čtyři selhání sdílela společný vzorec: instrukce říkala *co* dělat (např. “jedna branch per issue”), ale neukazovala *jak* to provést. Breunig (Breunig, 2025) ukazuje, že když agent pravidlo ignoruje, nepomůže ho zopakovat — je třeba ho přepsat tak, aby se stal součástí pracovního postupu. Razavi a Fard (Razavi & Fard, 2025) dokládají, že drobné změny formulace promptu mohou dramaticky změnit chování modelu, což motivuje cílené přestrukturování místo přidávání dalších pravidel. Každé selhání proto dostalo vlastní opravu:

- P2 (agent kombinoval branches): do pracovního postupu přidán příkaz `git checkout -b issue-N` na začátku každého cyklu — agent tak dostane konkrétní krok místo obecného pravidla
- P3 (agent nepsal testy zvlášť): testy a kód se nově odevzdávají odděleně — nejdřív `git add tests/`, pak `git add src/`
- P5 (agent upravoval vlastní testy): doplněno vysvětlení — pokud test po implementaci selhává, opravit kód, ne test
- Q7 (příliš složité funkce): přidána kontrola `npx eslint src/ --max-warnings 0` před každým pull requestem — agent tak vidí složitost dřív než odevzdá kód

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r1 a r2 ukazuje rozsah těchto změn.



Obrázek 4.2: Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2

[RAW]

Audit trail — předchozí verze pilot-r1:

Původní RAW verze s P1=3/6 (6 položek), P1.2/P1.3/P1.5 submetrikami, description list diagnostikou. Přepočteno na P1=2/5 (5 položek) po sladění s kap03.

4.2.2 Pilot-r2

[DRAFT]

Dvě opravy zabraly (P5, Q7), ale dvě klíčové slabiny přetrvaly (P2, P3) a kvalita kódu se zhoršila (Q2, Q3). Druhý běh ověřoval čtyři změny instrukcí. Tabulka 4.3 ukazuje metriky kde došlo ke změně.

Tabulka 4.3: Pilot-r2: metriky s změnou oproti r1

Kód	Metrika	r1	r2	Trend
P5	Existující testy nezměněny	×	✓	opraveno
Q2	Ref. test pass rate	39/42	32/42	horší
Q3	Mutation score	84 %	68 %	pod prahem
Q5	Lint warnings	2	1	zlepšení
Q7	Složitost kódu	2 viol.	0	opraveno
Q8	Design quality	1/3	1/3	beze změny
E1	Vstup / výstup / Σ cache (tis.)	115 / 60 / 11528	76 / 41 / 3196	záznam
E2	Trvání	32,7 min	37,2 min	záznam

Diagnostika.

P5 a Q7 opraveny, lint warnings sníženy na 1. P2 a P3 přetrvala, ale jinak než v r1: v r1 agent psal kód bez testů, v r2 naopak sloučil testy a kód do jednoho commitu s prefixem `test:`. Formálně konvenci splnil (commit má správný prefix), ale záměr — mít oddělený commit pro test a pro kód — nesplnil. Q8 = 1/3 zůstala beze změny, přestože instrukce požadovala JSDoc dokumentaci. Samotný textový požadavek nestačí; agent potřebuje kontrolní krok, který ho donutí dokumentaci ověřit (Breunig, 2025).

Q2 a Q3 se zhoršily (32/42 resp. 68 %). Agent napsal víc testů než v r1 (70 vs. 59), ale horší kvality — pravděpodobně proto, že psal všechny testy najednou ze specifikace místo postupného test-fix cyklu. Při součtu tokenů přes všechny turny by srovnání zkreslovalo opakované počítání kontextu. E1 uvádíme jako vstupní a výstupní mezisoučty (viz kap. 3.2.3), oba v tisících. Mezi r1 a r2 **klesá max. vstup na kroku** (včetně `cache.read` v exportu, viz kap. 3.2.3) z 115 na 76 tis. tokenů; součet výstupu klesá z 60 na 41 tis. — změna instrukcí tedy v E1 souvisí se špičkou promptové strany v jednom requestu i s objemem generování (pole `tokens.total` v exportu mezi běhy nepoužíváme jako jednotnou bázi).

Změna instrukcí pro r3.

Z r2 přetrvaly tři slabiny. Každá dostala vlastní opravu:

- P2 (agent stále kombinoval branches): pracovní postup nově začíná příkazem `gh issue list -state open` — agent tak vidí další issue a ví odkud začít, místo aby si vymýšlel vlastní pořadí
- P3 (agent stále nepsal testy zvlášť): mezi commit s testy a commit s kódem přibyl kontrolní příkaz `git log -oneline -3` — agent tak musí ověřit, že test commit skutečně existuje dřív než začne implementovat
- Q8 (agent ignoroval požadavek na JSDoc): instrukce říkala “dokumentuj veřejné API”, ale agent to přehlížel. Zopakovat totéž by nepomohlo (Breunig, 2025). Požadavek byl proto přesunut do kontrolního kroku před pull requestem — agent musí ověřit JSDoc dřív než odevzdá kód. Přesun požadavku do kontrolního kroku odpovídá vzorci z r1→r2: obecné pravidlo → konkrétní příkaz v místě akce (Breunig, 2025)

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r2 a r3 ukazuje rozsah těchto změn.

AGENTS.md: pilot-r2 -> pilot-r3

```

1. Create issues – one concern per issue, starting with project setup
2. For each issue, follow this exact sequence:
  a. `git checkout main && git pull && git checkout -b issue-N` (one branch per issue, no exceptions)
  a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Implement: `git add src/ && git commit -m "feat: <description>"`
  d. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
  c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
  d. Implement: `git add src/ && git commit -m "feat: <description>"`
  e. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Before every PR, run these checks and fix all issues before opening:
  - **Modular architecture** – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
  - **Strict TypeScript** – no `any`, explicit return types on public API, `readonly` where applicable.
  - **Documentation** – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).
  - **Documentation** – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
  - **Clean API surface** – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

  - Never implement without a failing test first.
  - Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
  - Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
  - Never rewrite git history (no amend, squash, rebase, force-push).
  - Do not add dependencies beyond the dev toolchain (vitest, typescript).

```

Obrázek 4.3: Vizuální diff změn AGENTS.md mezi pilot-r2 a pilot-r3

4.2.3 Pilot-r3

[DRAFT]

Průlomový běh: poprvé $P1-P5 = 5/5$ a $Q8 = 3/3$. Třetí běh ověřoval tři změny instrukcí z r2: kontrolní příkaz po commitu s testy, výběr issue přes `gh issue list` a přesun požadavku na JSDoc do kontrolního kroku před pull requestem. Tabulka 4.4 ukazuje metriky s výraznou změnou.

Tabulka 4.4: Pilot-r3: metriky s změnou oproti r2

Kód	Metrika	r2	r3	Trend
P2	Branch per issue	×	✓	opraveno
P3	Test-first commity	×	✓	opraveno
P6	Kvalita commit zpráv	2/3	3/3	zlepšení
P7	Kvalita issue popisů	2/3	3/3	zlepšení
Q2	Ref. test pass rate	32/42	41/42	zlepšení
Q3	Mutation score	68 %	71 %	nad prahem
Q8	Design quality	1/3	3/3	průlom
E1	Vstup / výstup / Σ cache (tis.)	76 / 41 / 3196	62 / 30 / 4770	záznam
E2	Trvání	37,2 min	24,8 min	záznam
E3	Kompakce kontextu	0	1	záznam

AGENTS.md: pilot-r3 (nejlepší pilotní iterace)

```
# AGENTS.md – Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture, spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular, documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria – every one must be satisfied and tested
- API Contract – exact TypeScript signatures to export
- Domain Glossary – use these terms in code and documentation
- Out of Scope – do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues – one concern per issue, starting with project setup
2. For each issue, follow this exact sequence:
  a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
  d. Implement: `git add src/ && git commit -m "feat: <description>"`
  e. Open PR: `gh pr create --title "... " --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript – no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
- Clean API surface – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.4: Verze AGENTS.md po pilot-r3 — kompletní znění po iteracích r1→r2→r3.

Diagnostika.

Všechny tři změny zabraly. Agent poprvé splnil celý procesní checklist P1–P5 = 5/5: vytvořil issues před kódem, každý issue dostal vlastní branch, test commit předcházet kódu, pull requesty obsahovaly `Closes #N` a žádný test nebyl dodatečně upraven. Přepsání obecných pravidel na konkrétní příkazy (Breunig, 2025) a jejich zasazení do pracovního postupu jako kontrolní kroky funguje lépe než textové požadavky. Q8 = 3/3 to potvrzuje: když musel agent ověřit JSDoc před odevzdáním kódu, dokumentaci napsal.

Q2 = 41/42: zbývá jedna chyba v implementaci. Specifikace definuje chování pause/resume jednoznačně: acceptance criteria vyžadují “elapsed time are preserved” při pauzování a “timeout resumes from where it left off” při obnovení. API kontrakt navíc poskytuje pole `pausedElapsed` s komentářem “business days elapsed before pause”. Agent přesto zvolil vlastní přístup (posunutí `stateEnteredAt` zpět v čase) místo použití pole z kontraktu. Agentův přístup fungoval pro jedno pozastavení, ale selhal při opakovaném pause/resume. Spec není nejednoznačná — agent učinil implementační rozhodnutí navzdory explicitnímu poli v kontraktu.

Změna instrukcí pro r4.

Diagnostika r3 odhalila dva problémy, každý s vlastní opravou:

- Procesní hygiena: pre-PR checklist obsahoval kontrolu typů a lintu, ale ne spuštění agentových vlastních testů. Přidán příkaz `npx vitest run` jako kontrolní krok — agent musí ověřit, že jeho vlastní testy prošly dříve než otevře pull request. Tato změna necílí na Q2 (referenční testy, které agent nevidí), ale na základní ověření konzistence agentova kódu s jeho vlastními testy
- Agent nedodržel API kontrakt: spec definuje pole `pausedElapsed` v typu `DunningState`, ale agent ho nepoužil a vymyslel vlastní řešení. Do Constraints přidáno pravidlo: každé pole v API kontraktu musí být v implementaci použito (Breunig, 2025)

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r3 a r4 ukazuje obě změny: nová položka v pre-PR checklistu a nové pravidlo v Constraints.

```
AGENTS.md: pilot-r3 -> pilot-r4

- `tsc --noEmit` – zero type errors
- `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)
+ `npx vitest run` – zero test failures

## Package Quality
- Never implement without a failing test first.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
+ Every field in the API Contract types must be used in your implementation. If the spec defines a field (e.g., `pausedElapsed`), your code must read and write it – do not invent an alternative approach.
- Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
- Never rewrite git history (no amend, squash, rebase, force-push).
```

Obrázek 4.5: Vizuální diff změn `AGENTS.md` mezi pilot-r3 a pilot-r4

4.2.4 Pilot-r4

[DRAFT]

Regrese oproti r3 — další změny instrukcí nepomohly a nedeterminismus modelu přidal nové chyby. Čtvrtý běh ověřoval dvě úpravy: přidání `npv vitest run` do pre-PR checklistu a pravidlo o dodržení API kontraktu (každé pole v typech musí být použito v implementaci). Tabulka 4.5 ukazuje metriky s nejvýraznější změnou.

Tabulka 4.5: Pilot-r4: metriky s změnou oproti r3

Kód	Metrika	r3	r4	Trend
P2	Branch per issue	✓	×	regrese
P5	Testy nezměněny	✓	×	regrese
P8	Kvalita PR popisů	3/3	1/3	regrese
Q2	Ref. test pass rate	41/42	39/42	horší
Q3	Mutation score	71 %	66 %	pod prahem
Q8	Design quality	3/3	2/3	horší
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	81 / 36 / 5996	záznam
E2	Trvání	24,8 min	25,9 min	stabilní
E3	Kompakce kontextu	1	0	záznam

Diagnostika.

R4 je regrese oproti r3. Agent vytvořil 7 issues ale zpracoval jen 5 — dva issues zůstaly bez branch (P2 = 4/6). Upravil dva vlastní testové soubory po implementaci (P5). PR popisy obsahovaly jen “Closes #N” bez popisu změn (P8 = 1/3).

Q2 se zhoršilo z 41/42 na 39/42: k přetrvávajícímu elapsed time bugu přibyly dvě nové chyby (výpočet víkendů v eskalaci a boundary DUE_SOON přechodu). Pravidlo o dodržení API kontraktu nepomohlo — agent pole `pausedElapsed` opět nepoužil. Agentův vlastní test navíc obsahoval timezone bug a přesto agent mergoval kód, přestože pre-PR checklist zahrnoval `npv vitest run`.

R4 ukazuje dvě věci: (1) deklarativní pravidla v Constraints — pravidla která říkají *co má platit*, např. “každé pole musí být použito” — nepřinesla zlepšení, ačkoli v předchozích iteracích procedurální verifikace v Process — konkrétní příkazy které říkají *co udělat*, např. “spuštění `tsc -noEmit`” — fungovala; (2) nedeterminismus modelu způsobuje, že stejné instrukce v dalším běhu přinesou nové chyby. Obě změny r3→r4 byly deklarativní, zatímco úspěšné opravy r1→r2→r3 byly procedurální (konkrétní příkazy s verifikací).

Zbývající selhání r3 nejsou důsledkem nejednoznačné specifikace: acceptance criteria jednoznačně vyžadují zachování elapsed time při pause/resume a API kontrakt poskytuje pole `pausedElapsed` pro tento účel. Agent pole v kontraktu vidí a správně ho zapisuje při pauzo-

vání, ale při obnovení ho nepoužije a zvolí alternativní přístup. $Q5 = 1$ je způsobena složitostí funkce `processTick` (cyklomatická složitost 15, práh 10), kterou agent nerozložil. Obě selhání jsou kandidáty na procesní verifikaci — stejný vzorec, který opravil P2, P3 a Q8 v předchozích iteracích.

Změna instrukcí pro r5.

Návrat k předchozí verzi instrukcí po neúspěšné iteraci je standardní postup — iterativní cyklus nepředpokládá monotónní zlepšování (Peppers et al., 2008). R5 proto vychází z r3 (ne z r4) a cílí na dvě zbývající selhání procedurální verifikací. Obě změny používají stejný vzorec, který opravil P2, P3 a Q8 v předchozích iteracích: místo deklarativního pravidla přidáváme verifikační akci do pre-PR checklistu.

- Q2 (agent nepoužil `pausedElapsed`): do pre-PR checklistu přidán krok “verify that every field in the API Contract types is read and written in your implementation — if a field exists in the type but is unused, fix it before opening the PR.” Na rozdíl od r4, kde pravidlo bylo v Constraints (deklarativní), je verifikace v Process (procedurální) (Breunig, 2025)
- Q5/Q7 (složitost `processTick`): agent v r3 spustil eslint s vlastním configem, který neobsahoval pravidlo `complexity` — dostal nula warningů a kód považoval za hotový. Experiment měří Q5/Q7 s fixním configem kde `complexity` pravidlo je. Do instrukcí přidán explicitní požadavek: “your ESLint config must include `complexity: [warn, 10]`”

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r3 a r5 ukazuje obě změny: rozšířený pre-PR checklist o API contract verifikaci a explicitní `complexity` pravidlo.

```
AGENTS.md: pilot-r3 -> pilot-r5

4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)
  - `npx eslint src/ --max-warnings 0` – zero lint warnings. Your ESLint config must include `"complexity": ["warn", 10]`.
  - Verify API contract compliance: every field in the API Contract types (e.g. `pausedElapsed`, `pausedFrom`) must be read and used in your implementation logic – not just written. If a field is defined but never read, fix it before opening the PR.

## Package Quality
```

Obrázek 4.6: Vizuální diff změn `AGENTS.md` mezi `pilot-r3` a `pilot-r5`

4.2.5 Pilot-r5: verifikace kontraktu

[DRAFT]

Regrese horší než r4 — agent kompletně ignoroval workflow. Agent přečetl `AGENTS.md`

(v transcriptu potvrzeno), ale místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat: commitu` bez issues, branches a pull requestů.

Tabulka 4.6: Pilot-r5: metriky s změnou oproti r3

Kód	Metrika	r3	r5	Trend
P2	Branch per issue	✓	×	regrese
P3	Test-first commity	✓	×	regrese
P4	PRs linked	✓	×	regrese
Q2	Ref. test pass rate	41/42	38/42	horší
Q5	Lint warnings	1	0	opraveno
Q7	Složitost kódu	1 viol.	0	opraveno
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	65 / 23 / 3028	záznam
E2	Trvání	24,8 min	13,2 min	záznam
E3	Kompakce kontextu	1	0	záznam

Diagnostika.

Instrukce r5 se lišily od r3 pouze ve dvou řádcích pre-PR checklistu (API contract verifikace a ESLint complexity config). Obě cílené opravy zabraly: $Q5 = 0$ a $Q7 = 0$. Agent konfiguroval ESLint s pravidlem `complexity` a rozložil funkci `processTick`. Opravu Q5/Q7 tak způsobila jediná konkrétní instrukce: “your ESLint config must include `complexity: [warn, 10]`”. V r3 agent toto pravidlo ve svém ESLint configu neměl — spustil eslint, dostal nula warningů a kód považoval za hotový. Experiment přitom měří Q5/Q7 s fixním configem kde pravidlo je. Agent nemůže opravit problém, o kterém neví — explicitní požadavek na konkrétní config tento disconnect odstranil.

Přesto agent ignoroval celý Process workflow — žádné issues, branches ani pull requesty. Analýza transcriptu ukazuje posloupnost: agent přečetl instrukce (zpráva 4: “I need to decompose the spec into focused GitHub issues”), v následujícím kroku však místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat: commitu`. Pre-PR checks (tsc, eslint) přitom dodržel. Agent tedy selektivně plnil instrukce: jednokrokové příkazy s okamžitým výstupem (“spuště eslint”) ano, víceukrokový workflow (“vytvoř issue, pak branch, pak test, pak implementuj, pak PR”) ne.

Srovnání r3, r4 a r5 ukazuje, že tento vzorec není náhodný. R3 agent dodržel celý workflow ($P1-P5 = 5/5$). R4 dodržel částečně ($\sim 3/5$) — vytvořil issues ale ne pro všechny branch. R5 workflow ignoroval zcela ($\sim 1/5$). Instrukce se přitom mezi r3 a r5 liší minimálně (2 řádky v pre-PR checklistu). Nedeterminismus modelu je dominantní faktor pro dodržování víceukrokových sekvencí.

Závěr pilotních iterací.

Pět iterací (r1–r5) přineslo čtyři pozorování:

1. **Verifikační kroky fungují spolehlivě.** Jednokrokové příkazy s okamžitým výstupem (`tsc`, `eslint`, `git log -oneline -3`) agent dodržoval ve všech bězích kde byly přítomny (r3–r5). Cílená oprava Q5/Q7 v r5 (explicitní ESLint complexity config) zabrala i přes kolaps workflow.
2. **Multi-step workflow je nedeterministický.** Vícekroková sekvence (issue → branch → test → implement → PR) se pohybovala od 5/5 (r3) přes ~3/5 (r4) po ~1/5 (r5) při téměř identických instrukcích.
3. **Deklarativní pravidla nepomáhají, procedurální ano.** R4 přidalo deklarativní pravidlo v Constraints (“každé pole musí být použito”) — nepomohlo. R5 přidalo procedurální check (“ESLint config musí obsahovat complexity”) — zabralo. Substituční cyklus r1 → r3 potvrzuje: obecná pravidla → konkrétní příkazy → verifikační kroky = zlepšení.
4. **Implementační rozhodnutí instrukce obtížně ovlivňují.** Agent v r3 nepoužil pole `pausedElapsed` z API kontraktu přesto, že specifikace jednoznačně vyžaduje zachování elapsed time a kontrakt pole poskytuje. Agent pole správně zapisoval při pauzování, ale při obnovení zvolil vlastní přístup (Q2 = 41/42). Deklarativní pravidlo (r4) ani procedurální check (r5) toto selhání neopravily — na rozdíl od Q5/Q7, kde procedurální check zabral.

Pilotní baseline pro komparativní variace zůstává r3 — jediný běh kde agent workflow dodržel. Komparativní variace testují, které z těchto pozorování vydrží cílenou ablaci.

Obrázek 4.7 ukazuje souhrnnou evoluci instrukcí od baseline k nejlepší iteraci — bílé řádky zůstaly z r1, zelené byly přidány během iterací.

AGENTS.md: pilot-r1 -> pilot-r3 (souhrnná evoluce)

```
# AGENTS.md – Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture,
spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular,
documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria – every one must be satisfied and tested
- API Contract – exact TypeScript signatures to export
- Domain Glossary – use these terms in code and documentation
- Out of Scope – do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues – one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
2. For each issue, follow this exact sequence:
  a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
  d. Implement: `git add src/ && git commit -m "feat: <description>"`
  e. Open PR: `gh pr create --title "... " --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR – no type errors allowed
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript – no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
- Clean API surface – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec – fix the code, not the test.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.7: Souhrnná evoluce AGENTS.md: pilot-r1 → pilot-r3. Zelené řádky = přidáno, červené = odebráno.

4.3 Komparativní variace

[DRAFT]

Pilotní iterace demonstrovaly, jak se instrukce navrhují: substituční cyklus (obecná pravidla → konkrétní příkazy → kontrolní kroky) zlepšil procesní i produktové metriky. Zbývá otázka, které složky výsledných instrukcí jsou pro chování agenta nezbytné a které lze odebrat bez dopadu.

Existující studie měří efekt souboru `AGENTS.md` jako celku: Lulla et al. (Lulla et al., 2026) zjistili, že jeho přítomnost je spojena s 28,6 % nižším mediánovým runtimem, Gloaguen et al. (Gloaguen et al., 2025) zjistili marginální zlepšení úspěšnosti (+4 %). Které složky souboru k tomuto efektu přispívají, dosud nebylo izolováno — oba autoři to označují za otevřenou otázku. Naše ablace tuto mezeru částečně vyplňují na úrovni případové studie.

4.3.1 Výběr složek pro ablaci

[DRAFT]

Soubor `AGENTS.md` obsahuje sedm sekcí (Role, Goal, Specification, Environment, Process, Package Quality, Constraints). Pro účely výběru ablaci je rozlišujeme na sekce definující *co* implementovat (Role, Goal, Specification, Environment) a sekce definující *jak* pracovat a jakou kvalitu produkovat (Process, Package Quality, Constraints). Odebrání kontextových sekcí by testovalo jinou otázku — zda agent dokáže kontext projektu získat z jiných zdrojů (issues, README, kód) — a nesouvisí s naší výzkumnou otázkou o vlivu instrukcí na proces a kvalitu (viz Závěr).

Constraints obsahují zákazy které agent v pilotu dodržoval. Zároveň je pracovní postup v Process implicitně vynucuje — test-first workflow znamená, že agent píše testy před kódem a pak na ně nesahá (P5). Ablace Constraints je námětem pro další výzkum (sekce 5.5).

Zbývají verifikační kroky v Process a sekce Package Quality. U obou agent v pilotu vykazoval stabilní chování, ale nevíme zda je řízeno instrukcemi, nebo znalostmi modelu z tréninku — a to je otázka pro ablaci.

Ablace A: bez verifikačních kroků.

Agent verifikační příkazy (`tsc -noEmit`, `npx eslint`, `git log -oneline -3`) spouštěl spolehlivě ve všech pilotních bězích, včetně r5, kde ignoroval zbytek workflow. Zlepšují ale reálně výstup, nebo by je agent spustil i bez explicitní instrukce?

Co odebíráme: Z Process kroku 4 odebereme `tsc -noEmit`, `npx eslint` a z kroku 2c odebereme `git log -oneline -3`. Workflow struktura (issue → branch → test → implement → PR)

zůstane (obrázek 4.8).

```
AGENTS.md: pilot-r3 -> ablance-a (bez verifikačních kroků)

a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
b. Write tests: `git add tests/ && git commit -m "test: <description>"`
c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
d. Implement: `git add src/ && git commit -m "feat: <description>"`
e. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
c. Implement: `git add src/ && git commit -m "feat: <description>"`
d. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality
```

Obrázek 4.8: Změny AGENTS.md pro ablaci A: odebrány verifikační kroky (`git log`, `tsc`, `eslint`) z Process.¹

Očekávání: Verifikační kroky přímo ovlivňují metriky Q5 (lint warnings) a Q6 (typecheck errors). Pokud agent přesto spustí typecheck a lint → verifikační kroky v instrukcích jsou redundantní. Pokud ne → explicitní příkazy jsou klíčový mechanismus a jejich přítomnost v instrukčním souboru přispívá k efektu, který Lulla et al. měřili na úrovni celého souboru.

Ablance B: bez sekce Package Quality.

Agent konzistentně produkoval modulární kód (4 soubory, strict TypeScript, clean API) napříč všemi pilotními běhy, včetně r5, kde ignoroval celý workflow. Je kvalita kódu řízena sekci Package Quality, nebo jde o znalosti z tréninku modelu?

Co odebíráme: Celou sekci Package Quality (modulární architektura, strict TypeScript, JSDoc dokumentace, čisté veřejné API). Process a Constraints zůstanou beze změny (obrázek 4.9).

```
AGENTS.md: pilot-r3 -> ablance-b (bez Package Quality)

- `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript – no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
- Clean API surface – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints
```

Obrázek 4.9: Změny AGENTS.md pro ablaci B: odebrána celá sekce Package Quality (konvence, strict TypeScript, JSDoc, čisté API).

¹Zelené řádky v diffu neobsahují nový text — vznikly přečíslováním kroků po odebrání kroku c (Verify).

Očekávání: Pokud Q5, Q7 a Q8 zůstanou na úrovni r3 → konvenční instrukce jsou redundantní s tréninkem modelu. Pokud se zhorší → explicitní konvence v instrukcích jsou potřeba a přispívají k celkovému efektu instrukčního souboru.

[RAW]

Audit trail — předchozí verze komparativních variací:

V1: Ablace A = bez Constraints (Mao: exclusion constraints 46 %), Ablace B = bez Package Quality. V2: 7 rozporů v literatuře, Ablace A = bez verifikačních kroků, Ablace B = bez PQ. V3: Nejistoty z pilotních dat + literatura. V4: Oprava citací (Lulla/Gloaguen měří celý soubor, ne složky). Framing: nikdo neizoloval efekt jednotlivých složek, naše ablace tuto mezeru vyplňují. Výběr složek z pilotních dat. V5 (aktuální): Rozpor-based framing nahrazen. Nový argument: pilotní fáze generuje otázky (instrukce vs. model?), ablace na ně odpovídají. Výběr složek: (1) nejistota přínosu, (2) přímý vztah k měřeným dimenzím (proces + kvalita). Kontextové sekce vyloučeny — definují CO, ne JAK. Constraints vyloučeny — hypotéza redundance s workflow (test-first postup implicitně vynucuje zákazy z Constraints). V6: Constraints přeformulovány — nejsou neúčinné (baseline zákazy agent dodržoval), ale pravděpodobně redundantní s Process. Přidáno do future work. Insight: spec-first + test-first workflow zužuje scope na to jak uspět — agent musí napsat testy před kódem, čímž automaticky dodržuje P5 bez explicitního zákazu.

4.3.2 Ablace A: bez verifikačních kroků

[DRAFT]

Tabulka 4.7 shrnuje výsledky dvou běhů (A-1, A-2) ve srovnání s r3.

Tabulka 4.7: Ablace A: srovnání s r3 (bez verifikačních kroků)

Kód	Metrika	r3	A-1	A-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	×
P4	PRs linked to issues	✓	✓	×
P5	Testy nezměněny	✓	×	✓
P6	Commit msg quality	3/3	2/3	2/3
P7	Issue quality	3/3	2/3	3/3
P8	PR quality	3/3	3/3	1/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	35/42	37/42
Q3	Mutation score	71 %	62 %	— [†]
Q5	Lint warnings	1	4	3
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	4	1
Q8	Design quality	3/3	2/3	2/3
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	72 / 32 / 4558	92 / 36 / 8210
E2	Trvání	25 min	39 min	24 min
E3	Kompakce kontextu	1	0	0

[§] Sloupec r3: E1 z pilotního `transcript.json` (62 / 30 / 4770 tis.). U A-1 a A-2 chybí export v lokálním snapshotu — E1 a E3 pro tyto běhy nešly dopočítat.

[†] Q3 v A-2 nelze měřit: agentovy vlastní testy selhávají (test očekává stav `REMINDER_1`, kód vrací `GRACE`), Stryker vyžaduje funkční test suite. Bez verifikačních kroků agent commitl kód s nefunkčními testy.

Interpretace.

E1 u ablace nelze z GitHub repozitáře dopočítat (`transcript.json` v něm není; stejně v lokálním `experiments/runs/ablace-*`). Po doplnění exportu do lokálního snapshotu stejným postupem jako u pilotů by se sloupce A-1 / A-2 vyplnily. Tabulka 4.7 ukazuje pokles napříč metrikami kvality kódu (Q5, Q7) i funkční korektnosti (Q2, Q3). Bez explicitního příkazu `npmx eslint` agent neprovedl kontrolu kvality před odevzdáním kódu, což naznačuje, že verifikační kroky nejsou redundantní — agent je bez instrukce nespouští sám.

Procesní metriky vykazují vzorec známý z pilotních iterací: A-1 dodržel workflow kompletně, A-2 nikoliv. Instrukce se mezi běhy nelišily — variabilita je důsledkem nedeterminismu modelu, ne ablace.

Agent bez verifikačních kroků spotřeboval přibližně dvakrát více testovacích cyklů (44 spuštění `vitest` oproti ~ 20 u ablace B). To naznačuje, že verifikační kroky plní kromě kontroly kvality

i druhou funkci: fungují jako checkpointy které strukturují práci agenta a zabráňují cyklickému debugování.

[RAW]

Audit trail — **ablance A**: Původní RAW placeholder. Data z běhů ablance-a-1 (39 min, 6 PR, 73 testů) a ablance-a-2 (24 min, 5 PR). První pokus A-1 selhal (OpenCode hang), opakován. Druhý pokus A-2 selhal (OpenCode hang), opakován. Třetí pokusy úspěšné.

4.3.3 Ablance B: bez Package Quality

[DRAFT]

Tabulka 4.8 shrnuje výsledky dvou běhů (B-1, B-2) ve srovnání s r3.

Tabulka 4.8: Ablance B: srovnání s r3 (bez sekce Package Quality)

Kód	Metrika	r3	B-1	B-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	✓
P4	PRs linked to issues	✓	×	✓
P5	Testy nezměněny	✓	×	×
P6	Commit msg quality	3/3	3/3	3/3
P7	Issue quality	3/3	3/3	3/3
P8	PR quality	3/3	2/3	2/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	37/42	11/42
Q3	Mutation score	71 %	67 %	72 %
Q5	Lint warnings	1	2	2
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	1	2
Q8	Design quality	3/3	—*	2/3
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	78 / 40 / 6273	55 / 23 / 3604
E2	Trvání	25 min	28 min	21 min
E3	Kompakce kontextu	1	0	0

* Q8 v B-1: judge nedokončil hodnocení (API timeout).

Interpretace.

Stejně jako u tabulky 4.7 chybí u ablance export `transcript.json` v lokálním snapshotu — řádek E1 proto uvádí jen hodnoty pro baseline r3 (62 / 30 / 4770 tis.); u ablačních běhů E1

dopočítat nejde.

Automatizované metriky kvality kódu (Q5, Q6, Q7) zůstaly na podobné úrovni jako v r3 (tabulka 4.8). Agent produkoval modulární kód, striktní TypeScript a v B-1 i JSDoc dokumentaci — přestože instrukce tyto požadavky neobsahovaly. Jediný pokles zaznamenalo Q8 (design quality): judge identifikoval chybějící modularitu (veškerý kód v jednom souboru) a neúplnou dokumentaci. Data naznačují, že základní kódové konvence jsou u tohoto modelu řízeny převážně znalostmi z tréninku; strukturální rozhodnutí (modularita, dokumentace) instrukce pravděpodobně ovlivňují.

Funkční korektnost (Q2) vykazuje extrémní variabilitu: B-1 dosáhl 37/42 (srovnatelné s r3), zatímco B-2 pouze 11/42. Mutation score zůstal stabilní (67–72 %). Propad Q2 v B-2 není důsledkem ablace Package Quality — ta neobsahuje žádné instrukce o implementační logice. Jde o projev nedeterminismu modelu: agent v B-2 implementoval business day výpočty chybně, což způsobilo kaskádové selhání eskalačních přechodů. Stejný typ chyby se objevoval i v pilotních iteracích (r1, r4).

P5 (testy nezměněny) selhalo v obou bězích — agent modifikoval vlastní testy po implementaci. Stejný problém se objevoval i v pilotních iteracích (r1, r4) se stejnými instrukcemi, jde tedy o projev nedeterminismu modelu, ne ablace.

[RAW]

Audit trail — ablace B: Původní RAW placeholder. Data z běhů ablace-b-1 (28 min, 7 PR, 129 testů) a ablace-b-2 (21 min, 4 PR, 32 testů). Oba běhy dokončeny bez problémů.

4.3.4 Závěr komparativní fáze

[DRAFT]

Dvě ablace testovaly dvě složky instrukcí z r3: verifikační kroky v Process (ablace A) a sekci Package Quality (ablace B). Výsledky ukazují odlišný charakter obou složek.

Bez verifikačních kroků (`tsc`, `eslint`) klesly metriky kvality kódu i funkční korektnosti (tabulka 4.7). Agent bez kontrolních bodů navíc spotřeboval přibližně dvakrát více testovacích cyklů. Data naznačují, že verifikační kroky plní dvojí funkci: zajišťují kontrolu kvality a fungují jako checkpointy které strukturují práci agenta.

Bez sekce Package Quality zůstaly automatizované metriky kvality (Q5–Q7) na srovnatelné úrovni (tabulka 4.8), avšak Q8 (design quality) kleslo ve všech ablačních bězích — judge identifikoval chybějící modularitu a neúplnou dokumentaci. Data jsou konzistentní s hypotézou, že základní kódové konvence jsou u tohoto modelu řízeny znalostmi z tréninku, zatímco strukturální rozhodnutí instrukce ovlivňují.

Procesní metriky (P1–P5) vykazovaly v obou ablacích variabilitu srovnatelnou s pilotními

iteracemi (r3 vs. r4/r5). Tato variabilita není důsledkem ablace, ale nedeterminismu modelu: instrukce pro workflow zůstaly v obou ablacích beze změny.

4.4 Souhrnné výsledky

[DRAFT]

Tabulka 4.9 shrnuje deterministické metriky všech běhů: pět pilotních iterací a čtyři ablační běhy. Pilotní sloupce ukazují evoluci instrukcí, ablační sloupce ukazují dopad odebrání jednotlivých složek. Baseline pro srovnání je r3 — jediný pilotní běh kde agent splnil celý procesní checklist. Sloupec E1 uvádí vstup (max. na kroku včetně `cache` v exportu), výstup a součet `cache` přes kroky, vše v tisících; stejný přehled generuje skript `summary.ts` do `experiments/runs/SUMMARY.md`.

Tabulka 4.9: Souhrnné výsledky všech běhů (pilot + ablace)

Metrika	Pilotní iterace					Ablace A		Ablace B	
	r1	r2	r3	r4	r5	A-1	A-2	B-1	B-2
P1 issues before code	✓	✓	✓	✓	? [†]	✓	✓	✓	✓
P2 branch per issue	×	×	✓	×	×	✓	×	✓	×
P3 test-first	×	×	✓	✓	×	✓	×	✓	✓
P4 PRs linked	✓	✓	✓	✓	×	✓	×	×	✓
P5 testy nezměněny	×	✓	✓	×	✓	×	✓	×	×
P6 commit msg	2/3	2/3	3/3	3/3	2/3	2/3	2/3	3/3	3/3
P7 issue quality	2/3	2/3	3/3	3/3	3/3 [‡]	2/3	3/3	3/3	3/3
P8 PR quality	3/3	3/3	3/3	1/3	1/3	3/3	1/3	2/3	2/3
Q1 API contract	match	match	match	match	match	match	match	match	match
Q2 ref. testy	39	32	41	39	38	35	37	37	11
Q3 mutation score	84 %	68 %	71 %	66 %	—	62 %	— [*]	67 %	72 %
Q4 AC coverage	25	25	25	23	23	24	24	24	22
Q5 lint warnings	2	1	1	1	0	4	3	2	2
Q6 typecheck errors	0	0	0	0	0	0	0	0	0
Q7 složitost	2	0	1	1	0	4	1	1	2
Q8 design quality	1/3	1/3	3/3	2/3	2/3	2/3	2/3	2/3	2/3
E1 in/out/cache (tis.)	115/60/11k	76/41/3k	62/30/5k	81/36/6k	65/23/3k	72/32/5k	92/36/8k	78/40/6k	55/23/4k
E2 trvání (min)	32,7	37,2	24,8	25,9	13,2	39	24	28	21
E3 kompakce	0	0	1	0	0	0	0	0	0

[†] P1 v r5 nelze určit — agent nevytvořil issues.

[‡] P7 v r5: judge hodnotil specifikační issue, ne agentovu.

^{*} Stryker v A-2 nedokončil analýzu (timeout).

Q4 v ablacích používá 24 AC (viz sekce 3.5); pilotní běhy přepočteny na 25.

[DRAFT]

Tabulka ukazuje dva vzorce. Za prvé, pilotní iterace potvrzují že instrukce lze iterativně zlepšovat: r1 (baseline) splnila 4/10 deterministických metrik, r3 (po dvou cyklech úprav) splnila 9/10. Za druhé, ablace ukazují že ne všechny složky instrukcí přispívají stejně: odebrání verifikačních kroků (ablace A) zhoršilo 4 metriky (Q2, Q3, Q5, Q7), zatímco odebrání sekce Package Quality (ablace B) ponechalo automatizované metriky kvality na srovnatelné úrovni.

Napříč všemi devíti běhy zůstaly tři metriky stabilní: Q1 (API contract match), Q6 (typecheck errors) = 0 a nízký počet kompaktací kontextu (E3 (kompakce kontextu): 0–1 u pilotů, u ablace neznámé). Nejvyšší variabilitu vykazovaly procesní metriky P2–P5, které kolísaly i mezi běhy se stejnými instrukcemi (r3 vs. r4/r5). Interpretaci těchto výsledků ve vztahu k cílům práce a existující literatuře uvádí kapitola 5.

[RAW]

Kontext pro diskuzi (přesunout do kap05):

- Gloaguen et al. (Gloaguen et al., 2025): generické AGENTS.md mají marginální efekt (+4 % human-written). Náš procedurální scaffolding vykazuje lepší výsledky — evidence pro hypotézu že *typ* instrukcí (procedurální vs. popisné) je klíčovým faktorem.
- Li et al. (Li et al., 2025): curated Skills +16,2pp, self-generated −1,3pp. Naše instrukce jsou human-curated a iterativně vylepšované — odpovídá kategorii “curated”.
- Porovnat E1 (vstup/výstup tokenů) s Gloaguenovým nálezem +20 % cost.

5. Vyhodnocení a diskuse

[DRAFT]

Předchozí kapitola popsala průběh případové studie a naměřená data. Tato kapitola výsledky interpretuje: nejprve ve vztahu ke třem cílům práce (sekce 5.1), poté v kontextu existujícího výzkumu (sekce 5.2) a nakonec diskutuje, jak se metodologická omezení identifikovaná v sekci 3.5 projevila v praxi (sekce 5.3).

[RAW]

Kostra kapitoly — zde bude interpretace výsledků z kap04.

TODO (2026-03-26) — po přeformulaci cílů (PoC framing):

- **Úvod 5.1 (ř. 24–28):** aktualizovat znění cílů — nové formulace: (1) “navrhnout sadu metrik pokrývajících proces, kvalitu a efektivitu”, (2) “demonstrovat iterativní postup... měřitelné zlepšení”, (3) “z instrukcí vytvořených v cíli 2 ablacemi prozkoumat”.
- **5.1.1 Cíl 1:** otázka “zachytily metriky rozdíly?” je příliš slabá (tautologická). Správná otázka: “pokrývá sada dimenze, které výsledkové metriky nepokrývají?” Přidat příklad: r3 na výsledku vypadá dobře (Q2=41/42), ale P2–P5 ukazují nespolehlivý proces. Přidat malou inline tabulku.
- **5.1.2 Cíl 2:** zkontrolovat že odpovídá “měřitelné zlepšení dodržování praktik”, ne “dosažení exit kritérií”.
- **5.1.3 Cíl 3:** “identifikovat” → “prozkoumat”, doplnit propojení na cíl 2 (“z instrukcí vytvořených...”).
- **REVIEW-LAYERS:** 5 zbývajících značek v 5.1–5.2 (interpretace jako fakt, závěr bez pozorování) — opravit hedging.

5.1 Interpretace výsledků

[DRAFT]

Následující tři podsektory hodnotí, do jaké míry případová studie naplnila cíle stanovené v kapitole 1.

[RAW]

Co výsledky znamenají — propojení s cíli práce.

- Které složky scaffoldingu jsou nezbytné a které redundantní?
- Jaký kontext agent potřebuje k dodržování SWE praktik?
- Odpovědi na cíle práce (viz kap01)

5.1.1 Cíl 1: Sada metrik

[DRAFT]

Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývající proces, kvalitu kódu a efektivitu (dimenze, které stávající benchmarky neměří).

Sada 19 metrik byla navržena (kapitola 3) a použita na devíti bězích (kapitola 4). Co se z tohoto použití dozvídáme?

Procesní metriky (P1–P8).

procesní metriky (P1–P8) měří zda agent dodržoval vývojové praktiky (issues, branching, test-first, PR workflow), tedy dimenzi kterou stávající benchmarky pokrývají jen okrajově (sekce 2.3.1). Na devíti bězích se ukázaly jako nejcitlivější složka sady: byly jediné metriky které kolísaly i mezi běhy se stejnými instrukcemi. Možným vysvětlením je, že kvalita kódu je u tohoto modelu řízena převážně tréninkovými daty (Q metriky zůstaly stabilní), zatímco dodržování vícekrokového postupu (issue → branch → test → implement → PR) je inherentně náchylnější k selhání: agent musí udržet sekvenci kroků v kontextu a každý krok je příležitost k odchylce.

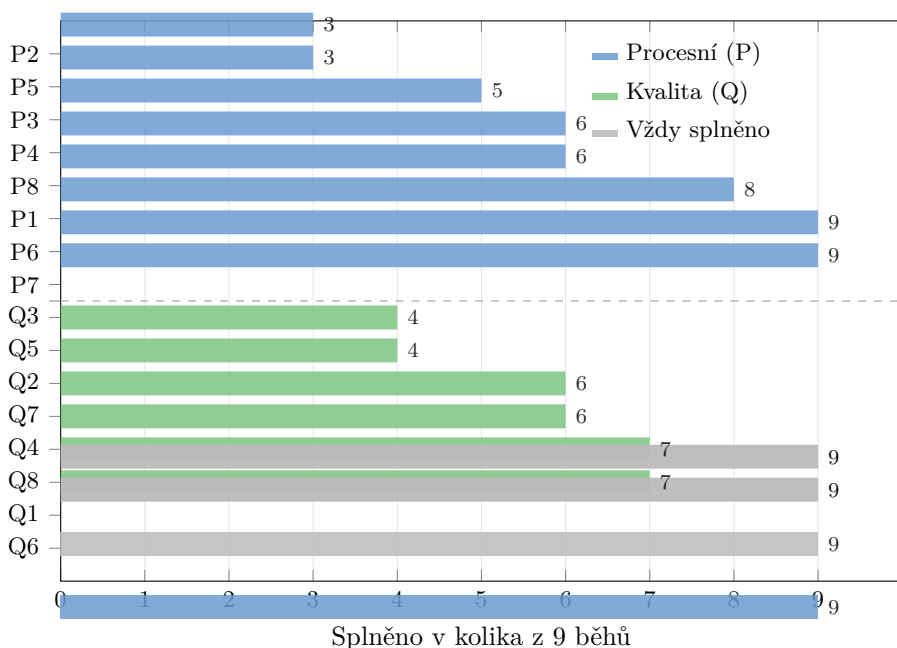
P1–P5 ale říkají jen splněno/nesplněno, což přináší ztrátu nuancí. Například P3 (test-first commits) hlásila v prvních dvou bězích stejný výsledek “nesplněno”, ale z git historie a transcriptu agenta bylo vidět, že se chování lišilo: v prvním běhu agent testy nepsal vůbec, v druhém je psal ale kombinoval s implementací do jednoho commitu. Odpověď ano/ne řekne *co* nefunguje, ale ne *proč*. K diagnostice a návrhu opravy instrukcí bylo vždy nutné doplnit ji o ruční analýzu transcriptu.

Podobně P4 měří *zda* agent vytvořil PR linkovaný na issue, ale ne *jak kvalitní* ten PR je. P6–P8 (kvalita commit zpráv, issue popisů a PR popisů; hodnocené na škále 1–3) tuto mezeru doplňují. V běhu r4 P4 hlásila “splněno” (agent PR vytvořil a linkoval), ale P8 zachytila, že popis obsahoval jen identifikátor bez popisu změn. Binární a kvalitativní metriky tedy měří odlišné aspekty téhož procesu a jejich kombinace se ukázala jako nutná.

Obrázek 5.1 shrnuje variabilitu všech 19 metrik napříč devíti běhy.

Kvalita kódu (Q1–Q8).

Q8 (design quality) v praxi zachytila problémy které automatizované metriky nevidí. V ablačnických bězích Q5 (lint warnings), Q6 (typecheck errors) a Q7 (složitost kódu) zůstaly na srovnatelné úrovni, ale Q8 klesla: agent soustředil kód do jednoho souboru a vynechal dokumentaci. Automatizované metriky tento typ problému ze své podstaty nezachytí.



Obrázek 5.1: Míra splnění jednotlivých metrik napříč devíti běhy. Procesní metriky P2 a P5 byly splněny nejméně často (3/9), zatímco Q1 a Q6 zůstaly stabilní ve všech bězích. Přerušovaná čára odděluje procesní a produktové metriky.

Q2 (referenční test pass rate) a Q3 (mutation score) měří odlišné aspekty korektnosti: Q2 říká zda implementace odpovídá specifikaci, Q3 zda agentovy testy zachytí injektované chyby. V jednom běhu agent dosáhl $Q3 = 72\%$ při $Q2 = 11/42$: napsal kvalitní testy na chybnou implementaci. Zahrnutí obou metrik se proto ukázalo jako opodstatněné.

Q1 (API contract match) a Q6 zůstaly stabilní ve všech bězích. To pravděpodobně souvisí s designem úlohy: specifikace poskytuje explicitní TypeScript typy a agent z tréninku zná konvence veřejného API. Pro složitější API nebo dynamicky typované jazyky by výsledek mohl být jiný.

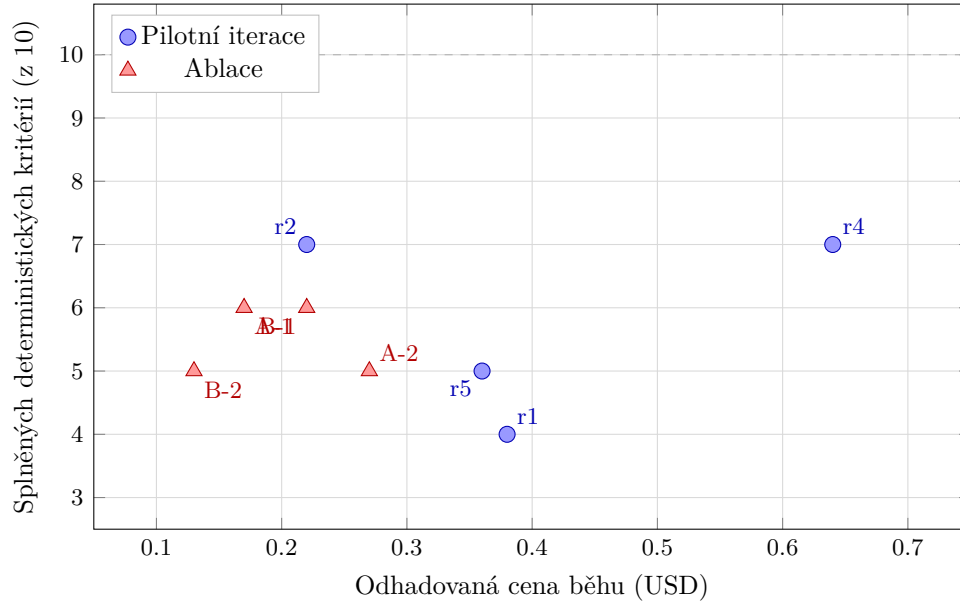
Efektivita (E1–E3).

E1 (vstup/výstup/cache tokeny) ukázala užitečný vzorec: bez verifikačních kroků agent spotřeboval přibližně dvakrát více testovacích cyklů, což se projevilo na vyšší spotřebě tokenů. E2 (trvání) závisí převážně na rychlosti poskytovatele API (rate limits, latency), ne na kvalitě agentovy práce; nejkratší běh (r5, 13 min) vznikl tím, že agent ignoroval celý pracovní postup. E2 je proto nutné číst společně s procesními metrikami.

E3 (kompakce kontextu) nepřinesla na této úloze rozlišení: všechny běhy měly 0–1 kompakcí. Pro rozsáhlejší projekty, kde agent pracuje déle a kontext narůstá, by E3 pravděpodobně rozlišovala lépe.

Při orientačním přepočtu na cenu (ceník poskytovatele Alibaba Cloud Model Studio) se

ukázalo, že vyšší spotřeba tokenů nekoreluje s lepším výsledkem: nejlevnější běh (r3, přibližně \$0,17) dosáhl nejlepších metrik ($Q2 = 41/42$, $P1-P5 = 5/5$), zatímco nejdražší běh (r4, přibližně \$0,64) měl horší výsledky. Agent v r4 spotřeboval víc tokenů na cyklické debugování, ne na kvalitnější práci (obrázek 5.2).



Obrázek 5.2: Vztah mezi odhadovanou cenou běhu (E1) a počtem splněných deterministických kritérií ($P1-P5$, $Q1$, $Q2 \geq 41$, $Q5 \leq 1$, $Q6 = 0$, $Q7 \leq 1$). Běh r3 dosáhl maxima 10/10 při nejnižší ceně z pilotních iterací. Ablací běhy (trojúhelníky) se pohybují v rozmezí 5–6 splněných kritérií.

Limity a reflexe.

Sada se ukázala jako proveditelná pro opakované měření: deterministické metriky se extrahují automatizovaně, kvalitativní metriky hodnotí LLM-as-judge s fixní rubrikou. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace.

Iterativní postup ale optimalizuje instrukce na fixní sadu metrik. Přidání příkazu `npx eslint` do instrukcí zlepší $Q5$, ale to neznamená že agent píše lepší kód, jen že ho zkontroluje. Podobně $Q6 = 0$ ve všech bězích může odrážet vlastnosti TypeScriptu (kompilátor odmítne nevalidní kód), ne kvalitu agentovy práce. Při dalším použití by stálo za úvahu doplnit jemnější škály pro metriky které v této studii nerozlišovaly a rozšířit sadu o transparentnost agentova rozhodovacího procesu, kterou současné $P1-P8$ nepokrývají.

Přenositelná je taxonomie (proces / kvalita / efektivita) a postup měření. Konkrétní prahy a výběr nástrojů (ESLint, Stryker, TypeScript strict mode) jsou vázané na technologii případové studie.

[RAW]

Audit trail — předchozí verze 5.1.1 (DRAFT v1):**Procesní metriky jako hlavní přínos.**

Stávající benchmarky (SWE-bench, HumanEval) měří funkční korektnost: projdou testy, tedy agent úlohu vyřešil. Procesní metriky P1–P5 měří *jak* agent pracoval — zda vytvořil issues, branche, testy před kódem, PR. Tuto dimenzi žádný z existujících benchmarků nepokrývá. Na naší případové studii se ukázala jako informativní: běh r3 dosáhl $Q2 = 41/42$ (funkčně téměř bezchybný), ale procesní metriky v následujících bězích r4 a r5 se stejnými instrukcemi odhalily, že dodržování pracovního postupu je nestabilní. Bez procesních metrik by tyto běhy vypadaly srovnatelně; s nimi vidíme podstatný rozdíl v chování agenta.

Kvalitativní metriky.

Metriky kvality kódu (Q3–Q8) rozšiřují binární pohled benchmarků na hlubší analýzu. Q8 (design quality, LLM-as-judge) identifikovalo chybějící modularitu a dokumentaci v ablačních bězích, přestože automatizované metriky (Q5, Q6, Q7) žádný problém nehlásily. Kombinace automatizovaných a kvalitativních metrik zachycuje víc než kterýkoliv typ sám.

Stabilní a variabilní metriky.

Tři metriky zůstaly stabilní napříč všemi devíti běhy: Q1 (API contract match = shoda), Q6 (typecheck errors = 0) a E3 (kompakce kontextu = 0–1). U Q1 a Q6 lze nabídnout hypotézu proč: Q1 naznačuje, že model z tréninku zná konvenci veřejného API natolik, že ji dodrží i bez explicitní instrukce. Q6 pravděpodobně souvisí s tím, že statické typování v TypeScriptu je pro LLM explicitnější než dynamické jazyky — kompilátor odmítne nevalidní kód dříve, než se projeví v metrice. Alternativním vysvětlením je, že obě metriky mají příliš hrubý práh: Q1 je binární (match/no match — nezachytí drobné odchylky v pojmenování), Q6 počítá chyby kompilace (nezachytí problematické `any` typy). Jemnější měření — například srovnání názvů metod s kontraktem, nebo podíl explicitních typů — by mohlo odhalit rozdíly, které binární metriky nezachytí.

Nejvyšší variabilitu vykazovaly procesní metriky P2–P5. Kolísaly nejen mezi iteracemi s různými instrukcemi, ale i mezi běhy se stejnými instrukcemi (r3: 5/5 vs. r4: $\sim 3/5$ vs. r5: $\sim 1/5$). Tato variabilita pravděpodobně neodráží slabinu metrik, ale podstatnou vlastnost měřeného systému: dodržování víceřadového pracovního postupu je u tohoto modelu nedeterministické. Metriky tento fenomén zachytily, což umožnilo diagnostiku v cíli 2.

Proveditelnost a limity.

Sada se ukázala jako proveditelná pro opakované měření. Deterministické metriky se extrahují automatizovaně, kvalitativní metriky hodnotí LLM-as-judge s fixní rubrikou. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace. Při dalším použití by stálo za úvahu přidat jemnější škálu pro Q1 a Q6 (viz výše) a rozšířit procesní metriky o transparentnost agentova reasoning procesu — naše P1–P8 měří pozorovatelné artefakty (commity, issues, PR), ale ne kvalitu rozhodovacího procesu, který k nim vedl.

[RAW]

TODO (2026-03-26):

- **Přeučení na metriky (overfitting):** Iterativní postup optimalizuje instrukce na fixní sadu metrik. Hrozí, že instrukce se “přeučí” na konkrétní metriky místo aby zlepšily obecné chování. Příklad: přidání `npx eslint` zlepší Q5, ale neznamená že agent píše lepší kód — jen ho zkontroluje. Kam patří: future work nebo limity? Zvážit.
- **E3 (compaction):** HOTOVO — všechny hodnoty validní, label v kap03 opraven (restarts → kompakce).
- **Benchmark research (HOTOVO):** Žádný benchmark neměří všechny 3 dimenze (proces + kvalita + efektivita) společně. Doplnit do 5.1.1 nebo do porovnání s literaturou (5.2):
 - **METR (2026):** polovina SWE-bench passing PRs by nebyla merged — přímo validuje náš argument že pass/fail nestačí. Citovat!
 - **GitGoodBench (2025, JetBrains):** měří kvalitu commitů (LLM-as-judge) — nejbližší našim P6–P8, ale pokrývá jen git operace, ne celý SDLC.
 - **RACE (Zheng 2024):** readability, maintainability, efficiency — blízké našim Q5–Q8, ale function-level.
 - **DevBench (Li 2024):** 5 SDLC fází včetně design quality — částečně proces, ale implementation je stále pass/fail.
 - **MAESTRO (2026):** execution traces, run-to-run variance — transparentnost procesu, ale ne pro kódování.
 - **Terminal-bench:** jen pass/fail, neměří proces.
- **Kam doplnit:** METR + GitGoodBench do kap02 (gap argument — i když pass/-fail projde, kvalita nemusí stačit). RACE, DevBench do kap02 (existující multi-dim benchmarky). Do 5.2 porovnání naší sady s těmito benchmarky.

Audit trail — klíčové rozhodnutí 5.1.1 (2026-03-26):

Předchozí verze (DRAFT v1, viz audit trail výše) míchala dvě věci: prezentaci výsledků (hodnoty metrik z běhů) a diskuzi o designu sady. Hodnoty patří do kap04, v 5.1.1 diskutujeme co jsme se dozvěděli **o metrikách samotných** — jejich design, co funguje, co ne, co bychom změnili. Konkrétní čísla z běhů jen jako ilustrace, ne jako narování tabulky.

Rozlišení: kap04 = co jsme naměřili; kap05 5.1.1 = co jsme se dozvěděli o designu metrik.

Struktura 5.1.1 sleduje 3 dimenze z cíle 1 (proces, kvalita, efektivita) + limity.

TODO — úpravy mimo kap05:

- **CLAUDE.md:** odstranit reference na DSR terminologii (artefakt, build-evaluate), ponechat ML styl.
- **kap03 metriky efektivty:** příliš technický odstavec (token výpočty), patří do kap04/přílohy. Odkaz na pilot-r1 nepatří do metodiky (TODO již přidáno).
- **kap03 Fenton:** upřesnit že Fentonova taxonomie (Process/Product/Resource) je z doby před LLM — naše adaptace (P/Q/E) přidává LLM-as-judge metriky které Fenton neměl. Zmapovat kde přesně mapujeme a kde rozšiřujeme.

5.1.2 Cíl 2: Iterativní postup

[DRAFT]

Na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a ukázat, že vede k měřitelnému zlepšení dodržování vývojových praktik.

Pilotní iterace ukazují, že ano. Baseline (r1) splnila 4 z 10 deterministických kritérií; po dvou cyklech úprav (r3) agent splnil 9 z 10. Každá iterace identifikovala konkrétní selhání, přiřadila mu příčinu a navrhla cílenou opravu. Diagnostika se přitom opírala o naměřená data, ne o subjektivní dojem: metriky poskytly konkrétní vodítka kam zasáhnout.

Analýza průběhu iterací odhalila vzorec, kterým se instrukce vyvíjely: obecná pravidla (r1: “jedna branch per issue”) → konkrétní příkazy (r2: `git checkout -b issue-N`) → verifikační kroky (r3: `git log -oneline -3` jako kontrola po commitu). Tento vzorec se opakoval u tří nezávislých selhání (P2, P3, Q8) a odpovídá zjištění Breunigu (Breunig, 2025), že opakování ignorovaného pravidla nepomáhá, je třeba ho přestrukturovat.

Regrese r4 a r5 ukazují limity postupu. R4 přidalo dvě deklarativní pravidla do Constraints, přesto se zhoršily P2, P5 a Q3. R5 přidalo dvě procedurální opravy do pre-PR checklistu, obě cílené opravy zabraly (Q5 = 0, Q7 = 0), ale agent ignoroval celý pracovní postup (P2–P4 nesplněny). Nedeterminismus modelu znamená, že stejné instrukce nezaručují stejné výsledky. Návrat k předchozí verzi instrukcí po neúspěšné iteraci je standardní postup v iterativním návrhu (Peppers et al., 2008). R5 proto vycházel z r3, ne z r4.

Z hlediska praktické proveditelnosti: každá iterace trvala 25–40 minut (běh agenta) plus řádově desítky minut na diagnostiku a úpravu instrukcí. Diagnóza se opírala o tabulku metrik a behaviorální popis z FINDINGS.md. Postup je tedy použitelný pro praktika bez specializovaného vybavení.

5.1.3 Cíl 3: Ablace

[DRAFT]

Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní. Dvě ablace testovaly dvě složky s deterministickým chováním v pilotních iteracích: verifikační kroky (ablace A) a sekci Package Quality (ablace B).

Verifikační kroky nejsou redundantní. Ablace A odebrala příkazy `tsc -noEmit`, `npx eslint` a `git log -oneline -3` z pracovního postupu. Dopad se projevil v pěti metrikách: Q2 kleslo z 41/42 na 35–37/42, Q3 z 71 % na 62 %, Q5 se zhoršilo z 1 na 3–4 warningů, Q7 z 1 na 1–4 violations a agent spotřeboval přibližně dvakrát více testovacích cyklů. Verifikační kroky plní dvojí funkci: zajišťují kontrolu kvality kódu a současně strukturují práci agenta tím, že zabráňují cyklickému debugování. Bez nich se agent častěji dostal do smyčky opakovaného spouštění testů, což v jednom běhu vedlo k vyčerpání kontextového okna.

Sekce Package Quality je pro automatizované metriky převážně redundantní. Ablace B odebrala celou sekci (modulární struktura, strict TypeScript, JSDoc, čisté veřejné API). Metriky Q5 (lint warnings), Q6 (typecheck errors) a Q7 (složitost kódu) zůstaly na srovnatelné úrovni; model produkoval kvalitní kód i bez explicitních konvencí. Q8 (design quality) však kleslo z 3/3 na 2/3 ve všech ablačních bězích kde judge hodnocení dokončil: chybějící modularita (veškerý kód v jednom souboru v B-2) a neúplná dokumentace. Základní kódové konvence jsou u tohoto modelu řízeny znalostmi z tréninku; strukturální rozhodnutí jako modularita a dokumentace instrukce ovlivňují.

procesní metriky (P1–P8) vykazovaly v obou ablacích variabilitu srovnatelnou s pilotními iteracemi. Instrukce pro pracovní postup zůstaly v obou ablacích beze změny — variabilita je důsledkem nedeterminismu modelu, ne ablace. Tento poznatek je sám o sobě důležitý: ukazuje, že procesní compliance je u tohoto modelu stochastická vlastnost, kterou instrukce ovlivňují jen částečně.

5.2 Porovnání s literaturou

[DRAFT]

Výsledky případové studie zasazujeme do kontextu šesti existujících prací, které se zabývají vlivem instrukcí na chování AI agentů.

Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost souboru `AGENTS.md` koreluje se snížením mediánového runtime o 28,6 %. Naše data jsou s tímto nálezem kompatibilní: agent s instrukcemi dokončil všechny běhy v čase 13–39 minut (E2). Naše studie však přidává rozlišení, které Lulla et al. neizolovali: ne všechny obsah instrukčního souboru přispívá stejně. Verifikační kroky měly měřitelný dopad na čtyři metriky (Q2 referenční testy, Q3 mutation

score, Q5 lint, Q7 složitost), zatímco sekce Package Quality ponechala automatizované metriky kvality na srovnatelné úrovni. Efekt instrukčního souboru jako celku, měřený Lullou et al., je tedy kompozicí nerovnoměrně přispívajících složek.

Gloaguen et al. (Gloaguen et al., 2025) zjistili pouze marginální zlepšení úspěšnosti (+4 %) u generických instrukčních souborů a upozornili na zvýšený inference cost (+20 %). Naše instrukce jsou procedurální, ne generické: obsahují konkrétní příkazy a verifikační kroky místo popisu adresářové struktury. Rozdíl ve výsledcích (naše instrukce vykazují silnější efekt) podporuje hypotézu, že typ instrukčního obsahu (procedurální scaffolding vs. popisná dokumentace) je důležitější než samotná přítomnost instrukčního souboru.

Li et al. (Li et al., 2025) rozlišují kurátorované a automaticky generované instrukce. Kurátorované přinesly zlepšení +16,2 procentních bodů, automaticky generované naopak mírné zhoršení (−1,3 pp). Naše instrukce odpovídají kategorii kurátorovaných: byly konstruovány z literatury (sekce 4.1.3) a iterativně zpřesňovány na základě naměřených dat. Zlepšení z 4/10 na 9/10 deterministických kritérií je konzistentní s pozorováním Li et al., že kvalita obsahu instrukcí je rozhodující.

Breunig (Breunig, 2025) formuloval princip “neboť se s váhami”: opakování ignorované instrukce nepomáhá, přestrukturování ano. Evoluce instrukcí v naší pilotní fázi tento princip přímo potvrzuje. R1 obsahovala obecné pravidlo “jedna branch per issue”, agent ho ignoroval. R2 přidala konkrétní příkaz (`git checkout -b`), agent ho stále nesplnil. Až r3, která přidala verifikační krok (`git log -oneline -3`) jako kontrolní bod, dosáhla splnění. Stejný vzorec se opakoval u P3 a Q8. Naše data doplňují Breunigovo pozorování o konkrétní mechanismus: obecné pravidlo → konkrétní příkaz → verifikační krok jako kontrolní bod. Tento vzorec (postupná operacionalizace od abstraktního pravidla ke konkrétnímu checkpointu) koresponduje se širším výzkumem o vlivu specifity instrukcí na chování jazykových modelů: Kim et al. zjistili, že explicitnější instrukce zlepšují výsledky zejména u procedurálních úloh, ale přílišný detail může omezit schopnost modelu reagovat na nepředvídané situace (Kim, 2025; Zi et al., 2025). Ablace B naznačuje podobný efekt: sekce Package Quality fungovala jako *enabling constraint*, tedy obecný rámec, v jehož mezích agent sám volil konkrétní řešení. Její odebrání neovlivnilo automatizované metriky (Q5–Q7), ale zhoršilo designová rozhodnutí (Q8), což naznačuje, že některé instrukce nepůsobí přímo, ale vytvářejí podmínky pro kvalitní emergentní chování.

Mao et al. (Mao et al., 2025) identifikovali sedm typů komponent promptových šablon a zjistili, že Role a Directive se nejčastěji objevují na začátku, Constraints na konci. Naše baseline instrukce sledovaly toto pořadí. Ablace ukázala, že Constraints se částečně překrývají s Process — deklarativní pravidlo v Constraints (r4: “každé pole musí být použito”) nemělo efekt, zatímco totéž jako procedurální verifikace v Process (r5) ano. To koresponduje s pozorováním Mao et al., že exclusion constraints jsou nejčastějším typem omezení (46 % výskytů), ale naše data naznačují, že jejich efektivita závisí na tom, zda jsou formulovány deklarativně nebo procedurálně.

Razavi a Fard (Razavi & Fard, 2025) varují před prompt sensitivity: malé změny

formulace mohou dramaticky změnit chování modelu. Naše pilotní data toto riziko přímo demonstrují. Instrukce r3 a r5 se lišily ve dvou řádcích pre-PR checklistu; procesní compliance se přitom propadla z 5/5 na $\sim 1/5$. R4 přidalo dvě věty do Constraints a procesní compliance klesla z 5/5 na $\sim 3/5$. Tyto výkyvy nelze přičítat pouze změnám instrukcí; jde o kombinaci prompt sensitivity a nedeterminismu modelu, přesně jak Razavi a Fard predikovali.

Specificita instrukcí a jejich mechanismus účinku. Existující výzkum instrukcí pro AI agenty se soustředí na to, jak konkrétně agentovi říct co dělat. Mao et al. klasifikují komponenty, Lulla et al. měří přítomnost souboru, Breunig doporučuje operacionalizaci. Implicitní předpoklad je, že instrukce fungují jako příkazy. Naše data naznačují, že instrukce mohou působit i jiným mechanismem.

Verifikační kroky (nejkonkrétnější typ instrukce) agent dodržoval spolehlivě a přímo zlepšovaly měřitelné metriky. Fungují *vynucením*: kontrolují konkrétní výstup. Konvence v Package Quality stojí na opačném konci spektra: neříkají *jak* cíle dosáhnout, pouze *co* je žádoucí (“modulární struktura”, “strict TypeScript”). Jejich odebrání neovlivnilo automatizované metriky (Q5–Q7), ale zhoršilo designová rozhodnutí (Q8). Fungují *aktivací*: připomínají modelu znalosti z tréninku, aniž by předepisovaly konkrétní řešení. Analogie existuje v prompt engineeringu: chain-of-thought prompting ukazuje, že minimální nápověda aktivuje latentní schopnost modelu efektivněji než detailní instrukce (Min et al., 2022; J. Wei et al., 2022).

Pravidla v Constraints neplní ani jednu funkci: jsou příliš abstraktní na vynucení a příliš explicitní na aktivaci. Tento mechanismus (instrukce jako nápověda podporující emergentní chování, ne jako příkaz vynucující konkrétní akci) je v kontextu instrukčních souborů pro coding agenty dosud nepopsaný. Naše data jsou indikativní (jedna ablace, jeden model); systematické testování navrhujeme v sekci 5.5.

[RAW]

RAW TODO (2026-03-24): Spektrum specificity a struktura AGENTS.md.

Při revizi kap05 prozkoumat a diskutovat: sekce AGENTS.md se přirozeně řadí od nejobecnějších (Role, Goal = nápověda/kontext) přes středně specifické (Package Quality = enabling constraint) po nejkonkrétnější (Process = příkazy a verifikační kroky). Toto pořadí odpovídá pozorování Mao et al. (Role na začátku, Constraints na konci), ale paradoxně Constraints (pravidla/zákazy) jsou v praxi *nejméně* efektivní, ačkoliv jsou sémanticky “silné”. Porovnat s Searleho škálou direktivní síly (suggestion → request → command → order) a Cynefin enabling vs. rigid constraints. Formulovat opatrně — indikativní pozorování z jedné case study.

Zasazení výsledků do kontextu existujícího výzkumu.

- Porovnání s Lulla et al. (AGENTS.md efekt)
- Porovnání s Breunig (prompt vs. model)
- Porovnání s ablačními studiemi (SWE-agent, CCA, RePrompt)

- Kde naše výsledky potvrzují / rozporují existující evidence

5.3 Limity ve světle výsledků

[DRAFT]

Sekce 3.5 identifikovala metodologická omezení designu studie. Zde diskutujeme, jak se tato omezení projevila v naměřených datech.

Nedeterminismus jako dominantní faktor. Interní validita studie je nejvíce ohrožena nedeterminismem modelu. Procesní compliance (P1–P5) kolísala od 5/5 (r3) přes $\sim 3/5$ (r4) po $\sim 1/5$ (r5) při instrukcích, které se lišily minimálně. Dva běhy per ablační variaci tento problém zmírňují (umožňují odlišit systematický dopad ablance od náhodného šumu), ale neposkytují statistickou sílu. Výsledky ablací proto interpretujeme jako indikace, ne jako statisticky podložené závěry. Tam, kde oba ablační běhy vykazují stejný trend (Q5 v ablaci A: 4 a 3 oproti r3: 1), je důvěra v efekt ablance vyšší. Tam, kde se běhy výrazně liší (Q2 v ablaci B: 37/42 vs. 11/42), nelze efekt ablance spolehlivě odlišit od nedeterminismu.

Prompt sensitivity potvrzena. Razavi a Fard (Razavi & Fard, 2025) varují, že drobné změny promptu mohou mít nepředvídatelné dopady. R4 přidalo dva řádky do Constraints a procesní compliance se zhoršila, přestože žádná změna přímo necílila na workflow. Toto pozorování potvrzuje, že změny instrukcí mohou mít vedlejší efekty, které diagnostika nepředvídá. V kontextu iterativního postupu to znamená, že každá úprava instrukcí vyžaduje měření celé sady metrik, ne jen těch, na které úprava cílila.

Diagnostická chyba výzkumníka. V jedné iteraci došlo k záměně referenčních testů (metrika Q2, agentovi neviditelné) s agentovými vlastními testy. Na základě tohoto falešného předpokladu byla navržena instrukční změna, která vedla k regresi. Chyba byla odhalena při revizi, referenční testy opraveny na behavioral testy přes veřejné API a metrika Q2 přeměřena (sekce 3.5). Tento incident ukazuje důležitost auditovatelného řetězce rozhodnutí: bez changelogu a korekčních poznámek v FINDINGS.md by chyba zůstala neodhalena.

Disconnect ESLint konfigurační. Agent v r3 spustil `eslint` s vlastní konfigurací, která neobsahovala pravidlo `complexity` — dostal nula warningů a kód považoval za hotový. Experiment přitom měří Q5 a Q7 s fixní konfigurací, kde pravidlo je. Agent nemůže opravit problém, o kterém neví. Explicitní požadavek na konkrétní ESLint konfiguraci v r5 tento disconnect odstranil. Jde o příklad situace, kdy nestačí říct agentovi “spuštění lint”, ale je třeba specifikovat i s jakou konfigurací.

Cohenovo κ neprovedeno. Jak uvádí sekce 3.5, pět kvalitativních metrik (P6–P8, Q4, Q8) slouží jako podpůrné indikátory. Hlavní závěry studie stojí na deterministických metrikách. Jedinou výjimkou je Q8, které odhalilo pokles designové kvality v ablacích; tento nálezný automatizované metriky nezachytily, ale samotné hodnocení Q8 nelze bez validace proti lidskému hodnocení považovat za objektivní měření.

Znamé nedostatky měřicí infrastruktury. Při validaci byly identifikovány tři nedostatky: (1) metrika Q4 používá v rubrice 24 AC místo skutečných 25 (chybí AC pro konfiguraci svátků), (2) metrika P7 nefiltruje specifikační issue vytvořenou infrastrukturou a v bězích bez agentových issues hodnotí nesprávný artefakt, (3) detekce `any` v Q6 zachytává anglické slovo v komentářích (false positive). Všechny tři nedostatky jsou konzistentní napříč běhy a neovlivňují srovnání mezi iteracemi. Dále: u E1 srovnáváme vstupní a výstupní tokeny ze `transcript.json` (viz kap. 3.2.3); u vstupní strany počítáme `input+cache.read+cache.write` na krok. Třetí číslo E1 uvádí součet `cache.read+cache.write` přes kroky (viz kap. 3.2.3). Mezi běhy může hrát roli změna reportingu tokenů v OpenCode nebo úprava `limit.context`.

Změny prostředí mezi běhy. Běhy r1–r2 proběhly bez kontejnerové izolace, od r3 běží agent v Docker kontejneru. Od r4 byla změněna konfigurace modelu. Hlavní proměnné (model, instrukce, specifikace) zůstaly konzistentní. Temperature modelu nebyla explicitně nastavena; experiment používal výchozí konfiguraci poskytovatele. Nižší temperature by mohla snížit variabilitu, ale omezila by ekologickou validitu (praktik typicky používá výchozí nastavení). OpenCode neukládá obsah `AGENTS.md` do exportovaného transcriptu. Nepřímým důkazem přijetí instrukcí je, že agent v reasoning části cituje specifický obsah instrukcí.

Tiché ukončení nástroje. Při ablačních bězích se OpenCode v některých případech tiše ukončil uprostřed práce agenta. Běhy s vyšší spotřebou kontextového okna (ablace A: ~44 spuštění testů oproti ~20 u ablance B) byly postiženy častěji. Postižené běhy byly opakovány; příčina nebyla identifikována.

Generalizovatelnost. Omezení plynoucí z jednoho modelu, jednoho projektu a deterministické specifikace diskutuje sekce 3.5. V praxi se tato omezení projevila zejména u procesních metrik: variabilita P1–P5 neumožňuje odlišit efekt instrukcí od nedeterminismu modelu bez většího počtu běhů. Zda by instrukce fungovaly stejně na projektech s vyšší nejednoznačností specifikace nebo na jiných modelech, zůstává otevřenou otázkou (sekce 5.5).

[RAW]

RAW TODO (2026-03-26): Překryv kap03/kap05 vyřešen — “Jeden model, jeden projekt” a “Nízká entropie specifikace” sloučeny do “Generalizovatelnost” s odkazem na kap03. Cohe-novo κ zkráceno. Zbývá opravit 5 REVIEW-LAYERS značek v sekci 5.1–5.2 (interpretace jako fakt, závěr bez pozorování).

5.4 Doporučení pro praxi

[DRAFT]

Případová studie přinesla poznatky, které mohou využít vývojáři pracující s AI coding agenty. Následující doporučení vycházejí z pilotních iterací a ablací; platí pro tento model a typ projektu, ale principy jsou přenositelné.

Verifikační kroky jsou klíčové.

Explicitní příkazy v pracovním postupu (`tsc -noEmit`, `npx eslint`, `git log -oneline -3`) agent dodržoval spolehlivě i v bžích kde ignoroval zbytek pracovního postupu. Bez nich se zhoršila kvalita kódu i funkční korektnost (sekce 4.3.2). Doporučení: každý měřitelný požadavek na kvalitu převést na konkrétní příkaz v pre-PR checklistu.

Konkrétní instrukce fungují lépe než obecná pravidla.

Pravidlo (“každé pole musí být použito”) v r4 nepomohlo. Verifikační krok (“spustě eslint a ověř nula warningů”) v r3 a r5 fungoval. Vzorec z pilotních iterací: obecné pravidlo → konkrétní příkaz → verifikační krok s okamžitým výstupem. Breunig (Breunig, 2025) formuloval stejný princip jako operacionalizaci: když agent pravidlo ignoruje, nepomůže ho zopakovat, je třeba ho převést na konkrétní akci.

Konvence kvality kódu jsou z velké části redundantní.

Sekce Package Quality (modulární struktura, strict TypeScript, JSDoc) měla minimální vliv na automatizované metriky Q5–Q7; model produkoval srovnatelný kód i bez ní (sekce 4.3.3). Strukturální rozhodnutí (modularita, dokumentace) se nicméně zhoršila (Q8). Doporučení: konvence na úrovni kódu zkrátit na minimum, investovat čas do verifikačních kroků.

Počítat s nedeterminismem.

Stejně instrukce vedly k procesní compliance 5/5 (r3), ~3/5 (r4) i ~1/5 (r5). Vícekrokový pracovní postup agent dodržuje nedeterministicky, nelze spoléhat na jednorázové ověření. Doporučení: zavést měřitelná exit kritéria a opakované kontroly místo jednorázového review.

Minimální efektivní instrukce.

Na základě ablací lze doporučit soubor `AGENTS.md`, který obsahuje: (1) kontext projektu (role, cíl, specifikace), (2) procedurální pracovní postup s konkrétními příkazy, (3) pre-PR checklist s verifikačními kroky. Konvence kvality kódu a deklarativní omezení jsou volitelné; verifikační kroky plní jejich funkci efektivněji.

[RAW]

Praktické závěry pro vývojáře pracující s AI coding agenty.

- Co zahrnout do `AGENTS.md` / instrukčních souborů

- Co je zbytečné / kontraproduktivní
- Minimální efektivní scaffolding

5.5 Náměty pro další výzkum

[DRAFT]

Případová studie otevřela několik otázek, které přesahují rozsah této práce.

Nedeterminismus více krokového pracovního postupu.

Agent dodržoval verifikační příkazy (jednokrokové akce) deterministicky, ale více krokový pracovní postup (issue → branch → test → implement → PR) nedeterministicky: r3 kompletně, r5 vůbec, při téměř identických instrukcích. Možná vysvětlení zahrnují ztrátu pozornosti v dlouhém kontextovém okně, stochastickou volbu mezi strukturovaným a nestrukturovaným vývojem z tréninku modelu a pozici instrukce v kontextu. Granulární ablace pracovního postupu (které kroky jsou kritické?) a opakované běhy se stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

Kontinuita práce mezi agenty.

procesní metriky (P1–P8) měří transparentnost artefaktů (issues, commit zprávy, PR popisy). Otevřená otázka: stačí tato transparentnost k tomu, aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní instrukce vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.

Automatizace iteračního postupu.

Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně autorem. Frameworky jako PromptWizard (Agarwal et al., 2024) a Prompt Alchemy („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025) ukazují, že cyklus analýzy selhání a návrhu úprav lze automatizovat. Kombinace automatizovaného prompt optimizeru s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.

Ingestion checkpoint.

Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale ne vždy je aktivuje jako řídicí smyčku běhu. Místo pracovního postupu z `AGENTS.md` použije vlastní

postup odvozený ze specifikace. Ověřitelný checkpoint na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci pracovního postupu před prvním zápisem kódu) by mohl tento problém adresovat.

Mechanismy účinku instrukcí.

Naše data naznačují dva odlišné mechanismy: *vynucení* (verifikační kroky přímo kontrolují výstup) a *aktivace* (obecné konvence připomínají modelu znalosti z tréninku). Analogie existuje v prompt engineeringu: chain-of-thought prompting ukazuje, že minimální nápověda může aktivovat komplexní latentní schopnost modelu efektivněji než detailní instrukce. Systematický experiment s odstupňovanou specifikitou instrukcí, od nápověd (“piš modulární kód”) přes pravidla (“každé pole musí být použito”) po verifikační kroky (“spuť `tsc`, oprav chyby”), by mohl identifikovat, která míra konkrétnosti je pro který typ požadavku optimální. Otevřená je i otázka, zda se toto optimum posouvá se schopnostmi modelu; lepší modely by mohly potřebovat méně specifické instrukce (analogicky k expertise reversal effect v kognitivní psychologii, kde instrukce účinné pro nováčky zpomalují experty).

Rozšíření na další modely a projekty.

Výsledky platí pro jeden model (Minimax) a jeden typ projektu (deterministická business logika). Replikace s jinými modely, nedeterministickými doménami (UI, strojové učení) a většími projekty by ukázala, které poznatky jsou přenositelné a které specifické pro tento případ. Zvláště zajímavá je otázka projektů s vyšší nejednoznačností specifikace (sekce 5.3), kde by obecnější instrukce (enabling constraints) mohly být efektivnější než detailní příkazy.

[RAW]

Future work.

- Více modelů, více projektů
- Systematické porovnání formátů specifikace
- Multi-agent vs. single-agent scaffolding
- Longitudinální studie (více issues, evoluce projektu)
- **Proč agent nedodrží multi-step workflow.** Pilotní data ukazují, že agent dodržuje verifikační příkazy (jednokrokové akce) deterministicky, ale multi-step workflow (issue → branch → test → PR) nedeterministicky — r3 dodržel kompletně, r5 zcela ignoroval při téměř identických instrukcích. Možné vysvětlení: (a) jednoduché příkazy nevyžadují plánování, agent je kopíruje do terminálu; workflow vyžaduje udržet plán přes desítky kroků. (b) Tréninková data modelu obsahují oba vzorce — strukturovaný i nestrukturovaný vývoj — a model mezi nimi volí stochasticky. (c) Pozice instrukce v kontextovém okně: s rostoucí délkou konverzace klesá vliv dřívějších instrukcí. Granulární ablace workflow struktury (které kroky jsou kritické?) a opakované běhy se

stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

- **Kontinuita práce mezi agenty.** procesní metriky (P1–P8) měří transparentnost artefaktů — issues, commit zprávy, PR popisy. Otevřená otázka: stačí tato transparentnost k tomu, aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní scaffolding vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.
- **Automatizace iteračního postupu.**
Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně. Frameworky jako PromptWizard (Agarwal et al., 2024) (Score→Critique→Synthesize) nebo Prompt Alchemy („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025) ukazují, že tento cyklus lze automatizovat — model sám analyzuje kde instrukce selhaly a navrhuje úpravy. Kombinace automatizovaného APO s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.
- **Ingestion checkpoint.** Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale neaktivuje je jako řídicí smyčku běhu — místo workflow z AGENTS.md použije vlastní implementační smyčku odvozenou ze specifikace. Ověřitelný checkpoint na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci workflow před prvním zápisem kódu) by mohl tento problém adresovat.

[RAW]

Feature work / námět k diskuzi — exit kritéria jako součást instrukcí:

Otázka: co by se stalo kdybychom agentovi předali exit kritéria přímo v AGENTS.md? (“Musíš dosáhnout P1=5/5, Q3≥70 %, Q5=0 — ověř si to před PR.”)

Argument PRO: pokud máme dostatečný počet metrik pokrývajících různé dimenze (proces, kvalita, efektivita), optimalizace na metriky = dobrá práce. Goodhartův zákon platí pro jednorozměrné metriky — agent který optimalizuje jen coverage může zanedbat kvalitu. Ale s 19 metrikami napříč P/Q/E je těžší gaminovat všechny najednou bez skutečné kvality.

Argument PROTI: agent může najít zkratky (napr. trivální testy pro Q3, verbose kód pro Q7). Také: explicitní metriky v instrukcích mění povahu experimentu — měříme pak schopnost agenta sledovat metriky, ne schopnost dodržovat vývojové praktiky.

Kompromis k prozkoumání: přidat do AGENTS.md jen deterministické quality gates jako self-check (“Run tsc --noEmit before PR, run eslint src/ — zero warnings required”) bez číselných prahů. Agent ví co zkontrolovat, ale neví jaký je náš práh → nelze triválně optimalizovat.

Závěr

[DRAFT]

Tato práce se zabývala návrhem sady metrik a iterativního postupu pro systematické navrhování a vyhodnocování instrukcí pro AI coding agenty. Oba výstupy byly ověřeny na případové studii systému upomínek faktur, která zahrnovala pět pilotních iterací a čtyři ablační běhy.

Prvním cílem bylo navrhnout sadu metrik, která měří proces a kvalitu práce agenta, ne jen výsledek. Výsledná sada 19 metrik (procesní procesní metriky (P1–P8), produktové metriky (Q1–Q8), metriky efektivity (E1–E3)) pokrývá tři dimenze odvozené z taxonomie Fentona a Biemana. Metriky zachytily jak zlepšení při iterativních úpravách instrukcí, tak regrese způsobené nedeterminismem modelu i cílené dopady ablací. Kombinace deterministických a kvalitativních metrik (LLM-as-judge) přináší komplementární pohled — automatizované metriky nezachytí slabiny designu, kvalitativní samy o sobě nezměří procesní compliance.

Druhým cílem bylo iterativním postupem navrhnout instrukce, které dovedou agenta k dodržování stanovených exit kritérií. Cyklus Spuštění/Měření/Diagnóza/Úprava zlepšil procesní compliance ze 4/10 na 9/10 splněných deterministických kritérií ve dvou iteracích. Analýza průběhu odhalila opakující se vzorec: obecné pravidlo, které agent ignoroval, bylo nahrazeno konkrétním příkazem a následně doplněno verifikačním krokem. Každá iterace trvala 25–40 minut, postup je tedy proveditelný pro praktika bez specializovaného vybavení.

Třetím cílem bylo ablacemi identifikovat, které složky instrukční sady přispívají k měřenému chování agenta. Verifikační kroky se ukázaly jako neredundantní — jejich odebrání zhoršilo čtyři metriky a vedlo k výrazně vyššímu počtu testovacích cyklů. Sekce Package Quality byla pro automatizované metriky převážně redundantní, ale ovlivnila designovou kvalitu hodnocenou LLM-as-judge. Dominantním faktorem variability procesní compliance byl nedeterminismus modelu, ne obsah instrukcí.

Hlavním přínosem práce jsou přenositelné výstupy: sada metrik a iterativní postup, které může kdokoli aplikovat na vlastním projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Práce je omezena na jeden model a jeden projekt — závěry mají povahu analytické generalizace na teorii, nikoliv statistické generalizace na populaci.

Bibliography

- ACE-Bench: End-to-End Feature Development Benchmark [212 tasks from 16 repos; Claude 4 Sonnet + OpenHands: 7.5% on feature-level tasks]. (2025). <https://openreview.net/forum?id=41xrZ3uGuI>
- Agarwal, A., et al. (2024). PromptWizard: Task-Aware Prompt Optimization Framework [Microsoft Research. arXiv:2405.18369. Iterativní Score→Critique→Synthesize cyklus pro optimalizaci promptů.].
- Bockeler, B. (2025). *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel* [Critical analysis of SDD tools. Spec-kit files “repetitive, both with each other, and with the code”]. <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- Breunig, D. (2025). Don’t Fight the Weights [Defines fighting-the-weights: when prompt instructions oppose trained model behavior; recognition signs and mitigation strategies]. <https://www.dbreunig.com/2025/11/11/don-t-fight-the-weights.html>
- Cao, L., & Ramesh, B. (2008). Agile Requirements Engineering Practices: An Empirical Study [16 organizations, 7 agile RE practices identified]. *IEEE Software*, 25(1), 60–67. <https://doi.org/10.1109/MS.2008.1>
- FeatureBench: Benchmarking Agentic Coding for Complex Feature Development [Agents struggle when scope exceeds single-issue; feature-level tasks significantly harder]. (2026). *arXiv preprint arXiv:2602.10975*.
- Fenton, N. E., & Bieman, J. (2014). *Software Metrics: A Rigorous and Practical Approach* (3rd ed.) [Process vs. product vs. resource metrics taxonomy; GQM framework]. CRC Press.
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd). Addison-Wesley.
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2025). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? [LLM-generated context files reduce success rate; developer-written marginally help (+4%); context files increase cost 20%+; agents follow instructions but no performance gain; AGENTBENCH benchmark (138 instances, 12 repos)]. *arXiv preprint arXiv:2602.11988*. <https://arxiv.org/abs/2602.11988>
- Gotel, O. C., & Finkelstein, A. C. (1994). An Analysis of the Requirements Traceability Problem. *Proceedings of IEEE International Conference on Requirements Engineering*, 94–101.
- Harman, M., Mao, K., Moran, K., Tufano, R., Gligoric, M., Shi, A., & Havrilla, A. (2025). Mutation-Guided LLM-Based Test Generation at Meta: A White Paper [70% mutants unkillable despite full coverage; 73% engineer acceptance rate; deployed on 10,795 classes].
- Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic Software Engineering: Foundational Pillars and a Research Roadmap [SASE framework, BriefingScript, MentorScript, SE4H/SE4A duality]. *arXiv preprint arXiv:2509.06216*. <https://arxiv.org/abs/2509.06216>

- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research [Foundational DSR framework: build-evaluate cycle, 7 guidelines for design science research]. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Chen, J., Lin, J., Xiong, Y., Lu, J., Zhang, H., & Xie, T. (2025). Rethinking the Value of Agent-Generated Tests in Autonomous Code Repair [83.2% tasks same outcome regardless of test writing; tests serve as observational feedback]. *arXiv preprint arXiv:2505.21615*.
- IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge* (Version 4.0). <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [2294 tasks from real GitHub repositories, de facto benchmark for AI coding agents]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>
- Kim, O. (2025). DETAIL Matters: Measuring the Impact of Prompt Specificity on Reasoning in Large Language Models [Emory University. Framework pro evaluaci vlivu specifičnosti promptu na reasoning. 30 úloh, GPT-4 a O3-mini]. *arXiv preprint arXiv:2512.02246*. <https://arxiv.org/abs/2512.02246>
- Li, X., Chen, W., Liu, Y., Zheng, S., Chen, X., He, Y., Li, Y., et al. (2025). SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks [Curated Skills +16.2pp; self-generated Skills -1.3pp; 2–3 focused modules optimal; comprehensive documentation hurts (-2.9pp); smaller model + Skills can exceed larger model without; 84 tasks, 11 domains, 7308 trajectories]. *arXiv preprint arXiv:2602.12670*. <https://arxiv.org/abs/2602.12670>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts [U-shaped performance curve; info in middle performs worse than no-context baseline]. *Transactions of the Association for Computational Linguistics*, 12.
- Lulla, K., Zhu, M., Kalliamvakou, E., & Mohylevskyy, Y. (2026). On the Impact of AGENTS.md Files on AI Coding Agents [124 PRs across 10 repos: -28% runtime, -20% output tokens with AGENTS.md; architectural info and coding conventions most effective]. *arXiv preprint arXiv:2601.20404*. <https://arxiv.org/abs/2601.20404>
- Mao, Y., He, J., & Chen, C. (2025). From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications [Merged 4 frameworks (Google Cloud, Elavis Saravia, CRISPE, LangGPT); 7 component types; Directive 87% prevalence]. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. <https://arxiv.org/abs/2504.02052>
- Mathews, N. S., & Nagappan, M. (2024). LLM-Based Test Generation as Bug Validation: Insights from Reproducing Real-World Bugs [68.1% generated test suites validate bugs instead of detecting them]. *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*. <https://doi.org/10.1145/3664646.3664766>
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>

- METR. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [RCT: 16 developers, 246 tasks; AI tools slowed experienced developers by 19%; developers believed they were 24% faster]. *arXiv preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? [Label space, input distribution a formát jsou klíčové pro ICL, ne správnost label-input mapování]. *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2202.12837>
- Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations [Causal link: self-recognition $\rightarrow$ self-preference; linear correlation; fine-tuning pushes recognition to 90%+]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2404.13076>
- Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey [Mutation score stronger predictor of fault detection than structural coverage]. *Advances in Computers*, 112, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2008). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Piskala, D. B. (2026). *Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants* (tech. rep.) (Technical report. Three levels: spec-first, spec-anchored, spec-as-source. SDD workflow: Specify $\rightarrow$ Plan $\rightarrow$ Implement $\rightarrow$ Validate). arXiv. <https://arxiv.org/abs/2602.00180>
- Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation [arXiv:2503.11085. Iterativní refinement promptů pro code generation, evaluace přes execution.]. (2025).
- Razavi, S. P., & Fard, F. H. (2025). What Did I Do Wrong? Quantifying LLMs’ Sensitivity and Consistency to Prompt Engineering [Up to 76 accuracy points difference from subtle formatting changes in few-shot settings]. *Proceedings of NAACL*. <https://arxiv.org/abs/2406.12334>
- Runeson, P., & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Scacchi, W. (2002). Understanding the Requirements for Developing Open Source Software Systems [Software informalisms in OSS – issue trackers as de facto requirements]. *IEEE Proceedings – Software*, 149(1), 24–39. <https://doi.org/10.1049/ip-sen:20020202>
- Scale AI. (2025). SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? [1865 human-verified tasks, adds explicit Requirements and Interface sections to issues]. *arXiv preprint arXiv:2509.16941*. <https://arxiv.org/abs/2509.16941>
- Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt Engineering or Fine-Tuning: An Empirical Assessment of Large Language Models

- for Code. *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2310.10508>
- Sommerville, I. (2016). *Software Engineering* (10th). Pearson.
- Verga, P., Hofstätter, S., Cer, D., & Thorne, J. (2024). Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models [Panel of LLM evaluators (PoLL) outperforms single GPT-4 judge; disjoint model families reduce intra-model bias; 7x cheaper]. *arXiv preprint arXiv:2404.18796*. <https://arxiv.org/abs/2404.18796>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Google Research. Few-shot CoT exempláře zlepšují reasoning na aritmetických, commonsense a symbolických úlohách]. *Advances in Neural Information Processing Systems (NeurIPS)*, 35. <https://arxiv.org/abs/2201.11903>
- Wei, Y., et al. (2025). SWE-EVO: Long-Horizon Software Evolution Tasks [Multi-step modifications spanning avg 21 files; GPT-5 + OpenHands only 21% vs 65% on single-issue SWE-Bench]. *arXiv preprint arXiv:2512.18470*. <https://arxiv.org/abs/2512.18470>
- Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents [Komplexní agenti vs. levnější bez-agent přístup na SWE-bench; publikováno též na ICML 2025]. *arXiv preprint arXiv:2407.01489*. <https://arxiv.org/abs/2407.01489>
- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.) [Embedded single-case design; analytic generalization (to theory, not population)]. SAGE Publications.
- Yin, Z., Wang, J., Li, J., Shi, D., Wei, Y., Yue, Y., Liu, Z., Xia, X., & Lo, D. (2025). A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks. *arXiv preprint arXiv:2511.00872*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Foundational LLM-as-judge paper; position bias, verbosity bias, self-enhancement bias; GPT-4 >80% human agreement]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2306.05685>
- Zhu et al. (2026). The Necessity of a Unified Framework for LLM-Based Agent Evaluation [arXiv:2602.03238. Benchmarky jsou konfundovány system prompty — instrukce jsou nezávislá proměnná.].
- Zi, Y., Menon, H., & Guha, A. (2025). More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation [PartialOrderEval framework. Partial order of prompts from minimal to maximally detailed. HumanEval + ParEval]. *arXiv preprint arXiv:2508.03678*. <https://arxiv.org/abs/2508.03678>

[RAW]

Zdroje k zpracování do BibTeX (původně z notes/sources.md)

1. PRIMÁRNÍ ZDROJE (Frameworky a Metodiky)

GITHUB NEXT. Spec Kit: Spec-Driven Development for AI Agents [online]. 2024. <https://github.com/github/spec-kit> – Klíčový zdroj pro koncept "Spec-Driven Development".

BMAD-CODE-ORG. BMAD METHOD: Breakthrough Method for Agile AI-Driven Development [online]. GitHub, 2025. <https://github.com/bmad-code-org/BMAD-METHOD> – Metodika pro řízení AI agentů v agilním vývoji.

ANTHROPIC. Effective Harnesses for Long-Running Agents [online]. Anthropic Engineering Blog, 2024. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents> – Technický popis problémů s dlouhodobou pamětí agentů.

METR. Model Evaluation and Threat Research [online]. 2024. <https://metr.org/> – Standardy pro hodnocení bezpečnosti a schopností modelů.

THEDOTMACK. claude-mem: Persistent Memory System for Claude Code [online]. GitHub, 2025. <https://github.com/thedotmack/claude-mem> – Příklad implementace hooks v CLI agentech - persistentní paměť přes session.

1.2 SYSTÉMOVÉ MYŠLENÍ V SWE

PETKOV, Doncho et al. Information Systems, Software Engineering, and Systems Thinking: Challenges and Opportunities. International Journal of Information Technologies and Systems Approach. <https://www.igi-global.com/gateway/article/2534> – Mapuje historii systémového přístupu v IS a SWE. Propojení systémového myšlení s praxí SWE je stále nedotažené.

MONAT, Jamie a GANNON, Thomas. Systems Thinking: A Review and Bibliometric Analysis. MDPI Systems, 2020. <https://www.mdpi.com/2079-8954/8/3/23> – Přehled co systémové myšlení je a kde se používá. Interdisciplinární - SWE, management, vzdělávání.

ALHARTHI, Sultan et al. A Systems Thinking Approach to Improve Sustainability in Software Engineering. MDPI Sustainability, 2023. <https://www.mdpi.com/2071-1050/15/11/8766> – Praktická aplikace - dívají se na vývoj jako systém (developeři, zákazníci, stakeholders).

2. ODBORNÉ STUDIE

2.1 Přehledové studie (Surveys) - LLM agenti v SE

LIU, Junwei et al. Large Language Model-Based Agents for Software Engineering: A Survey. arXiv preprint arXiv:2409.02977. 2024. <https://arxiv.org/abs/2409.02977> – Komplexní přehled 106 prací o LLM agentech v SE, kategorizace z pohledu SE i agentů.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. arXiv preprint arXiv:2408.02479. 2024. <https://arxiv.org/abs/2408.02479> – Pokrývá requirements, code generation, testing, maintenance - celý SDLC.

Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. arXiv preprint arXiv:2510.09721. 2025. <https://arxiv.org/h>

tml/2510.09721 – Přes 150 paperů, taxonomie řešení a benchmarků.

A Survey on Code Generation with LLM-based Agents. arXiv preprint arXiv:2508.00083. 2025. <https://arxiv.org/abs/2508.00083> – Single-agent a multi-agent architektury, aplikace napříč SDLC.

2.2 Agentic Software Engineering - Klíčové práce

Agentic Software Engineering: Foundational Pillars and a Research Roadmap. arXiv preprint arXiv:2509.06216. 2025. <https://arxiv.org/html/2509.06216v2> – Přehodnocení SE pro spolupráci člověk-agent. Framework podobný SAE úrovní autonomie. Rozlišuje SE 2.0 (AI-augmented) vs SE 3.0 (Agentic SE).

AKBAR, Muhammad Azeem et al. Agentic AI in Software Engineering: Practitioner Perspectives Across the Software Development Life Cycle. SSRN. 2025. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5520159 – Rozhovory s 21 experty, pokrývá celý SDLC. Zjištění: agenti redefinují hranice mezi fázemi SDLC.

Autonomous Agents in Software Development: A Vision Paper. Springer, 2024. https://link.springer.com/chapter/10.1007/978-3-031-72781-8_2 – 12 LLM agentů spolupracujících na celém SDLC.

2.3 Evaluate a produktivita

METR. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. 2025. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/> – RCT studie: zkušeni vývojáři s AI jsou o 19% pomalejší - překvapivé zjištění.

2.4 Metriky kvality software a AI agentů

ISO/IEC. ISO/IEC 25010:2023 - Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. 2023. <https://www.iso.org/standard/78176.html> – Industry standard pro kvalitu software. 8 charakteristik, Functional Suitability obsahuje Completeness, Correctness, Appropriateness.

ISO 25000. ISO 25010 Software Quality Model. 2023. <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010> – Přehledný popis ISO 25010 modelu kvality.

LXT. AI Agent Evaluation: Comprehensive Framework for Measuring Agent Performance. 2024. <https://www.lxt.ai/blog/ai-agent-evaluation/> – Moderní framework pro evaluaci AI agentů: Task completion, Accuracy, Safety/trust (policy compliance, transparency), Tool usage.

Weights & Biases. AI Agent Evaluation: Metrics, Strategies, and Best Practices. 2024.

<https://wandb.ai/onlineinference/genai-research/reports/AI-agent-evaluation-Metrics-strategies-and-best-practices--VmlldzoxMjM0NjQzMQ>

– Praktický průvodce metrikami pro AI agenty.

SOMMERVILLE, Ian. Software Engineering. 10th ed. Pearson, 2016. Chapter 24: Quality Management (s. 705–728). – Rozlišení control metrics (procesní) vs. predictor metrics (produktové). Vztah proces-produkt u SW. Relevantní pro oporu dimenzí Functional Quality a Compliance v kap03.

McCONNELL, Steve. Code Complete: A Practical Handbook of Software Construction. 2nd ed. Microsoft Press, 2004. Chapter 28 (s. 715, Table 28-2). – Tabulka “Useful Software-Development Measurements” – kategorie Size a Overall Quality (defekty, defekty/KLOC, mean time between failures).

IEEE COMPUTER SOCIETY. SWEBOK: Guide to the Software Engineering Body of Knowledge. Version 4.0. IEEE, 2024. Chapter 12: Software Quality (s. 248–256), Chapter 6: Software Maintenance (s. 176). – Software Quality Measurement (s. 253) – kvantifikace atributů pro rozhodování. Míry údržby (s. 176): complexity, maintainability, testability, reliability.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-world Github Issues? In: ICLR. 2024. Appendix C.7 (s. 28). – Software Engineering Metrics v benchmarku – cyklomatická složitost (McCabe), Halstead measures pro hodnocení kódu agentů.

2.5 Základní LLM studie

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-world Github Issues? In: The Twelfth International Conference on Learning Representations (ICLR). 2024. <https://arxiv.org/abs/2310.06770> – Hlavní benchmark pro hodnocení schopností programovacích agentů.

LIU, Nelson F. et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv preprint arXiv:2307.03172. 2023. <https://arxiv.org/abs/2307.03172> – Klíčová studie vysvětlující, proč pouhé zvětšení kontextového okna nestačí.

VASWANI, Ashish et al. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017. <https://arxiv.org/abs/1706.03762> – Základní paper definující Transformer architekturu a mechanismus pozornosti (self-attention).

WEI, Jason et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Advances in Neural Information Processing Systems, 2022. <https://arxiv.org/abs/2201.11903> – Základ pro techniky prompt engineeringu používané v práci.

3. TEORIE SOFTWAREOVÉHO INŽENÝRSTVÍ A ARCHITEKTURY

RICHARDS, Mark a FORD, Neal. Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media, 2020. ISBN 978-1492043454. – Moderní přehled architektonických stylů a charakteristik ("ilities").

FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley Profes-

sional, 2002. ISBN 978-0321127426. – Katalog základních návrhových vzorů pro podnikové aplikace.

KHONONOV, Vlad. Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy. 1. vyd. O'Reilly Media, 2021. ISBN 978-1098100131. – Definuje pojmy jako "Bounded Context" a "Ubiquitous Language", které jsou analogií pro kontext LLM.

ISO/IEC. ISO/IEC/IEEE 42010:2011 Systems and software engineering — Architecture description. 2011. – Mezinárodní standard definující základní pojmy popisu architektury.

IEEE COMPUTER SOCIETY. SWEBOK: Guide to the Software Engineering Body of Knowledge. Version 3.0. IEEE, 2014. <https://www.swebok.org/> – Standardní taxonomie softwarového inženýrství.

BROWN, Simon. The C4 model for visualising software architecture. 2024. <https://c4model.com/> – Metodika pro hierarchický popis architektury, vhodná pro strojové zpracování.

ARC42. arc42 Template for Software Architecture Documentation. 2024. <https://arc42.org/> – Pragmatická šablona pro strukturování architektonické dokumentace.

4. DALŠÍ ZDROJE (Historický kontext a Procesy)

NATO SCIENCE COMMITTEE. Software Engineering: Report on a conference sponsored by the NATO Science Committee. Garmisch, Germany, 1968. – Historický kontext vzniku disciplíny.

KAUR, Rupinder a SENGUPTA, Jyotsna. Software Process Models and Analysis on Failure of Software Development Projects. In: arXiv preprint arXiv:1306.1068. 2013.

Přílohy

A. Formulář v plném znění

B. Zdrojové kódy výpočetních procedur

C. Baseline AGENTS.md

Baseline verze AGENTS.md (pilot-r1) a její evoluce do pilot-r3 jsou uvedeny v kapitole 5 (obrázky 4.1, 4.4 a 4.7).

D. Využití nástrojů umělé inteligence

Při přípravě práce byl využit nástroj Claude (Anthropic, modely Opus 4 a Sonnet 4) jako AI asistent v následujících oblastech:

Psaní textu práce. Nástroj byl využit pro drafting a úpravu textových částí (kapitoly 1–5), kontrolu konzistence argumentace a LaTeX formátování. Autor formuloval záměr a strukturu každé sekce; AI asistent navrhoval formulace, které autor revidoval a upravoval. Výzkumný design, argumentace a finální znění jsou autorovy vlastní.

Vyhledávání a shrnutí literatury. Nástroj byl využit k identifikaci relevantních zdrojů a shrnutí obsahu papers. Autor ověřoval relevanci a správnost každého zdroje před zařazením do textu; citace odkazují výhradně na zdroje přečtené a posouzené autorem.

Diagnostická analýza v experimentu. V kroku Diagnóza iteračního cyklu (sekce 3.3.4) byl nástroj využit jako analytický asistent: výzkumník popisoval naměřené výsledky, AI asistent navrhoval možné příčiny selhání a opravy instrukcí na základě literatury. Finální rozhodnutí o každé změně `AGENTS.md` bylo vždy na autorovi.

Implementace měřicí infrastruktury. Experimentální skripty (`new-run.ts`, `analyze-run.ts`, `judge.ts`) byly implementovány s asistencí AI nástroje na základě autorova návrhu specifikace metrik. Autor navrhl architekturu, definoval požadované výstupy a provedl validaci správnosti měření (příloha D).

AI nástroj nepouštěl experimenty, nevyhodnocoval výsledky a nerozhodoval o směru výzkumu. Za celý obsah práce nese plnou odpovědnost autor.